Quick Start Guide

Renesas System Release Package

RZ Common, SBC, EVK & RDK

Quick Start Guide

Renesas RZ Family
RZ G/V Series

All information contained in these materials, including products and product specifications, represents information on the product at the time of publication and is subject to change by Renesas Electronics Corp. without notice. Please review the latest information published by Renesas Electronics Corp. through various means, including the Renesas Electronics Corp. website (http://www.renesas.com).

Renesas Electronics
www.renesas.com

Rev.3.30 Sep.11.26

Notice

1. Descriptions of circuits, software and other related information in this document are provided only to illustrate the operation of semiconductor products and application examples. You are fully responsible for the incorporation or any other use of the circuits, software, and information in the design of your product or system. Renesas Electronics disclaims any and all liability for any losses and damages incurred by you or third parties arising from the use of these circuits, software, or information.
2. Renesas Electronics hereby expressly disclaims any warranties against and liability for infringement or any other claims involving patents, copyrights, or other intellectual property rights of third parties, by or arising from the use of Renesas Electronics products or technical information described in this document, including but not limited to, the product data, drawings, charts, programs, algorithms, and application examples.
3. No license, express, implied or otherwise, is granted hereby under any patents, copyrights or other intellectual property rights of Renesas Electronics or others.
4. You shall be responsible for determining what licenses are required from any third parties, and obtaining such licenses for the lawful import, export, manufacture, sales, utilization, distribution or other disposal of any products incorporating Renesas Electronics products, if required.
5. You shall not alter, modify, copy, or reverse engineer any Renesas Electronics product, whether in whole or in part. Renesas Electronics disclaims any and all liability for any losses or damages incurred by you or third parties arising from such alteration, modification, copying or reverse engineering.
6. Renesas Electronics products are classified according to the following two quality grades: “Standard” and “High Quality”. The intended applications for each Renesas Electronics product depends on the product’s quality grade, as indicated below.
   "Standard": Computers; office equipment; communications equipment; test and measurement equipment; audio and visual equipment; home electronic appliances; machine tools; personal electronic equipment; industrial robots; etc.
   "High Quality": Transportation equipment (automobiles, trains, ships, etc.); traffic control (traffic lights); large-scale communication equipment; key financial terminal systems; safety control equipment; etc.

   Unless expressly designated as a high reliability product or a product for harsh environments in a Renesas Electronics data sheet or other Renesas Electronics document, Renesas Electronics products are not intended or authorized for use in products or systems that may pose a direct threat to human life or bodily injury (artificial life support devices or systems; surgical implantations; etc.), or may cause serious property damage (space system; undersea repeaters; nuclear power control systems; aircraft control systems; key plant systems; military equipment; etc.). Renesas Electronics disclaims any and all liability for any damages or losses incurred by you or any third parties arising from the use of any Renesas Electronics product that is inconsistent with any Renesas Electronics data sheet, user’s manual or other Renesas Electronics document.
7. No semiconductor product is absolutely secure. Notwithstanding any security measures or features that may be implemented in Renesas Electronics hardware or software products, Renesas Electronics shall have absolutely no liability arising out of any vulnerability or security breach, including but not limited to any unauthorized access to or use of a Renesas Electronics product or a system that uses a Renesas Electronics product. RENESAS ELECTRONICS DOES NOT WARRANT OR GUARANTEE THAT RENESAS ELECTRONICS PRODUCTS, OR ANY SYSTEMS CREATED USING RENESAS ELECTRONICS PRODUCTS WILL BE INVULNERABLE OR FREE FROM CORRUPTION, ATTACK, VIRUSES, INTERFERENCE, HACKING, DATA LOSS OR THEFT, OR OTHER SECURITY INTRUSION (“Vulnerability Issues”). RENESAS ELECTRONICS DISCLAIMS ANY AND ALL RESPONSIBILITY OR LIABILITY ARISING FROM OR RELATED TO ANY VULNERABILITY ISSUES. FURTHERMORE, TO THE EXTENT PERMITTED BY APPLICABLE LAW, RENESAS ELECTRONICS DISCLAIMS ANY AND ALL WARRANTIES, EXPRESS OR IMPLIED, WITH RESPECT TO THIS DOCUMENT AND ANY RELATED OR ACCOMPANYING SOFTWARE OR HARDWARE, INCLUDING BUT NOT LIMITED TO THE IMPLIED WARRANTIES OF MERCHANTABILITY, OR FITNESS FOR A PARTICULAR PURPOSE.
8. When using Renesas Electronics products, refer to the latest product information (data sheets, user’s manuals, application notes, “General Notes for Handling and Using Semiconductor Devices” in the reliability handbook, etc.), and ensure that usage conditions are within the ranges specified by Renesas Electronics with respect to maximum ratings, operating power supply voltage range, heat dissipation characteristics, installation, etc. Renesas Electronics disclaims any and all liability for any malfunctions, failure or accident arising out of the use of Renesas Electronics products outside of such specified ranges.
9. Although Renesas Electronics endeavors to improve the quality and reliability of Renesas Electronics products, semiconductor products have specific characteristics, such as the occurrence of failure at a certain rate and malfunctions under certain use conditions. Unless designated as a high reliability product or a product for harsh environments in a Renesas Electronics data sheet or other Renesas Electronics document, Renesas Electronics products are not subject to radiation resistance design. You are responsible for implementing safety measures to guard against the possibility of bodily injury, injury or damage caused by fire, and/or danger to the public in the event of a failure or malfunction of Renesas Electronics products, such as safety design for hardware and software, including but not limited to redundancy, fire control and malfunction prevention, appropriate treatment for aging degradation or any other appropriate measures. Because the evaluation of microcomputer software alone is very difficult and impractical, you are responsible for evaluating the safety of the final products or systems manufactured by you.
10. Please contact a Renesas Electronics sales office for details as to environmental matters such as the environmental compatibility of each Renesas Electronics product. You are responsible for carefully and sufficiently investigating applicable laws and regulations that regulate the inclusion or use of controlled substances, including without limitation, the EU RoHS Directive, and using Renesas Electronics products in compliance with all these applicable laws and regulations. Renesas Electronics disclaims any and all liability for damages or losses occurring as a result of your noncompliance with applicable laws and regulations.
11. Renesas Electronics products and technologies shall not be used for or incorporated into any products or systems whose manufacture, use, or sale is prohibited under any applicable domestic or foreign laws or regulations. You shall comply with any applicable export control laws and regulations promulgated and administered by the governments of any countries asserting jurisdiction over the parties or transactions.
12. It is the responsibility of the buyer or distributor of Renesas Electronics products, or any other party who distributes, disposes of, or otherwise sells or transfers the product to a third party, to notify such third party in advance of the contents and conditions set forth in this document.
13. This document shall not be reprinted, reproduced or duplicated in any form, in whole or in part, without prior written consent of Renesas Electronics.
14. Please contact a Renesas Electronics sales office if you have any questions regarding the information contained in this document or Renesas Electronics products.

(Note1) “Renesas Electronics” as used in this document means Renesas Electronics Corporation and also includes its directly or indirectly controlled subsidiaries.

(Note2) “Renesas Electronics product(s)” means any product developed or manufactured by or for Renesas Electronics.

(Rev.5.0-1 October 2020)

Corporate Headquarters

TOYOSU FORESIA, 3-2-24 Toyosu,
Koto-ku, Tokyo 135-0061, Japan
www.renesas.com

Contact information

For further information on a product, technology, the most up-to-date version of a document, or your nearest sales office, please visit:
www.renesas.com/contact/

Trademarks

Renesas and the Renesas logo are trademarks of Renesas Electronics Corporation. All trademarks and registered trademarks are the property of their respective owners.

©2026 Renesas Electronics Corporation. All rights reserved.

Trademarks (continued)

For the “Cortex” notation, it is used as follows;
— Arm® Cortex®-A55
— Arm® Cortex®-M33
Note that after this page, they may be noted as Cortex-A55 and Cortex-M33 respectively.

Examples of trademark or registered trademark used in the RZ/G2L SMARC Module Board RTK9744L23C01000BE User's Manual: Hardware;
CoreSight™: CoreSight is a trademark of Arm Limited.
MIPI®: MIPI is a registered trademark of MIPI Alliance, Inc.
eMMC™: eMMC is a trademark of MultiMediaCard Association.

Note that in each section of the Manual, trademark notation of ® and TM may be omitted.
All other trademarks and registered trademarks are the property of their respective owners.

©2026 Renesas Electronics Corporation. All rights reserved.

General Precautions in the Handling of Microprocessing Unit and Microcontroller Unit Products

The following usage notes are applicable to all Microprocessing unit and Microcontroller unit products from Renesas. For detailed usage notes on the products covered by this document, refer to the relevant sections of the document as well as any technical updates that have been issued for the products.

1. Precaution against Electrostatic Discharge (ESD)
   A strong electrical field, when exposed to a CMOS device, can cause destruction of the gate oxide and ultimately degrade the device operation. Steps must be taken to stop the generation of static electricity as much as possible, and quickly dissipate it when it occurs. Environmental control must be adequate. When it is dry, a humidifier should be used. This is recommended to avoid using insulators that can easily build up static electricity. Semiconductor devices must be stored and transported in an anti-static container, static shielding bag or conductive material. All test and measurement tools including work benches and floors must be grounded. The operator must also be grounded using a wrist strap. Semiconductor devices must not be touched with bare hands. Similar precautions must be taken for printed circuit boards with mounted semiconductor devices.
2. Processing at power-on
   The state of the product is undefined at the time when power is supplied. The states of internal circuits in the LSI are indeterminate and the states of register settings and pins are undefined at the time when power is supplied. In a finished product where the reset signal is applied to the external reset pin, the states of pins are not guaranteed from the time when power is supplied until the reset process is completed. In a similar way, the states of pins in a product that is reset by an on-chip power-on reset function are not guaranteed from the time when power is supplied until the power reaches the level at which resetting is specified.
3. Input of signal during power-off state
   Do not input signals or an I/O pull-up power supply while the device is powered off. The current injection that results from input of such a signal or I/O pull-up power supply may cause malfunction and the abnormal current that passes in the device at this time may cause degradation of internal elements. Follow the guideline for input signal during power-off state as described in your product documentation.
4. Handling of unused pins
   Handle unused pins in accordance with the directions given under handling of unused pins in the manual. The input pins of CMOS products are generally in the high-impedance state. In operation with an unused pin in the open-circuit state, extra electromagnetic noise is induced in the vicinity of the LSI, an associated shoot-through current flows internally, and malfunctions occur due to the false recognition of the pin state as an input signal become possible.
5. Clock signals
   After applying a reset, only release the reset line after the operating clock signal becomes stable. When switching the clock signal during program execution, wait until the target clock signal is stabilized. When the clock signal is generated with an external resonator or from an external oscillator during a reset, ensure that the reset line is only released after full stabilization of the clock signal. Additionally, when switching to a clock signal produced with an external resonator or by an external oscillator while program execution is in progress, wait until the target clock signal is stable.
6. Voltage application waveform at input pin
   Waveform distortion due to input noise or a reflected wave may cause malfunction. If the input of the CMOS device stays in the area between V_{IL} (Max.) and V_{IH} (Min.) due to noise, for example, the device may malfunction. Take care to prevent chattering noise from entering the device when the input level is fixed, and also in the transition period when the input level passes through the area between V_{IL} (Max.) and V_{IH} (Min.).
7. Prohibition of access to reserved addresses
   Access to reserved addresses is prohibited. The reserved addresses are provided for possible future expansion of functions. Do not access these addresses as the correct operation of the LSI is not guaranteed.
8. Differences between products
   Before changing from one product to another, for example to a product with a different part number, confirm that the change will not lead to problems. The characteristics of a microprocessing unit or microcontroller unit products in the same group but having a different part number might differ in terms of internal memory capacity, layout pattern, and other factors, which can affect the ranges of electrical characteristics, such as characteristic values, operating margins, immunity to noise, and amount of radiated noise. When changing to a product with a different part number, implement a system-evaluation test for the given product.

Renesas RZ Family

Renesas System Release Package

Introduction

This user manual describes the unified system release package. The system release package contains supported hardware and software.

The result is a consistent experience across the different platforms. This streamlines the development effort for user applications.

Package Contents

The system release package contains the following:

- Multiple Images that are geared to general baseline use cases.
- Yocto build scripts.
- Host-side tools.
- Environmental files.
- SDKs for all images
- Documentation, which includes a user manual and copyright & license information

Features

The following are the general features of the system release package.

- Architected to support multiple platforms with the same image and tools over time.
- Common frameworks
- Open-source packages using GPLv2 and GPLv3 packages
- Carefully considered base images that allow for a quick starting point to build a product.
- Complete set of features working out of the box.
- Seamless out-of-box experience.
- Automated Yocto build scripts that can rebuild the entire package with only a few commands.
- Host tools to flash the firmware in multiple processes.
- Tools supporting both Linux and Windows workflows.
- Docker-friendly build scripts.
- Extensive documentation covering the hardware, software, and application development and deployment.

Contents

Introduction 1

Package Contents 1

Features 1

Glossary 5

1. Overview 7
1.1 Supported Distributions 7
1.1.1 Yocto Images 7
1.1.2 Renesas Custom Images 7
1.1.3 Ubuntu Images 8
1.2 Supported Platforms 8

2. Quick Start 10
2.1 SD/MMC Card Flashing 10
2.2 Universal Flash 11
2.2.1 Prerequisites 11
2.2.2 Usage and Flashing Operations 14
2.2.3 Flashing the Bootloader 18
2.2.4 Flashing the uLoad-bootloader 19
2.2.5 Flashing the SD Card Image 22
2.3 RZ/G2L-SBC 26
2.3.1 Overview of Connectors 26
2.3.2 Hardware Requirements 30
2.3.3 Essential Hardware Setup 31
2.3.4 Complete Hardware Setup 32
2.3.5 Booting 33
2.3.6 Known Hardware and Functional Limitations on RZ/G2L-SBC 33
2.4 RS-G2L100 35
2.4.1 Overview of Connectors 36
2.4.2 Hardware Requirements 36
2.4.3 Essential Hardware Setup and Booting 37
2.4.4 Complete Hardware Setup 40
2.4.5 Booting 40
2.4.6 Known Hardware and Functional Limitations 42
2.4.7 Solutions and Workarounds 43
2.5 RZ/G2L-EVK and RZ/V2L-EVK 44
2.5.1 Overview of Connectors 45

2.5.2 Hardware Requirements 45
2.5.3 Essential Hardware Setup and Booting 46
2.5.4 Complete Hardware Setup 49
2.5.5 Booting 49
2.5.6 Known Hardware and Functional Limitations 52
2.6 RZ/V2H-EVK 52
2.6.1 Overview of Connectors 53
2.6.2 Hardware Requirements 53
2.6.3 Essential Hardware Setup and Booting 54
2.6.4 Complete Hardware Setup 56
2.6.5 Booting 57
2.6.6 Known Hardware and Functional Limitations 59
2.7 RZ/V2H-RDK 60
2.7.1 Overview of Connectors 60
2.7.2 Hardware Requirements 60
2.7.3 Essential Hardware Setup and Booting 61
2.7.4 Complete Hardware Setup 63
2.7.5 Booting 64
2.7.6 Known Hardware and Functional Limitations 64
2.8 IMDT V2H-SBC 64
2.8.1 Overview of Connectors 65
2.8.2 Hardware Requirements 66
2.8.3 Essential Hardware Setup and Booting 67
2.8.4 Complete Hardware Setup 69
2.8.5 Booting 69
2.8.6 Known Hardware and Functional Limitations 71
2.9 Boot Notice 71

3. Appendix 73
3.1 Factory Firmware Flashing Using Serial Downloader (SCIF) Mode 73
3.1.1 RZ/G2L-SBC 73
3.1.2 RS-G2L100 74
3.1.3 RZ/G2L-EVK and RZ/V2L-EVK 74
3.1.4 RZ/V2H-EVK 75
3.1.5 RZ/V2H-RDK 76
3.1.6 IMDT V2H-SBC 78
3.2 Boot Mode Reference (Non-SCIF) 78
3.2.1 RS-G2L100 78
3.2.2 RZ/G2L-EVK & RZ/V2L-EVK 79
3.2.3 RZ/V2H-EVK 80

3.2.4 RZV2H-RDK 81
3.2.5 IMDT V2H-SBC 82
3.3 Prepare the eMMC root filesystem 82
3.4 Auto-managed U-boot Environment (imported each boot) 85
3.5 How To Get the Console After Bootup 86

Revision History 87

Glossary

Terms	Description
802.11 - Wi-Fi	The technical name of the standard specification for Wi-Fi is 802.11. This is also the working group that develops and maintains the standards for Wi-Fi that everyone conforms to.
ADC – Analog to digital converter	A hardware unit that converts an input analog signal to a digital value by measuring its immediate voltage at a fixed resolution.
BSP – Board Support Package	BSP is an essential software package that has bootloaders, Linux kernel, a minimal user space and programming tools, allowing the device to boot. This core software allows the system to boot into an operating system, enables all the features and allows application development.
CAN – Controller area network	This is a standardized communication protocol used widely on automotive and aerospace systems. It connects various ECU's known as nodes and uses two wires / lines as a pair carrying differential signals. This method of signaling allows long length cables to interface different systems on the machine with reliable signals. The CAN protocol has multiple specifications and is an ISO standard. It supports flexible data rates reaching as high as 8Mbps. Most automobiles have CAN networks in them, and it is a part of OBD-2 specification which is mandatory law in most of the world for automotive machines like cars.
DAC – Digital to analog converter	A hardware unit that takes digital value and exerts a corresponding analog voltage on an output line.
Firmware	For the scope of this document, the term 'firmware' refers to the low-level software that runs before an OS takes over. This includes arm trust zone, optee & u-boot at the very least. It also refers to the standalone binaries that run on the embedded real-time core like the CM33.
FCONF – Firmware Configuration Framework	FCONF stands for Firmware Configuration Framework, used primarily in Arm Trusted Firmware-A (TF-A). It provides a structured way to pass configuration data (such as hardware descriptions, SoC parameters, memory layout, boot parameters, etc.) into different firmware stages (BL1/BL2/BL31).
I²C - Inter Integrated circuit protocol:	This is a communication protocol used to implement digital communication between two devices (chips / board) using only two wires. It is a standardized specification and is used widely to implement low to medium data rate data transfers both among devices on the same circuit board as well as external add on peripheral boards. I²C can be implemented across a few meters in distance. I²C is half duplex meaning only one device can communicate at a time. Speeds range from 100 Kbps to 3Mbps while 100 / 400 Kbps are the typical operating mode. The other major advantage of this protocol is that it allows many devices to be on the same two lines reducing the cost of the interfacing. This is ideal when there are many devices like sensors that transfer limited amounts of data periodically. I²C can support up to 127 independent directly addressable devices on the same channel.
IEEE- Institute of Electrical and Electronics Engineers	IEEE is the world's largest technical professional organization dedicated to advancing technology for the benefit of humanity. It is a major technical organization covering vast fields of engineering and a major standards organization.
MCU – Micro controller unit	A micro controller unit is a self-contained unit that has the core processing as well as core memory within the same device. It often contains the core software programmed into the chip itself. This allows the device to start executing with minimal external devices / circuitry. Some microcontrollers can be powered on a mere breadboard.
MPU – Micro processing unit	An MPU is a processing unit: a CPU that contains only the processing core and interfaces for external peripherals. A microprocessor is usually a powerful CPU in its class. However, it requires a very large number of external circuitries to achieve its functionality like external memory, disk drives, etc.
PMIC – Power management IC	This is a specific chip on the board that manages multiple power supply lines at various levels. It manages the respective supplies along with sequences which control power on and power off cycles.

SBC – Single board computer	It is a standard term that means a tiny computer in the form factor of a single circuit board usually just inches in area. This board is self-sufficient in every way and can give you a usable computer with just a power supply, keyboard, mouse, and display.
EVK – Evaluation Kit	It is a reference hardware platform provided by the silicon vendor. EVKs are designed to help developers evaluate SoC features, validate software, test peripherals, and accelerate early development before moving to custom hardware.
RDK – Robot Development Kit	It is a hardware + software platform intended for robotics development, typically providing motor control, sensors, compute modules, and pre-integrated frameworks for rapid prototyping and testing robotic applications.
SiP – System in Package	SiP is a device where multiple silicon IP's are combined to form a single device. It is one of the densest chips where the external devices like flash memory, DDR RAM and even Wi-Fi module are all packaged into a single chip. These are used in very niche application that require ultra small size and low thermal requirement.
SoC- System on Chip	A system on chip is a complete hardware platform packaged on to a single chip. It contains the CPU, internal fast memory, interrupt controllers, pin controllers, ROM memory, and a few other peripherals and sensors; all packaged into the same IC. An SoC despite the high level of integration does not necessarily power on and run by itself. Microcontrollers are often independent SoC's that can work on their own. However, SoC's often combine MPU's and MCU's into the same chip. This allows very powerful systems to be built in a compact form factor but requires external supporting peripherals like DDR RAM and flash memory and power management IC's.
SPI - Serial Peripheral interface	SPI is another standard interface used to interface other devices on the board or attaching peripheral boards. It specifies 3 wires / lines to achieve fast full duplex data transfer. Two devices can send / receive data at the same time in this protocol. The protocol is also a high-speed protocol where typical operating speeds start at 5Mbps and go over 50Mbps. This high speed allows interfacing high speed devices like memory, Wi-Fi, subsystems made of independent microcontrollers, etc. While only 3 lines are needed to interface two devices, a fourth line is used as a device selector allowing multiple devices to share the same interface. However, only two devices may communicate at a time.

1. Overview

The Renesas System Release Package is a unified software package that aims to provide an easy-to-use yet comprehensive software platform for the Renesas RZ series of SoC-based boards. It aims to provide fully functional base images for supported reference designs, along with easy-to-use development and programming tools that allow the user to quickly get started on their application development. This package aims to provide a standardized and familiar workflow for a similar experience across a variety of Renesas RZ SoC-based product platforms.

This package provides comprehensive documentation, quick-start guides, multiple Linux-based distribution images, automated tools and scripts, and an ongoing expansion of supported products.

1.1 Supported Distributions

The System Release package supports a set of both Yocto images and custom images to enable the user to start quickly on their embedded end application. The large collection of images in prebuilt format provides a wide set of capabilities. This release focuses on Yocto images.

1.1.1 Yocto Images

This section lists the standard Yocto images, offering a variety of configurations that cater to different embedded use cases. From a minimal bootable environment to fully graphical systems, these images provide the essential building blocks for embedded Linux development.

Table 1. Yocto images

Distribution	Image file	Version	Description
Yocto minimal	core-image-minimal	styhead-5.1.4	A basic image that contains the minimal set of components required to boot the device. It focuses on essential system functions without extra tools or features.
Yocto BSP	core-image-bsp	styhead-5.1.4	Extends core-image-minimal with additional utilities and tools, providing a lightweight environment for system validation, hardware diagnostics, and basic development.
Yocto weston	core-image-weston	styhead-5.1.4	A standard graphical image with Wayland and Weston support for embedded GUI applications.

1.1.2 Renesas Custom Images

This section presents Renesas-specific custom images, which are customized and optimized for Renesas products. These images offer specialized features, including fast booting and tailored environments for both graphical and CLI-based applications.

Table 2. Renesas custom images

Distribution	Image file	Version	Description
Renesas CLI Base	renesas-core-image-cli	styhead-5.1.4	Based on core-image-bsp, this image offers a CLI environment for Renesas hardware development without graphical interfaces. Besides the useful tools inherited from the core-image-bsp, this image also contains new packages for SBC (Single Board Computer) development. For example, package managers (apt, dpgk), network utilities for Bluetooth and Wi-Fi.
Renesas Quickboot CLI	renesas-quickboot-cli	styhead-5.1.4	This image has the same system functionality as the renesas-core-image-cli but with Quickboot enabled, allowing for faster boot times and efficient system validation in a CLI environment.

Renesas Weston (Qt6)	renesas-core-image-weston	styhead-5.1.4	Renesas customized core image based on the core-image-weston, with Qt6 framework support (no QT demo apps included). This image offers a full graphical environment for Renesas hardware development and all the useful tools from the renesas-core-image-cli.
Renesas Quickboot Wayland	renesas-quickboot-wayland	styhead-5.1.4	This image has the same system functionality as the renesas-core-image-weston but with Quickboot enabled, allowing for faster boot times and efficient system validation in a graphical environment.

Note: Quickboot is a trade term that refers to the specific optimizations that are performed to achieve ultra-low start-up times in specific images. Depending on the board architecture, the startup time can be as low as 2s. While there is no assurance of the startup time in these images for every platform, these images are the most optimized on our platforms.

1.1.3 Ubuntu Images

This section presents custom Ubuntu-based images tailored for embedded systems, offering a variety of configurations to suit both headless and graphical environments. These images are optimized for performance and ease of use, providing a solid foundation for deploying embedded applications on Renesas platforms.

Table 3. Renesas Ubuntu images

Distribution	Image file	Version	Description
Ubuntu Core	ubuntu-core-image	ubuntu-base-24.04	A minimal, headless Ubuntu image tailored for embedded systems.
Ubuntu LXDE	ubuntu-lxde-image	ubuntu-base-24.04	A lightweight Ubuntu image featuring the LXDE desktop environment, providing a graphical interface while maintaining low resource consumption. This image also includes Qt framework support for GUI development.

1.2 Supported Platforms

Table 4. Supported Platforms

Platform	SoC	OPN	Description
RZ/G2L-SBC	RZ/G2L	US157-G2LSBCPOCZ	RZ/G2L-based Pi-compatible SBC.
Geniatech RS-G2L100	RZ/G2L	R9A07G044L24GBG#BC0	Geniatech AHAURA RS-G2L100
RZ/G2L-EVK	RZ/G2L	RTK9744L23S01000BE	Evaluation Board Kit for RZ/G2L MPU

RZ/V2L-EVK	RZ/V2L	RTK9754L23S01000BE	Evaluation Board Kit for RZ/V2L MPU
RZ/V2H-EVK	RZ/V2H	R9A09G057H48GBG#AC0 R9A09G057H48GBG#BC0	Evaluation Board Kit for RZ/V2H MPU
RZ/V2H-RDK	RZ/V2H	WS125-V2HRDKREFZ	RZ/V2H Robot Development Kit (RDK): AI-powered robotics kit for autonomous robots.
IMDT V2H SBC	RZ/V2H	N/A	Compact AI SBC board with RZ/V2H and rich connectivity.

Note: Boards without a product number (currently not for sale) are marked as N/A.

2. Quick Start

This section describes how to quickly get set up and start running the supported platforms with this release. The following are the essential steps for an SD/MMC card-based boot:

1. Select an image from the list of available images from the Supported Distributions section
2. Prepare an SD MMC card that has the image programmed onto it.
3. Prepare the hardware with power and a debug UART interface. Displaying the connection to one of the HDMI interfaces is highly recommended, but not essential.
4. Program the firmware using the appropriate scripts and process in the 'host/tools' directory of the package.
5. Boot normally with the SD/MMC card.

2.1 SD/MMC Card Flashing

The Linux bootable SD card creation is a very simple process. The idea is to use any filesystem imaging tool (etcher) to burn the required image's '.wic' file (core-image-weston.wic, for example) located in the 'target/images' directory of the release to the SD/-MMC card. We recommend installing Balena etcher, which is available for Linux, macOSm, and Windows.

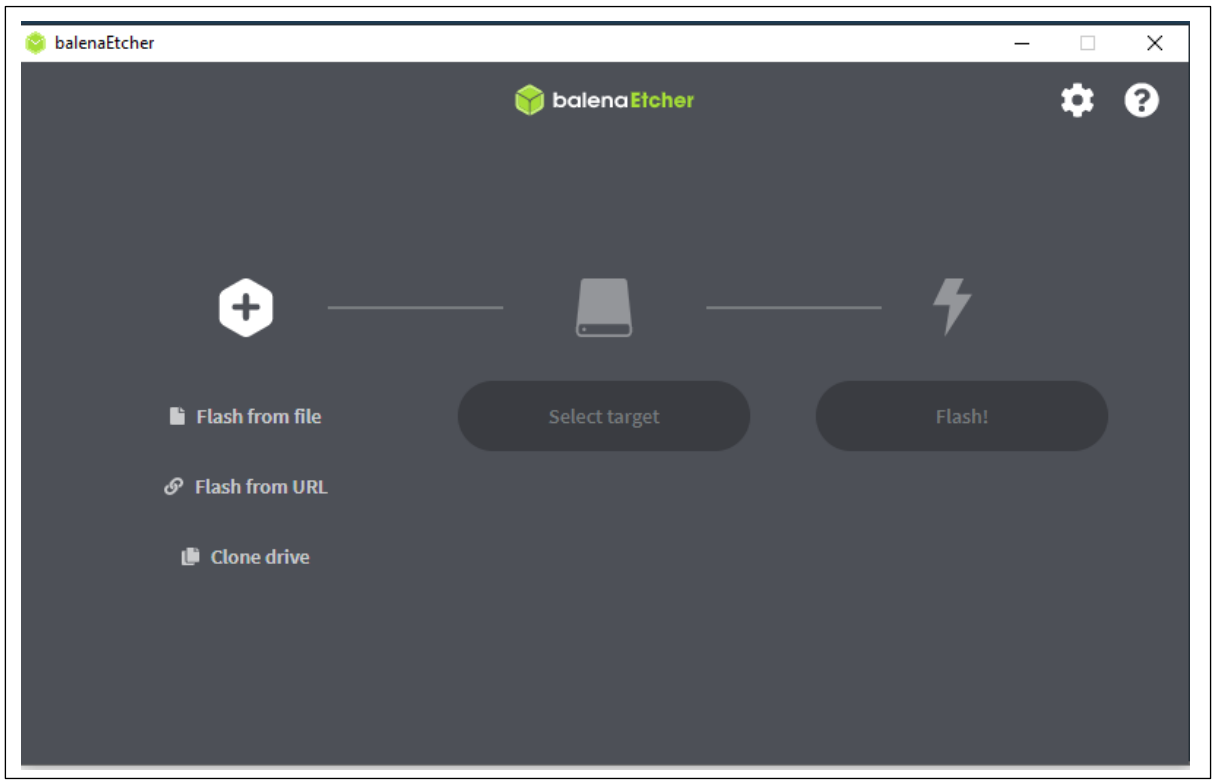


Figure 1. Balena Etcher UI

Steps:

1. Select "Flash from File".
2. In the pop-up window, navigate to your release and select one of the chosen image files (core-image-weston.wic).
3. Then click on 'Select target,' and it will list all available devices.
4. Select your SD MMC card.
   Be mindful not to select your primary laptop/desktop hard drive.
5. Select 'Flash'.
6. When flashing is completed, it will automatically dismount the SD MMC card device.

7. Insert the SD card into the SD/MMC card slot on the RZ board.

2.2 Universal Flash

This section explains how to program and flash various firmware onto Renesas boards. It covers firmware components, prerequisites, hardware setup for each board, and usage of the universal flashing script for seamless flashing workflows.

The universal flashing script (universal_flash.py) is included in the release package under the **host/tools** folder.

This package contains only the following firmware components:

Table 5. Firmware description

Module	Binary	Stack layer	Description
ROM code	N/A	BL1	This is the internal ROM code that the ARM Cortex SoC's primary core executes at POR.
Flash writer	Flash_Writer_SCIF_<board>.mot	BL2	This is meant for serial load in factory environments, which is directly loaded onto the SRAM by the BL1 (ROM code) through UART SCIF0. It is then executed to acquire another image on UART SCIF0 to directly flash onto qspi or emmc into the boot sector. It provides a command-based ui.
Arm Trusted Firmware-A	bl2-rz-cmn.bin bl31-rz-cmn.bin <board>.dtb	BL2 and BL31	Minimal Trusted Firmware-A implementation without a device tree. The flashing script will combine bl2-rz-cmn.bin with the device tree dynamically during flashing. It comes in only .bin format: • .bin – for raw flashing for native in-system flashing
U-Boot (BL33)	u-boot-nodtb-rz-cmn.bin <board>.dtb	BL33	U-Boot (nodtb) binary and matching device tree. The flashing script combines these into the FIP.
Board Identification	<board>-platform-settings.bin		This binary stores key platform settings for Renesas boards, like model IDs, revisions, memory locations, and image sizes, enabling firmware and bootloaders to identify hardware and locate boot components efficiently during startup or flashing.

Note: This release does **not** ship a prebuilt FIP. Instead, the flashing script will automatically build a valid FIP image at flash time by combining:

- bl31-rz-cmn.bin
- u-boot-nodtb-rz-cmn.bin
- <board>.dtb

It also dynamically merges bl2-rz-cmn.bin with <board>.dtb to create the BL2 binary that is flashed to the boot sector.

2.2.1 Prerequisites

Before flashing any images, ensure the following system requirements are met on the host PC, and that necessary files and tools are available.

- Operating System:
  - Linux (Ubuntu 20.04 or newer recommended)
  - Windows 10 or newer
- Software Requirements:
  - Python 3.8 or later
  - Firmware release package with images and tools
- Hardware requirements:
  - Required cables: USB, UART debug cables
  - SD card (8GB or larger)

2.2.1.1 Linux Setup

Follow these steps to prepare a Linux host:

1. Install Python, Binutils, and build tools

```
renesas@builder-pc:~# sudo apt update
renesas@builder-pc:~# sudo apt install python3 python3-pip build-essential
android-tools-fastboot
```

   - python3, python3-pip: Required to run host scripts
   - build-essential (optional): Installs gcc, g++, and make for rebuilding firmware
   - android-tools-fastboot: Install to get the fastboot binary

2. Install Python Dependencies

   It is recommended to use a virtual environment with any supported Python version (3.10, 3.11, or 3.12).

   Example for Python 3.12

```
renesas@builder-pc:~/rz-cmn-srp/host/tools# sudo apt install python3.12-venv
renesas@builder-pc:~/rz-cmn-srp/host/tools# python3 -m venv .venv
renesas@builder-pc:~/rz-cmn-srp/host/tools# source .venv/bin/activate
```

   If the distribution uses a different Python 3 version (for example, 3.10 or 3.11), replace 3.12 with the appropriate version.

   After activating the virtual environment, install the required tools using requirements.txt.

```
renesas@builder-pc:~# cd <path/to/the/package>/host/tools/
renesas@builder-pc:~/rz-cmn-srp/host/tools$ pip3 install -r requirements.txt
```

2.2.1.2 Windows Setup

Follow these steps to prepare a Windows host:

1. Install Python3
   - Download and install Python 3 from python.org
   - During installation, enable “Add Python to environment variables.”
2. If pip is missing, repair your Python installation or download get-pip.py and run:

```
PS C:\Users\renesas> py get-pip.py
```

3. Install Python Dependencies: Open one of the following terminals with Administrator privileges:
   - PowerShell
   - Git Bash

   The example below uses PowerShell, but the same applies to other terminals
   - Option 1 – Use **requirements.txt** (recommended)

```
PS C:\Users\renesas> cd <path/to/the/package/host/tools>
PS C:\Users\renesas\rz-cmn-srp\host\tools> py -m pip install -r requirements.txt
```

- Option 2 – Install manually
  - Using the Python launcher:

```
PS C:\Users\renesas> py -m pip install pyserial
PS C:\Users\renesas> py -m pip install tomli
PS C:\Users\renesas> py -m pip install dataclasses       # Only if Python < 3.7
```

  - Or using pip directly (if already in PATH)

```
PS C:\Users\renesas> pip install pyserial
PS C:\Users\renesas> pip install tomli
PS C:\Users\renesas> pip install dataclasses # required only if using Python
versions older than 3.7
```

4. OTG Flashing setup

Fastboot over USB OTG on Windows requires the device's Fastboot / USB-download interface to be bound to WinUSB.

- Put the board into Fastboot (from U-Boot)
  - Connect the board's USB-to-serial to the PC and open Tera Term (115200 8-N-1).
  - Power on and interrupt autoboot to get the U-Boot> prompt.
  - Connect the board's USB OTG port to the PC.
  - At U-Boot, run the following commands to go to OTG download mode

```
=> setenv serial# Renesas_RZ_CMN
=> saveenv
=> fastboot usb 27
```

- Bind WinUSB with Zadig
  - Download and run Zadig (no install required)
  - Click Options → List All Devices.
  - In the dropdown, select the Fastboot/bootloader interface (USB Download gadget).
  - On the right, choose WinUSB → click Install Driver (or Replace Driver).

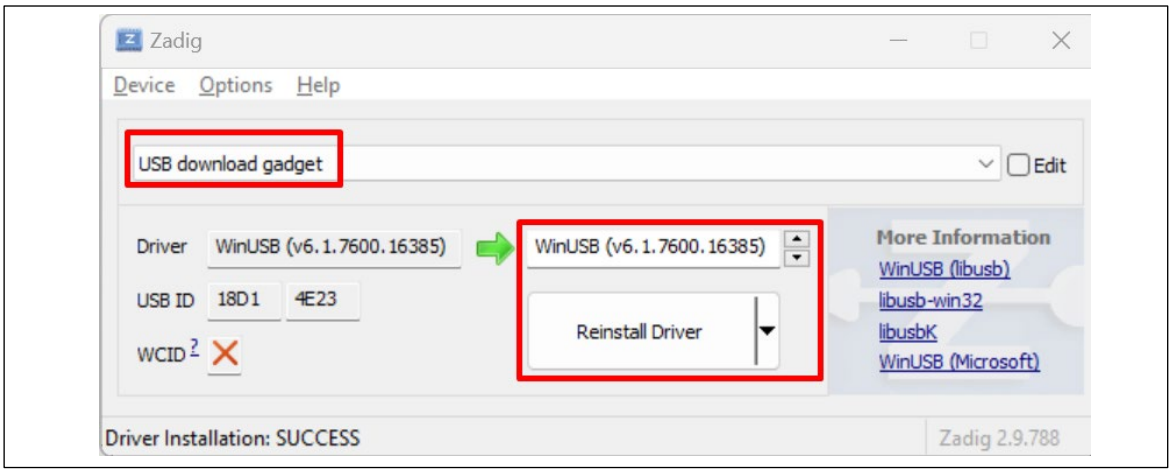


Figure 2. Install WinUSB via Zadig

2.2.2 Usage and Flashing Operations

The universal_flash.py serves as the main entry point for performing all flashing operations. It uses the board configuration defined in flash_images.json to select the appropriate images and procedures.

For this quick start guide, only bootloader flashing using Serial Download (SCIF) mode is required. Other operations supported by the script, such as rootfs flashing over UDP/OTG, are covered in the full user manual.

For this release, use the default image paths included in the package. Overrides in flash_images.json are only required when adding a new board or replacing images.

Table 6. Required Image Locations and Resolution Priority

Component	Expected Location	Purpose	Override (from flash_images.json)
BL2/BL31	• </path/to/yocto/package>/target/images/atf/bl2-rz-cmn.bin • </path/to/yocto/package>/target/images/atf/bl31-rz-cmn.bin	Bootloader stages are responsible for early initialization (BL2) and runtime services (BL31).	Not overridable – always taken from the default location.
BL2 FCONF Device Trees	</path/to/yocto/package>/target/images/atf/fdts/	Configuration device trees for BL2 (DDR, PFC, CPG, etc.).	**atf_fdts** can override with a per-board FCONF DTB
U-Boot (no DTB)	</path/to/yocto/package>/target/images/u-boot/u-boot-nodtb-rz-cmn.bin	Per-board U-Boot device trees. Selected automatically by the flashing script.	Not overridable – always taken from the default location.
U-Boot DTBs	</path/to/yocto/package>/target/images/u-boot/dtbs/	Per-board U-Boot device trees. Selected automatically by the flashing script.	**uboot_dtb** can override and select the exact DTB for the board.
Root filesystem	</path/to/yocto/package>/target/images/(e.g., core-image-weston.wic	Complete root file system image written to SD card or eMMC.	**rootfs** and **rootfs_flash_method** override the image and flashing method (e.g., udp, otg).
Flash writer	</path/to/yocto/package>/target/images/Flash-writer-<board>.mot	Initial loader for XSPI/eMMC flashing.	**flash_writer** must define the correct image for the board
Board identification	• </path/to/yocto/package>/target/images/<board>-platform-settings.srec • </path/to/yocto/package>/target/images/<board>-platform-settings.bin	Per-board platform/ID blob used during flashing.	**board_identification** can override and select the file.

After all required images are placed in their expected locations (and flash_images.json is configured if needed), proceed to run the script.

- **On Linux:** Open a terminal and run the following commands

```
renesas@builder-pc:~$ cd ~/renesas/rz-cmn-srp/host/tools/
renesas@builder-pc:~/renesas/rz-cmn-srp/host/tools/$ python3 universal_flash.py
```

- **On Windows:** Open one of the following terminals with Administrator privileges:
  - PowerShell
  - Git Bash
  - MobaXterm

  Navigate to the host/tools/ directory inside the rz-cmn-srp folder. The example below uses PowerShell, but the same applies to other terminals

```
PS C:\Users\renesas> cd C:\Users\renesas\rz-cmn-srp\host\tools\
PS C:\Users\renesas\rz-cmn-srp\host\tools\> py universal_flash.py
```

Upon execution, the script will present an interactive menu to choose the desired flashing operation.

The table below shows, for each supported board, which IPL flash method the IPL uses to write boot components (xspi or emmc) and which rootfs flash method is used to write the .wic image (udp or otg),, as defined in flash_images.json.

Table 7. Supported IPL and Root filesystem flash methods

Board	SoC/MPU	ipl_flash_method	Default	rootfs_flash_ method	Default
rzg2l-sbc	g2l	xspi	xspi	udp	udp
rzg2l-evk	g2l	xspi, emmc, esd	xspi	udp, otg	otg
rs-g2l100	g2l	xspi	xspi	udp, otg	otg
rzv2l-evk	v2l	xspi, emmc, esd	xspi	udp, otg	otg
rzv2h-evk	v2h	xspi, esd	xspi	udp, otg	otg
rzv2h-rdk	v2h	xspi	xspi	udp	udp
imdt-v2h-sbc	v2h	xspi	xspi	udp, otg	otg

Note:

- IPL flash method: emmc for rzv2h-evk is not supported yet.
- The RZ/G2L-SBC board does not provide a USB OTG port; accordingly, OTG is not supported

2.2.2.1 Flashing Flow

To flash the bootloader using universal_flash.py, follow these steps:

1. Connect the board to the host PC
2. Temporarily power on the board in its current mode so the serial interface can enumerate.
   - If the serial port is not visible on the host PC after connecting the USB cable, power on the board and wait for the serial interface to enumerate. On many boards, the UART-over-USB interface appears only after power is applied (e.g., RZ/G2L-EVK, RZ/V2L-EVK, RZ/V2H-RDK, RZ/V2H-EVK)
3. Run the flashing tool:
   - Start universal_flash.py. For details on how to run it on Windows or Linux, refer to section 2.2.2

4. Select the target board and press Enter.
5. Select the serial port and press Enter.
6. When prompted “Write IPL method:”, answer ‘y’ to display the available flashing methods. If ‘y’ is selected, the following options will be shown:
    - 1 – BootloaderFlash → standard bootloader flashing
    - 2 – Uload Flash – not recommended for quickstart; see the user manual for complete flashing steps.

    Confirm the choice and press Enter.
7. If option 1 is selected:
    - Please power off the board.
    - Set the DIP switches to SCIF download mode.:
    - Power the board back on; now the flashing will begin automatically.
8. When asked “Do you want to write the rootfs? (y/n)”, type n and press Enter.
    - y → to flash the root filesystem
    - n → to skip rootfs flashing
9. If ‘y’ is selected in step 5:
    - Please power off the board.
    - Set the DIP switches to normal boot mode.
    - Power the board back on; now the flashing rootfs will begin automatically; no other steps are needed.

After bootloader/IPL flashing completes successfully, power off the board, restore the DIP switches to the correct normal boot mode for your board, then power on again to use.

If the board is left in SCIF download mode, it will not boot Linux and may return to the serial downloader/XMODEM prompt.

For the correct DIP switch settings, please refer to the section **Factory Firmware Flashing Using Serial Downloader (SCIF) Mode** for SCIF download mode and section **Boot Mode Reference (Non-SCIF)** for normal boot mode.

The diagram below shows this simplified flow.

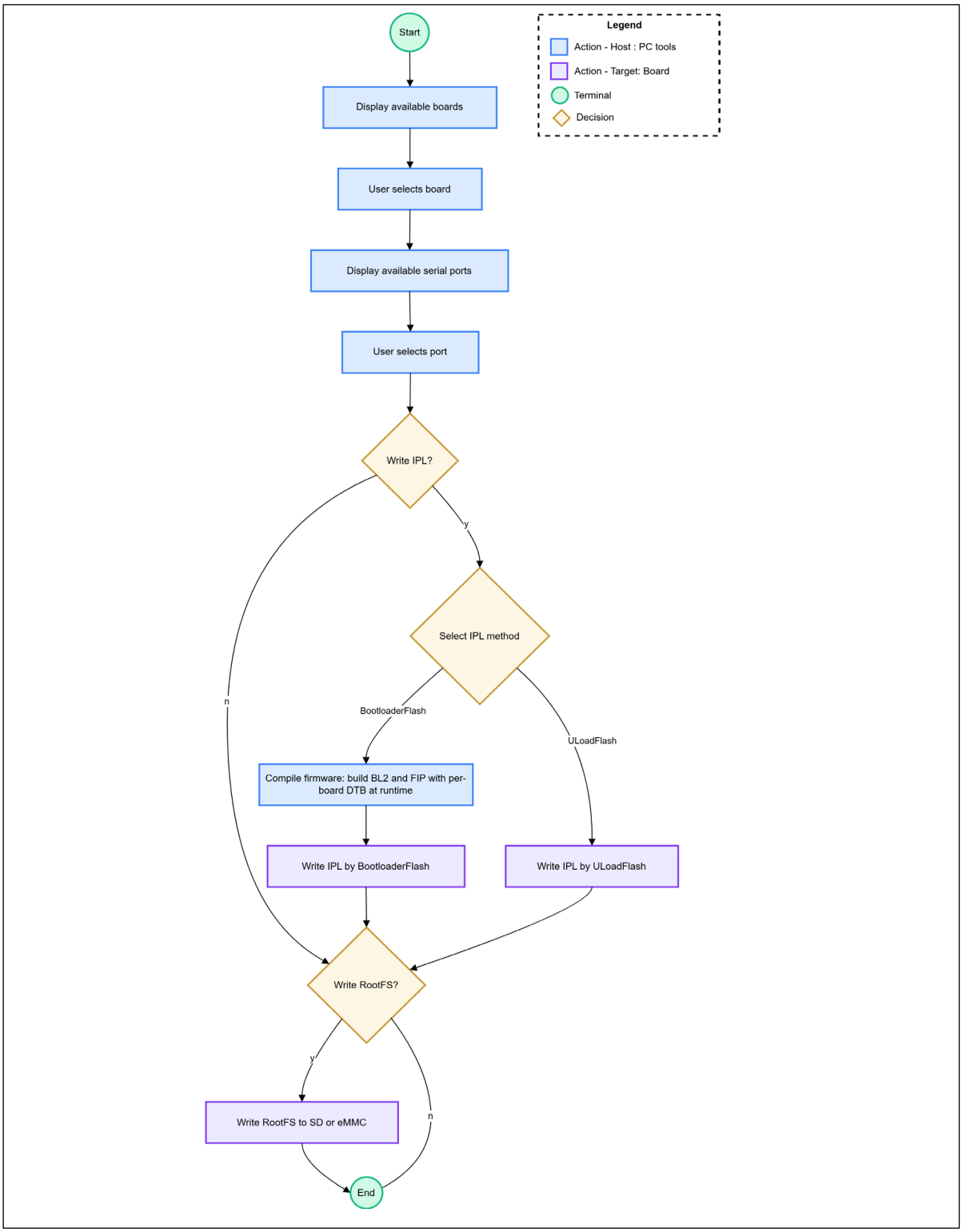


Figure 3. Universal Flashing Script — Interactive Workflow

2.2.3 Flashing the Bootloader

This section describes the dedicated bootloader flashing script, intended for writing the initial bootloader image to the board over a serial interface. It is suitable for first-time provisioning, factory recovery, or focused solely on IPL/bootloader updates.

The universal flashing script (section 2.2.2) is the master, multi-operation tool that handles rootfs and IPL flashing. The bootloader-only script is a streamlined utility that performs bootloader flashing only, with minimal prompts and faster turnaround.

Location:

```
host/tools/bootloader_flasher/bootloader_flash.py
```

2.2.3.1 Hardware Connection

Before starting the flashing process, ensure that the required hardware interfaces are properly connected between the board and the host PC.

- Connect the debug serial port of the board to the host PC.
- Place the board into **SCIF (serial) download mode before flashing**. The exact DIP-switch or jumper settings are **board-specific**.
    - Refer to section **Factory Firmware Flashing Using Serial Downloader (SCIF) Mode** for the required switch positions and steps for the target board.

2.2.3.2 Flashing Procedure

To begin using the bootloader_flash.py, execute it from the host machine:

- **On Linux:** Open a terminal and run the following commands

```
renesas@builder-pc:~$ cd ~/renesas/rz-cmn-srp/host/tools/bootloader_flasher
renesas@builder-pc:~/renesas/rz-cmn-srp/host/tools/bootloader_flasher$ python3
bootloader_flash.py
```

- **On Windows:** Open one of the following terminals with Administrator privileges:
    - PowerShell
    - Git Bash
    - MobaXterm

  Navigate to the host/tools/bootloader_flasher directory inside the rz-cmn-srp folder. The example below uses PowerShell, but the same applies to other terminals

```
PS C:\Users\renesas> cd C:\Users\renesas\rz-cmn-srp\host\tools\bootloader_flasher
PS C:\Users\renesas\rz-cmn-srp\host\tools\bootloader_flasher> py
bootloader_flash.py
```

If no arguments are provided, the script will use the following default values:

- Board name: rzg2l-sbc
- Flash method: xspi
- Serial port: most recently connected port (e.g., COM8 in Windows or /dev/ttyUSB0 in Linux)
- Serial port baud: 115200
- Flash Writer Image: /path/to/universal-scripts/target/images/Flash_Writer_SCIF_rzg2l-sbc.mot
- BL2 Image: /path/to/universal-scripts/target/images/bl2_bp_rzg2l-sbc.srec
- FIP Image: /path/to/universal-scripts/target/images/fip_rzg2l-sbc.srec

- Board identification Image: /path/to/universal-scripts/target/images/rzg2l-sbc-platform-settings.bin

Ensure that these files are present in the current directory before executing the script.

To specify custom file paths or override the defaults, the following arguments can be passed:

- **--board_name:** Board name to flash the bootloader.
- **--flash_method:** Flash method to use (xspi or emmc).
- **--serial_port:** Serial port to use for communication with the board.
- **--serial_port_baud:** Baud rate for the serial port.
- **--image_writer:** Path to the Flash Writer image.
- **--image_bl2:** Path to the BL2 image.
- **--image_fip:** Path to the FIP image.
- **--image_bid:** Path to the board identification image.

Example command:

- **On Linux:**

```
renesas@builder-pc:~/renesas/rz-cmn-srp/host/tools/bootloader_flasher$ py
bootloader_flash.py --board_name rzg2l-evk --flash_method emmc --serial_port
COM11 --serial_port_baud 115200 --image_writer
D:\custom_images\Flash_Writer_SCIF_rzg2l-sbc.mot --image_bl2
D:\custom_images\bl2_bp_rzg2l-sbc.srec --image_fip D:\custom_images\fip-rzg2l-
sbc.srec --image_bid D:\custom_images\rzg2l-evk-platform-settings.bin
```

- **On Windows:**

```
PS C:\Users\renesas\rz-cmn-srp\host\tools\bootloader_flasher> python3
bootloader_flash.py --board_name rzg2l-evk --flash_method emmc --serial_port
/dev/ttyUSB0 --serial_port_baud 115200 --image_writer
/home/renesas/custom_images/Flash_Writer_SCIF_rzg2l-sbc.mot --image_bl2
/home/renesas/custom_images/bl2_bp_rzg2l-sbc.srec --image_fip
/home/renesas/custom_images/fip-rzg2l-sbc.srec --image_bid
/home/renesas/custom_images/rzg2l-evk-platform-settings.bin
```

2.2.4 Flashing the uLoad-bootloader

This section describes the ULoad-bootloader flow for programming the bootloader from the U-Boot console. It supports QSPI/xSPI flashing and is intended for cases where the device can boot to U-Boot and program flash using images stored on the removable media.

Location:

```
host/tools/uload_bootloader/uload_bootloader_flash.py
```

2.2.4.1 Prerequisites

The release does not include prebuilt ULoad images on the SD card. The ULoad flow requires BL2 and FIP artifacts that are rebuilt for the selected board with the correct DTB/FCONF and configuration.

Run the firmware_compile.py script to generate these artifacts, then copy them to partition 1 (FAT32) under /uload-bootloader/ before running the ULoad flasher. This ensures the programmed bootloader matches the exact board and release configuration, minimizing risk of mismatch.

To begin with, to compile uLoad images, execute it from the host machine:

- **On Linux:** Open a terminal and run the following commands

```
renesas@builder-pc:~$ cd ~/renesas/rz-cmn-srp/host/tools/firmware_compile
renesas@builder-pc:~/renesas/rz-cmn-srp/host/tools/firmware_compile$ python3
firmware_compile.py
```

- **On Windows:** Open one of the following terminals with Administrator privileges:
  - PowerShell
  - Git Bash
  - MobaXterm

  Navigate to the host/tools/firmware_compile directory inside the rz-cmn-srp folder. The example below uses PowerShell, but the same applies to other terminals

```
PS C:\Users\renesas> cd C:\Users\renesas\rz-cmn-srp\host\tools\ firmware_compile
PS C:\Users\renesas\rz-cmn-srp\host\tools\firmware_compile> py
firmware_compile.py
```

If no CLI options are supplied, the script uses the following defaults:

Table 8. Firmware compiled CLI options

Option	Default	Description
--board	rzg2l-sbc	Target board name (must exist in boards_flash_config.toml and flash_images.json).
--soc	g2l	Target SoC family (g2l, v2l, v2h).
--method	xspi	Flash method (xspi, emmc or esd).
--bl2	auto from images	Path to BL2 binary (override default).
--atf-fdts	auto from JSON	TF-A FDT(s) to append to BL2.
--uboot-dtbs	auto from JSON	U-Boot DTB(s) to append to U-Boot nodtb.
--bl31	auto from images	Path to BL31 binary (override default).
--u-boot-nodtb	auto from images	Path to U-Boot (nodtb) binary (override default).
--bootparameter	auto search	Path to bpgen tool (override search path).
--fiptool	auto search	Path to the fiptool tool (override search path).
--objcopy	auto search	Path to objcopy tool (override search path).
--fip-align	16	FIP alignment.
--fip-vma	from TOML	Override virtual memory address (VMA) for FIP .srec.
--bl2-bp-vma	from TOML	Override VMA for BL2+BP.srec.

Note:

- auto from images: uses the **common artifacts** produced in target/images/… (not per-board overrides).
- auto from JSON: resolves per-board filenames/DTBs via flash_images.json.
- from TOML: uses addresses/layout defined in boards_flash_config.toml for the selected --board and --method

2.2.4.2 Collect the Output and Prepare Binaries on the SD Card

This step packages the artifacts built by firmware-compile.py and places them on the removable media so the ULoad-bootloader script (U-Boot console flow) can program QSPI/xSPI.

From <path/to/yocto/package/target/images/>, gather the per-board files:

- bl2: bl2_bp_<board>.bin.
- fip_<board>.bin

- <board>-<version>-platform-settings.bin

Place all files on partition 1 (FAT32) of the SD card under this directory.

```
/uload-bootloader
```

2.2.4.3 Hardware Connection

Before starting the flashing process, ensure that the required hardware interfaces are properly connected between the board and the host PC.

- Connect the debug serial port of the board to the host PC.
- Place the board into normal boot mode. The exact DIP-switch or jumper settings are board-specific.
    - Refer to section **Factory Firmware Flashing Using Serial Downloader (SCIF) Mode** for the required switch positions and steps for the target board.

2.2.4.4 Flashing Procedure

To begin using the uload_bootloader.py, execute it from the host machine:

- **On Linux:** Open a terminal and run the following commands

```
renesas@builder-pc:~$ cd ~/renesas/rz-cmn-srp/host/tools/uload_bootloader
renesas@builder-pc:~/renesas/rz-cmn-srp/host/tools/uload_bootloader$ python3
uload_bootloader_flash.py
```

- **On Windows:** Open one of the following terminals with Administrator privileges:
    - PowerShell
    - Git Bash
    - MobaXterma

  Navigate to the host/tools/uload_bootloader directory inside the rz-cmn-srp folder. The example below uses PowerShell, but the same applies to other terminals

```
PS C:\Users\renesas> cd C:\Users\renesas\rz-cmn-srp\host\tools\uload_bootloader
PS C:\Users\renesas\rz-cmn-srp\host\tools\uload_bootloader> py
uload_bootloader_flash.py
```

If no arguments are provided, the script will use the following default values for the RZ/G2L-SBC board.

- bl2 binary file: bl2_bp_<board-name>.bin
- fip file: fip_<board-name>.bin
- board identication file: <board-name>-platform-settings.bin

Ensure that these files are present in the current directory before executing the script.

To specify custom file paths or override the defaults, the following arguments can be passed:

- **--serial_port:** Serial port to use for communication with the board.
- **--serial_port_baud:** Baud rate for the serial port.
- **--image_bl2:** Path or filename of the BL2 image
- **--image_fip:** Path or filename of the FIP image
- **--image_bid:** Path or filename of the board-identification file

Example command:

- **On Linux:**

```
renesas@builder-pc:~/renesas/rz-cmn-srp/host/tools/uload_bootloader$ python3
uload_bootloader_flash.py \
  --serial_port /dev/ttyUSB0 \
  --serial_port_baud 115200 \
  --image_bl2 /mnt/sd/uload-bootloader/bl2_bp-rzg2l-sbc.bin \
  --image_fip /mnt/sd/uload-bootloader/fip-rzg2l-sbc.bin \
  --image_bid /mnt/sd/uload-bootloader/rzg2l-sbc-platform-settings.bin
```

- **On Windows:**

```
PS C:\Users\renesas\rz-cmn-srp\host\tools\uload_bootloader> py
uload_bootloader_flash.py \
  --serial_port /dev/ttyUSB0 \
  --serial_port_baud 115200 \
  --image_bl2 /mnt/sd/uload-bootloader/bl2_bp-rzg2l-sbc.bin \
  --image_fip /mnt/sd/uload-bootloader/fip-rzg2l-sbc.bin \
  --image_bid /mnt/sd/uload-bootloader/rzg2l-sbc-platform-settings.bin
```

Note:

- If only a filename is provided (no path), the script searches the default directory (e.g., /uload-bootloader on partition 1, FAT32).
- Ensure the filenames match the board that was built with firmware-compile.py.

2.2.5 Flashing the SD Card Image

This section explains how to use the dedicated SD flashing script, sd_creator.py, which helps create bootable SD cards using a fastboot-like approach.

```
host/tools/sd_creator/
```

Hierarchy:

```
sd_creator
├── README.md
├── sd_flash.py
└── tools
    ├── AdbWinApi.dll
    ├── AdbWinUsbApi.dll
    ├── fastboot.exe
    └── NOTICE.txt
```

2.2.5.1 Hardware Connection

Before starting the flashing process, ensure that the required hardware interfaces are properly connected between the board and the host PC. The type of connection depends on the flashing method selected.

- Connect the debug serial port of the board to the host PC for console access and monitoring.

- Connect the appropriate interface based on the fastboot method:
  - [UDP] Connect the Ethernet port of the board to the host PC.
  - [OTG] Connect the USB OTG port of the board to the host PC.
- Insert the SD card or choose to flash to an eMMC device based on the target board (see Table 11. Fastboot target device mapping for more detailed information)

Table 9. SD card flash - Hardware connection

Fastboot Type	Connection to Host	Board Ports Used	Notes
UDP	Ethernet cable + USB cable	RJ45 + Debug Serial	Requires an IP address and Ethernet port index
OTG	USB cable	USB OTG + Debug Serial	No Ethernet/IP required

U-Boot fastboot-udp uses a single active Ethernet MAC per board. If multiple RJ45/PHY ports are present, only one is active (board-dependent). Select the interface with --ether_port. This option maps to the U-Boot Ethernet device index and will be explained in detail in a later section.

Table 10. UDP Ethernet Port Index per board

Board	Ethernet port to use
RZ/G2L-SBC	1
RS-G2L100	0, 1
RZ/V2L-EVK	0
RZ/G2L-EVK	0
RZ/V2H-EVK	0, 1
RZ/V2H-RDK	0
IMDT V2H-SBC	0, 1

Both fastboot-otg and fastboot-udp write to U-Boot's current MMC device (typically mmc0). Depending on board revision, mmc0 may point to the SD card or eMMC.

Table 11. Fastboot target device mapping

Board/Rev	Fastboot Method	Typical mmc0 target	How to change target
RZ/G2L-SBC	UDP	Carrier SD (board default)	N/A (single device)
RS-G2L100	UDP, OTG	eMMC	N/A (single device)
RZ/V2L-EVK	UDP, OTG	SD (CN3 on SOM or eMMC device, depending on SW1)	Set SW1 ON to SD and OFF to eMMC
RZ/G2L-EVK	UDP, OTG	SD (CN3 on SOM or eMMC device, depending on SW1)	Set SW1 ON to SD and OFF to eMMC
RZ/V2H-EVK (Rev 1 – 2 SD cards)	UDP, OTG	SD card slot 0	N/A (single device)
RZ/V2H-EVK (Rev 2 – SD & eMMC)	UDP, OTG	eMMC	N/A (single device)
RZ/V2H-RDK	UDP	SD card	N/A (single device)
IMDT V2H-SBC	UDP, OTG	eMMC	N/A (single device)

2.2.5.2 Flashing Procedure

To begin using the sd_creator.py, execute it from the host machine:

- **On Linux:** Open a terminal and run the following commands

```
renesas@builder-pc:~$ cd ~/renesas/rz-cmn-srp/host/tools/sd_creator
renesas@builder-pc:~/renesas/rz-cmn-srp/host/tools/sd_creator$ python3
sd_flash.py
```

- **On Windows:** Open one of the following terminals with Administrator privileges:
  - PowerShell
  - Git Bash
  - MobaXterm

  Navigate to the host/tools/sd_creator directory inside the rz-cmn-srp folder. The example below uses PowerShell, but the same applies to other terminals

```
PS C:\Users\renesas> cd C:\Users\renesas\rz-cmn-srp\host\tools\sd_creator
PS C:\Users\renesas\rz-cmn-srp\host\tools\sd_creator> py sd_flash.py
```

If no arguments are provided, the script will use the following default values:

- Fastboot type: udp
- IP address: 169.254.187.89
- Serial port: most recently connected port (e.g., COM8 in Windows or /dev/ttyUSB0 in Linux)
- Serial port baud: 115200
- WIC file: </path/to/your/package>/target/images/core-image-minimal.wic

Ensure that these files are present in the current directory before executing the script.

To specify custom file paths or override the defaults, the following arguments can be passed:

- **--board_name**: Board name to flash the bootloader. Default is rzg2l-sbc.
- **--fastboot_type**: Fastboot type to use (udp or otg). Default is udp.
- **--ether_port**: [Only used in fastboot UDP] Ethernet port used for board communication. Defaults to 1.
- **--ip_address**: [Only used in fastboot UDP] Ethernet IP address used for board communication. Defaults to 169.254.187.89.
- **--serial_port**: Serial port to use for communication with the board. Default is the most recently connected port (e.g., COM8 in Windows or /dev/ttyUSB0 in Linux).
- **--serial_port_baud**: Baud rate for the serial port. Must be 115200.
- **--image_rootfs**: Path to the root filesystem image.

Example command

For UDP flash:

- **On Linux:**

```
renesas@builder-pc:~/renesas/rz-cmn-srp/host/tools/sd_creator$ python3 \
sd_flash.py --board_name rzg2l-evk --fastboot_type udp --ip_address \
169.254.187.9 --ether_port 1 --serial_port /dev/ttyUSB0 \
--serial_port_baud 115200 \
--image_rootfs /home/renesas/custom_images/core-image-weston.wic
```

- **On Windows:**

```
PS C:\Users\renesas\rz-cmn-srp\host\tools\sd_creator> py sd_flash.py \
--board_name rzg2l-evk --fastboot_type udp --ip_address 169.254.187.9 \
--ether_port 1 --serial_port COM11 --serial_port_baud 115200 \
--image_rootfs D:\custom_images\core-image-weston.wic
```

For OTG flash:

- **On Linux:**

```
renesas@builder-pc:~/renesas/rz-cmn-srp/host/tools/sd_creator$ python3 \
sd_flash.py --fastboot_type otg --serial_port /dev/ttyUSB0 \
--serial_port_baud 115200 --image_rootfs \
/home/renesas/custom_images/core-image-weston.wic
```

- **On Windows:**

```
PS C:\Users\renesas\rz-cmn-srp\host\tools\sd_creator> py sd_flash.py \
--fastboot_type otg --serial_port COM11 --serial_port_baud 115200 \
--image_rootfs D:\custom_images\core-image-weston.wic
```

Note:

- For OTG, `--ip_address` and `--ether_port` are not applicable.
- For UDP, choose the correct `--ether_port` for the target board (see the Table 10. UDP Ethernet Port Index per board)
- IMDT-V2H-SBC UDP-based flashing: Ethernet operation in U-Boot may be unstable; if UDP-based flashing is required, Ethernet must be initialized in the Linux kernel first, and the system rebooted without disconnecting power; otherwise, OTG (default option) should be used for flashing.

2.3 RZ/G2L-SBC

This section describes the hardware-specific processes for the RZ/G2L-SBC (Single-board Computer).

Note:

- The release consists of images that have user desktop and display support.
- At least one basic display, like a 1080p HDMI monitor, must be available for those images.
- You can also use the DSI touch panel described in the MIPI DSI Display Touch Panel.

It is recommended to use an FTDI cable for the UART and not any other converter chip.

2.3.1 Overview of Connectors

Given below is the basic positioning of the top-level connectors.

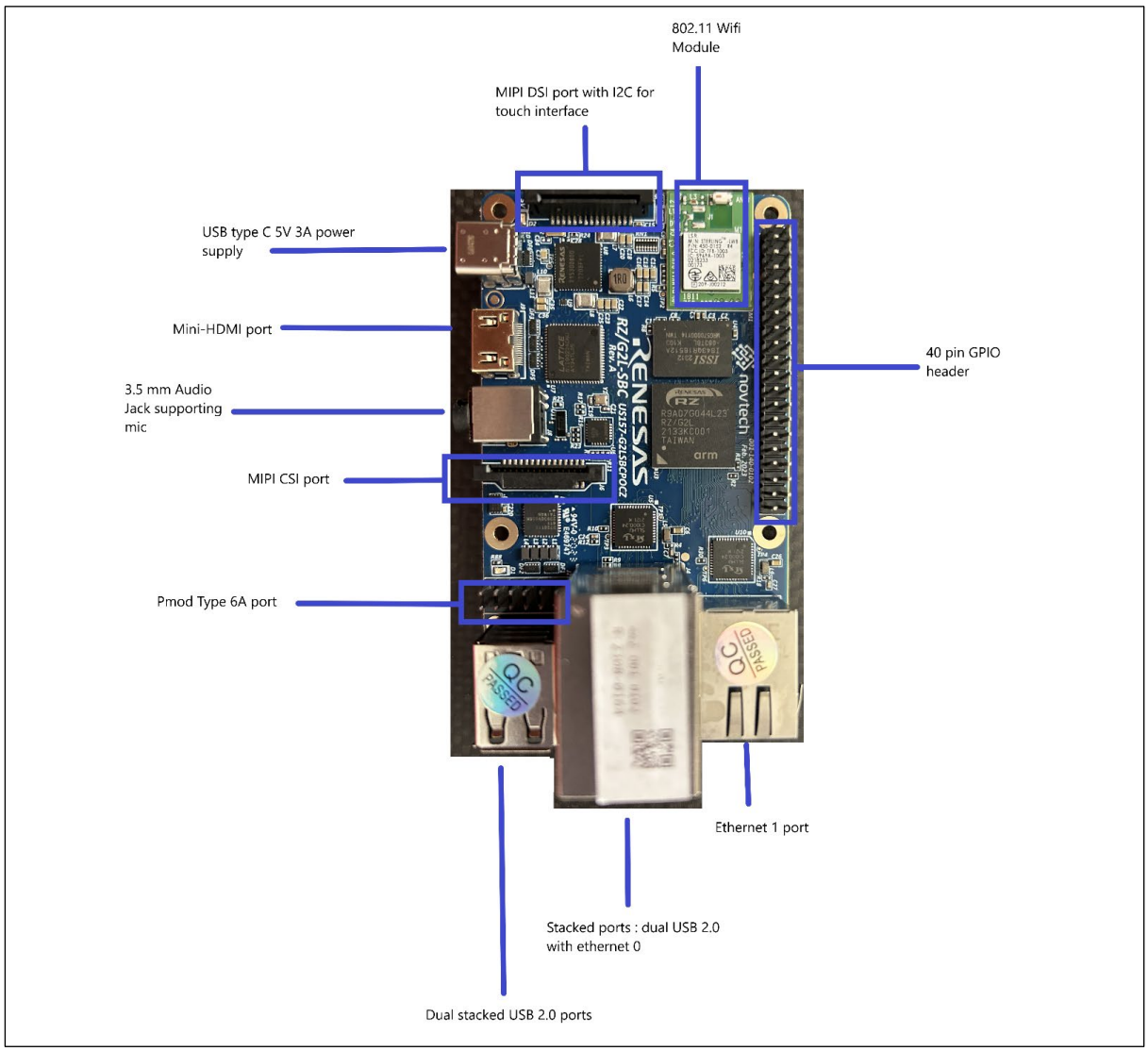


Figure 4. RZ/G2L-SBC top side connectors

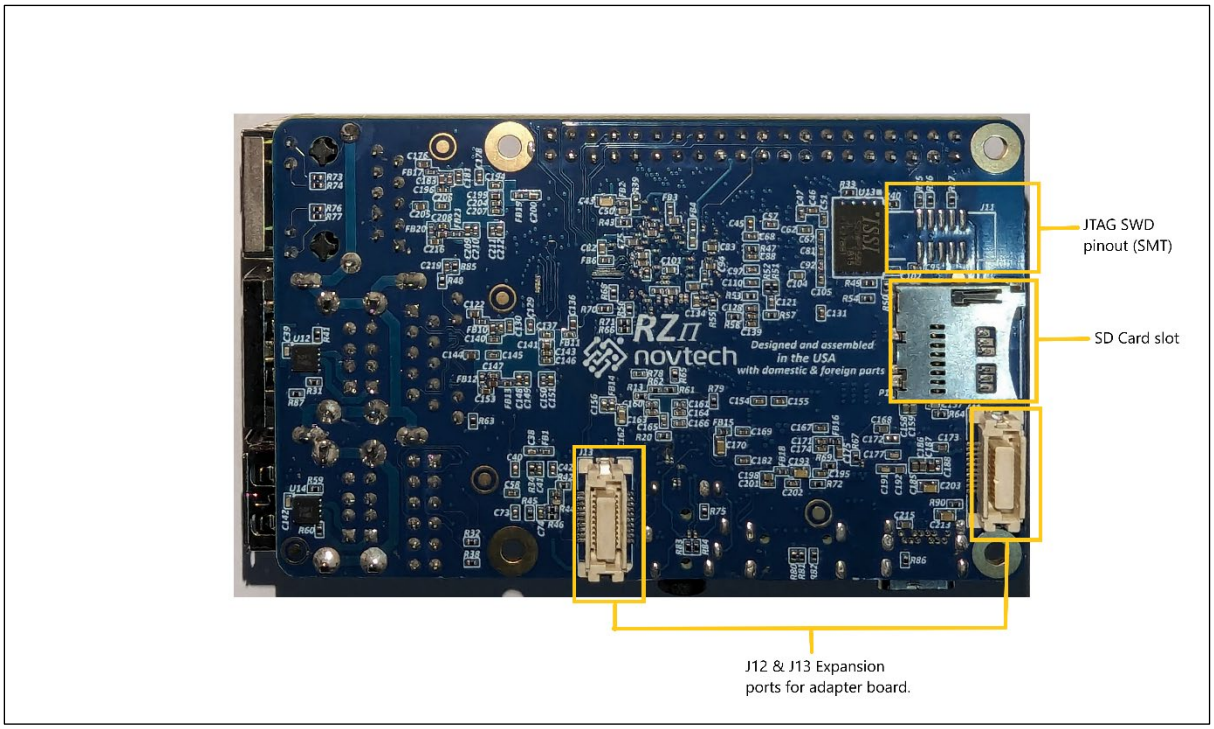


Figure 5. RZ/G2L-SBC Bottom view connectors

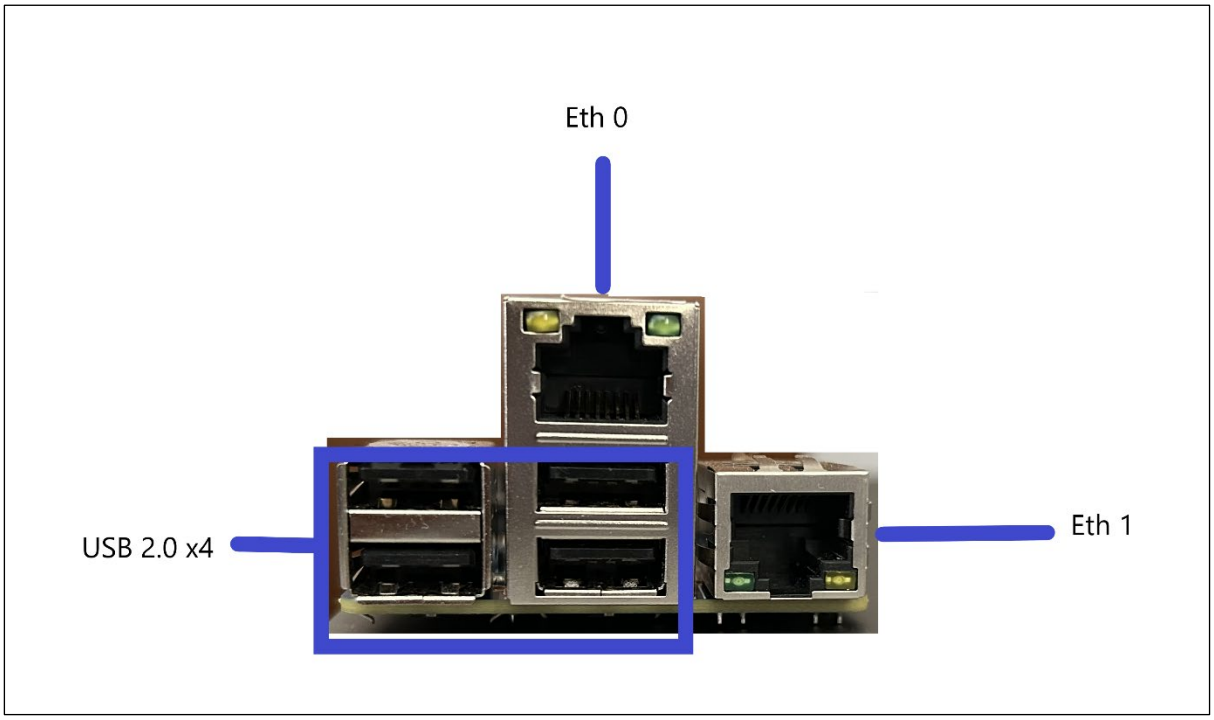


Figure 6. RZ/G2L-SBC side view I/O ports

2.3.1.1 Power Supply

This section delves into the RZ/G2L-SBC's power supply architecture. The RZ/G2L-SBC uses a simple design with a 5V supply as the single external power source.

(1) USB TYPE-C POWER

This board has one USB Type-C receptacle for power input with USB chargers. The USB Type–C power connector is meant to connect to a 5V power supply. The RZ/G2L-SBC requires a minimum of 3A power to prevent brownouts. However, we recommend a 4.5 -5A power supply, as several ports support peripherals that consume substantial power.

2.3.1.2 Peripheral Interface

(1) 40-PIN I/O HEADER

The RZ/G2L-SBC comes with a 40-pin GPIO interface, which is broadly compliant with the Raspberry Pi 3 40-pin GPIO interface and provides additional interfaces like two CAN ports. The diagram below shows the pin configuration along with the marking of the bottom I/O ports for reference to the orientation of the board.

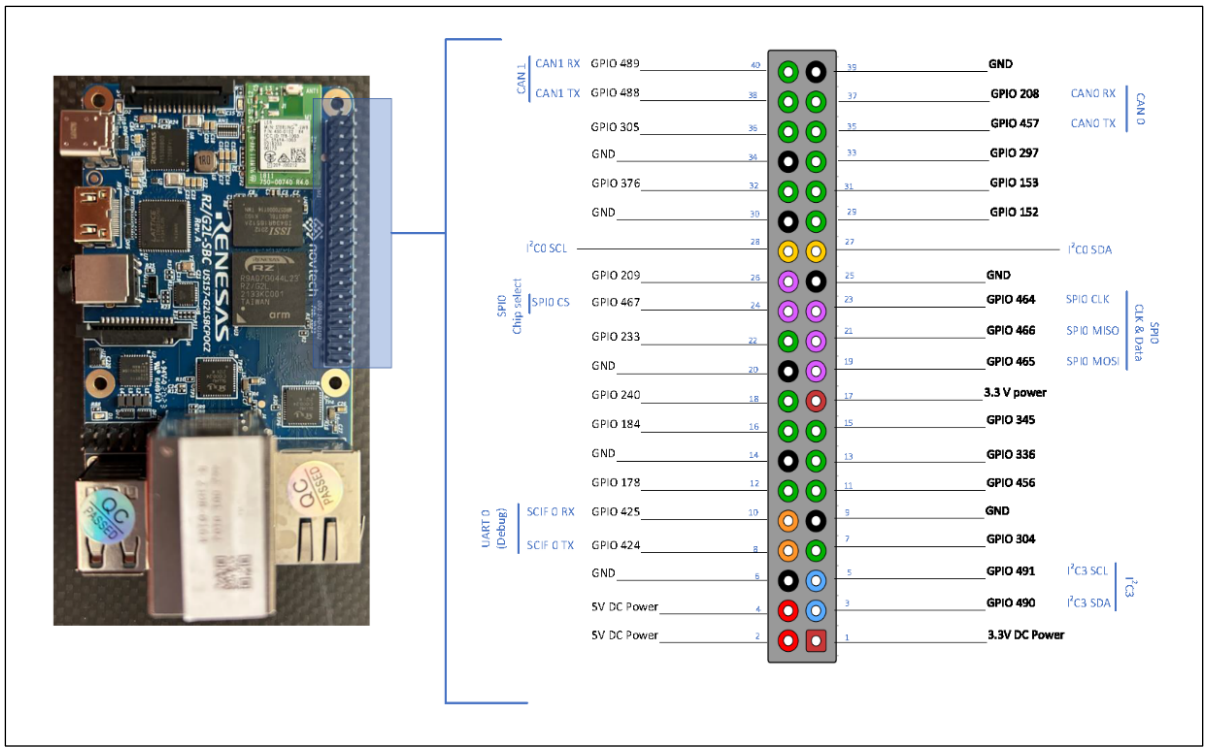


Figure 7. 40-pin GPIO map with orientation details

(2) PMOD TYPE 6A STANDARD INTERFACE

The RZ/G2L-SBC is equipped with a 2x6-pin header routed to the PMOD Type-6A interface, conforming to the 1.3.0 specification of PMOD. It includes the alternate pin functions from the specification.

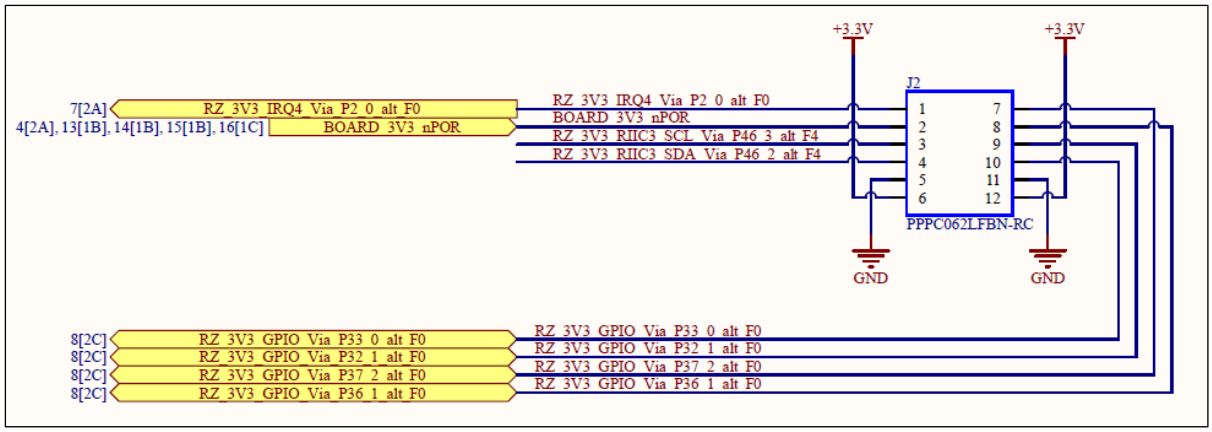


Figure 8. Schematic of PMOD Type 6A pin header J2

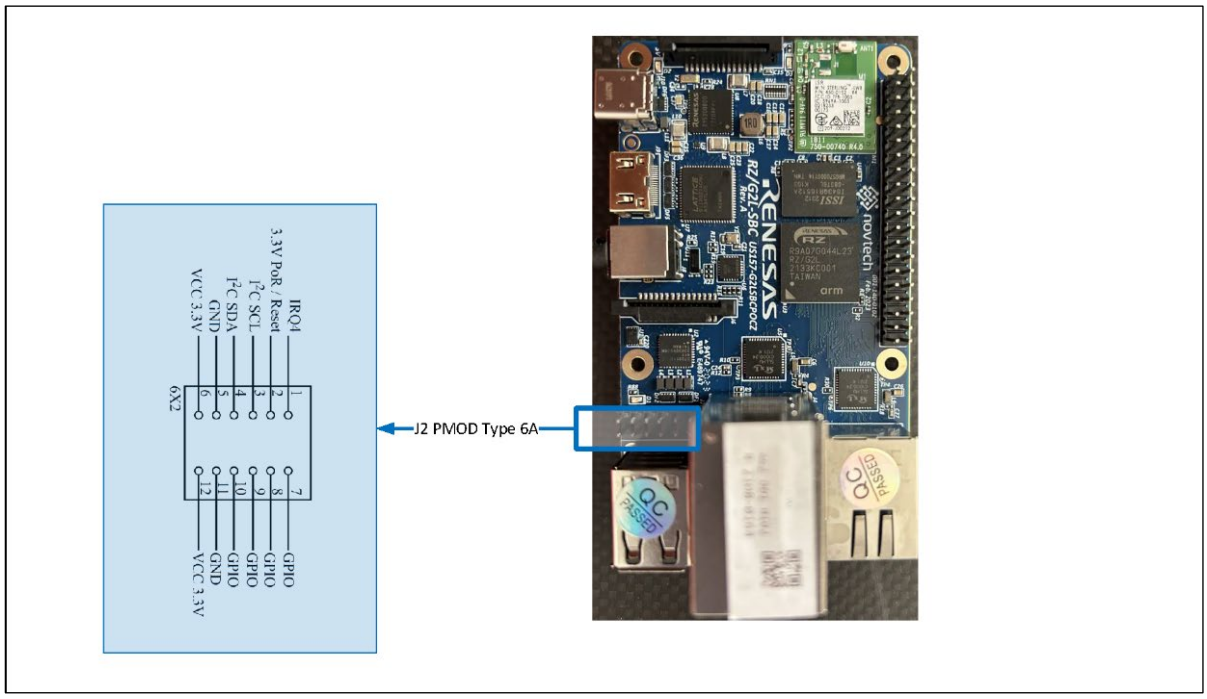


Figure 9. PMOD Type 6A 2x6 0.1 mm pinout with orientation details

(3) MICROSD CARD INTERFACE

The RZ/G2L-SBC comes with a spring-loaded microSD card slot. This is intended to be the primary storage as well as the OS boot device. The SD card is connected to channel 0 of the RZ/G2L SoC SD/MMC interface. The SoC SDIO interface is compliant with memory card standard version 3.0 and supports UHS-1 mode of 50 MB/s (SDR50) and 104 MB/s (SDR104).

2.3.2 Hardware Requirements

The hardware setup consists of the following:

Table 12. Accessory List for RZ/G2L-SBC

Item	Required/ Optional	Exact accessory	Interface / Connector Type	Notes
Board	Required	RZ/G2L-SBC	—	Main target board
Storage media	Required	microSD card, minimum 8GB	microSD	Used for Linux image
Debug cable	Required	FTDI RS232 UART cable	UART header	FTDI recommended; avoid PL2302
Power supply	Required	USB-C 5V 3A+ power supply	USB-C power	Required for power source
Display cable	Required for graphical images	Mini-HDMI to HDMI cable	HDMI	Required for HDMI display path
Display	Required for graphical images	1080p HDMI monitor	HDMI	Recommended for Weston / GUI images
DSI Display	Optional / recommended	1080p HDMI display/Waveshare 5” MIPI DSI display touch panel	MIPI DSI	Supported alternative to HDMI
CSI camera	Optional	OV5640 MIPI CSI camera	MIPI CSI	Required for camera usage
Ethernet cable	Optional / Recommended	Standard Ethernet cable	RJ45	Used for networking
USB keyboard	Optional	Standard USB keyboard	USB	Needed for local GUI use
USB mouse	Optional	Standard USB mouse	USB	Needed for local GUI use

Audio accessory	Optional	3.5mm headphone with microphone	Audio jack	Required for audio usage

2.3.3 Essential Hardware Setup

Figure 10. Essential minimum interfaces show the basic hardware setup. We expect a UART cable and an HDMI display to be available.

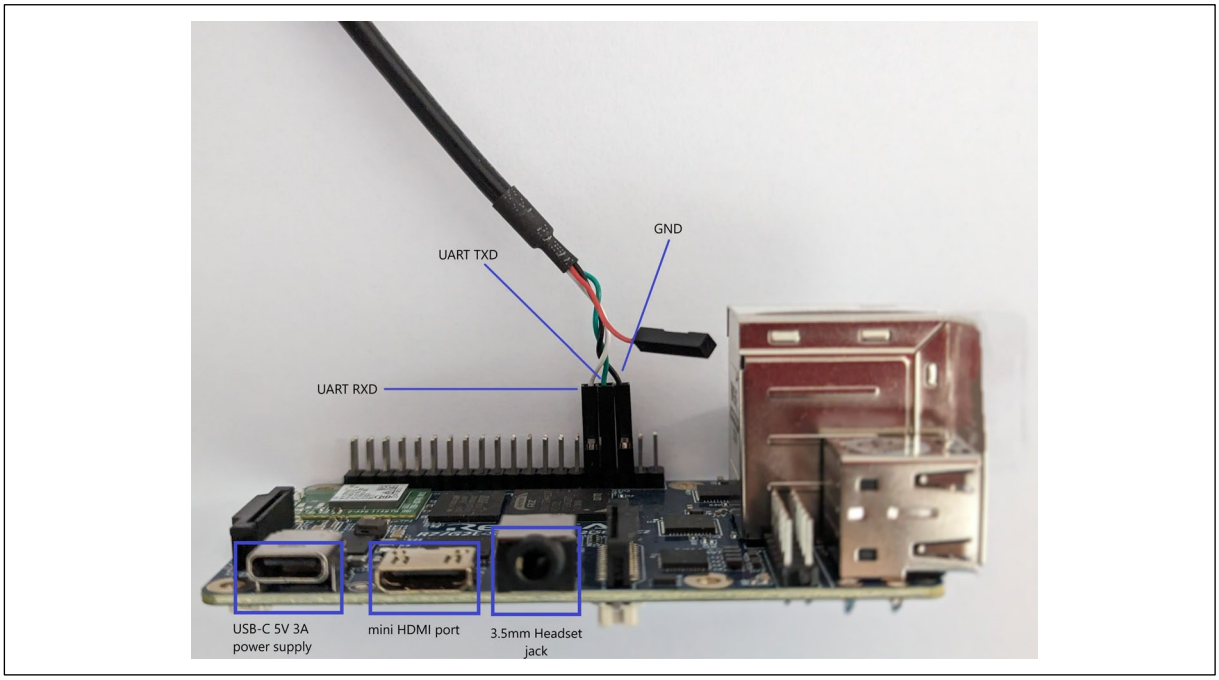


Figure 10. Essential minimum interfaces for RZ/G2L-SBC

2.3.4 Complete Hardware Setup

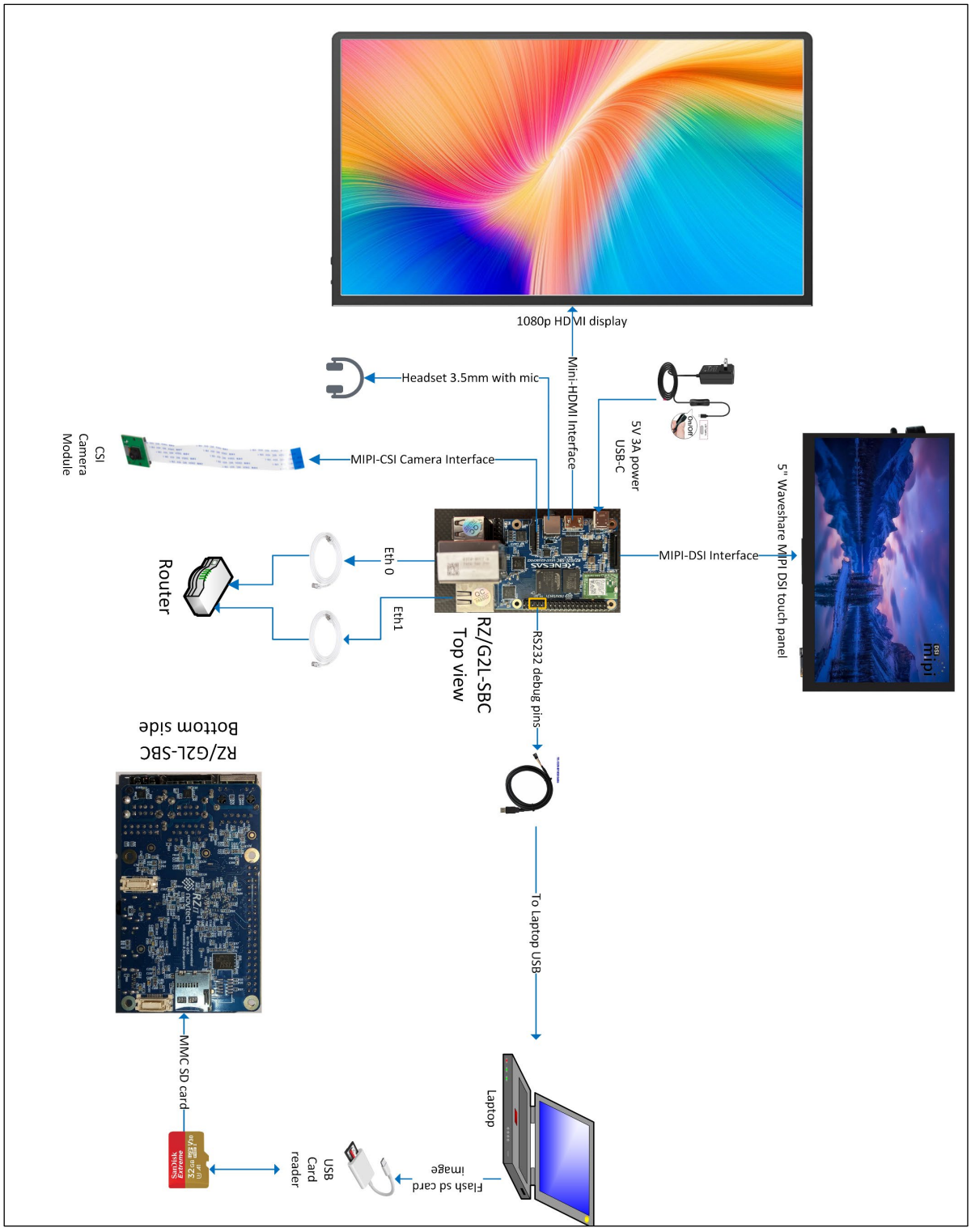


Figure 11. Complete setup for RZ/G2L-SBC board

2.3.5 Booting

Booting is straightforward.

1. Insert the MMC card into the MMC port on the bottom side of the RZ/G2L-SBC.
2. Connect the keyboard, mouse, and HDMI display; then insert the USB-C power supply and turn the power on.
3. You should see the boot log on the UART console and the Weston desktop on the HDMI screen.
4. Click on any of the applications and interact with them.

The image is fully featured and has powerful desktop-grade features. Read further to learn more about the features packed into the Linux image.

Notes: The default firmware shipped on the board may not recognize new images. If the board fails to boot, update the firmware using the Serial Download Mode (SCIF) procedure described in section 3.1.1

2.3.6 Known Hardware and Functional Limitations on RZ/G2L-SBC

2.3.6.1 Linux (CA55) Side Known Issues

1. HDMI audio

- Status: Unverified
- Description: The functionality of the HDMI audio output has not been tested yet, and its behavior remains uncertain. Additional development and testing are required to assess its reliability and performance on the RZ/G2L-SBC.

2. Audio Sampling Rate Limitation

- Status: Currently limited to 48 kHz (validated)
- Description: The board's clock design deviates from the standard RZ reference, which prevents the existing driver frameworks from generating the proper clocks when the SoC operates as an I²S/TDM master. As a result, only 48 kHz sampling has been successfully validated so far. This is not a fundamental hardware restriction — codec-master mode has not yet been evaluated, and wider sampling-rate support may be possible but remains unsupported.

3. Onboard Bluetooth (BT) Functionality

- Status: Non-functional (onboard BT only)
- Description: The onboard Bluetooth functionality is currently non-operational due to a schematic symbol error in the Laird Wi-Fi/BT module. The Bluetooth interface is missing from the module's schematic design, preventing Bluetooth connectivity. However, USB Bluetooth functionality remains operational. This issue requires a hardware revision to enable full Bluetooth functionality on the onboard module.

2.3.6.2 FreeRTOS/FSP (CM33) Side Known Issues

1. MIPI-CSI2 Camera and Peripherals Accessing Shared I2C1 Bus

In the RZ/G2L-SBC, the MIPI CSI Camera interface, HDMI Bridge, and MIPI DSI all share the same I2C1 channel. Due to this hardware constraint, controlling one of these devices may impact the functionality of the others.

Limitations:

- I2C1 can only be accessed by one core at a time, which can prevent both the camera and display from functioning simultaneously.
- Any device using I2C1 must be managed carefully to avoid conflicts with other peripherals.
- This limitation should be considered when designing the system to ensure both peripherals can operate as required.

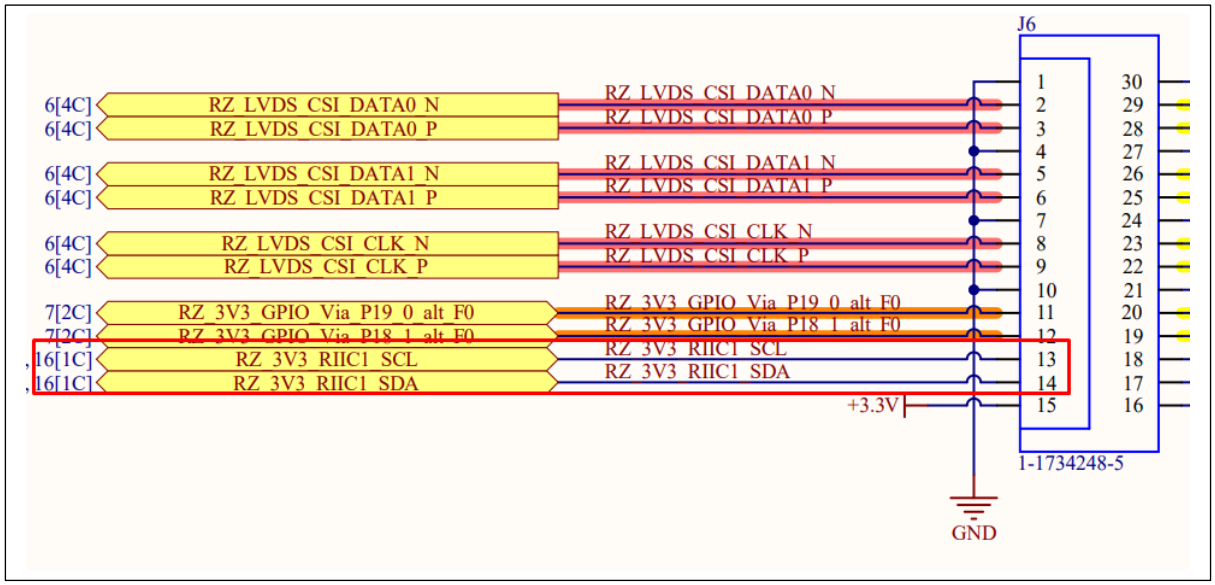


Figure 12. CSI using shared I2C1 bus

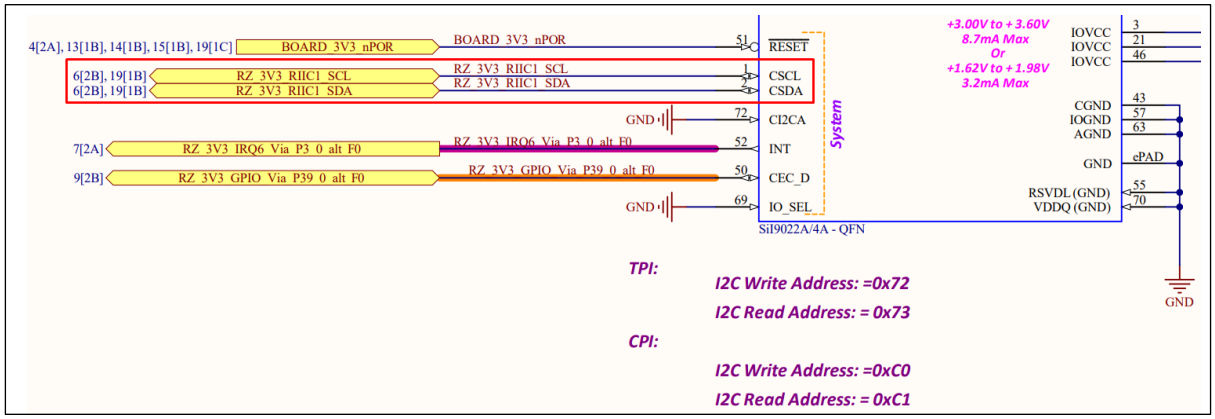


Figure 13. HDMI using the shared I2C1 bus

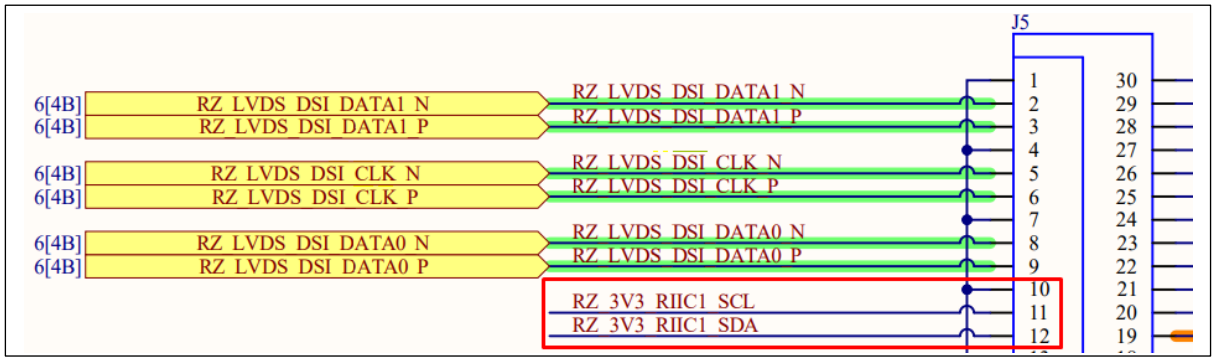


Figure 14. DSI using the shared I2C1 bus

As shown in the three figures above, the shared I2C1 bus is used by multiple peripherals, which may lead to conflicts if both cores are used simultaneously. To avoid issues, users should ensure that only one core accesses I2C1 at a time or consider alternative methods for managing communication between peripherals.

2. Limited SCIF Availability for Multi-Core Development

However, a limitation exists in the number of available SCIF (Serial Communication Interface with FIFO) channels, which impacts debugging and logging functionality for multi-core development.

Limitations:

- Single SCIF Channel: Only SCIF0 is available for serial communication, and it is exclusively allocated to the CA55 core.
- Restricted logging for CM33: Since SCIF0 is dedicated to CA55, the CM33 core lacks direct access to an SCIF channel, making it challenging to perform independent serial logging or debugging.

This limitation should be considered when designing multi-core applications, especially those requiring real-time logging, debugging, or inter-core communications.

2.4 RS-G2L100

This section describes the hardware-specific setup and booting process for the RS-G2L100

2.4.1 Overview of Connectors

Figure 15. Overview of Connectors - RS-G2L100

2.4.2 Hardware Requirements

The basic hardware setup consists of the following:

Table 13. Accessory List for RS-G2L100

Item	Required/ Optional	Exact accessory	Interface / Connector Type	Notes
Board	Required	RS-G2L100	—	Main target board

Storage media	Required	microSD card, minimum 8GB	microSD	Used for Linux image
Debug cable	Required	FTDI RS232 UART cable	UART header	FTDI recommended, avoid PL2302
Power supply	Required	DC 12V/3A adapter	DC jack	Required for power source
Display cable	Required for graphical images	HDMI to HDMI cable	HDMI	Required for HDMI display path
Display	Required for graphical images	1080p HDMI monitor	HDMI	Needed for display output
Ethernet cable	Optional / Recommended	Standard Ethernet cable	RJ45	Used for networking
USB keyboard	Optional	Standard USB keyboard	USB	Needed for local GUI use
USB mouse	Optional	Standard USB mouse	USB	Needed for local GUI use
Audio accessory	Optional	3.5mm headphone with microphone	Audio jack	Only HDMI audio is supported

2.4.3 Essential Hardware Setup and Booting

The figure below shows the basic essential hardware setup for RS-G2L100. Follow these steps to prepare and power on the board:

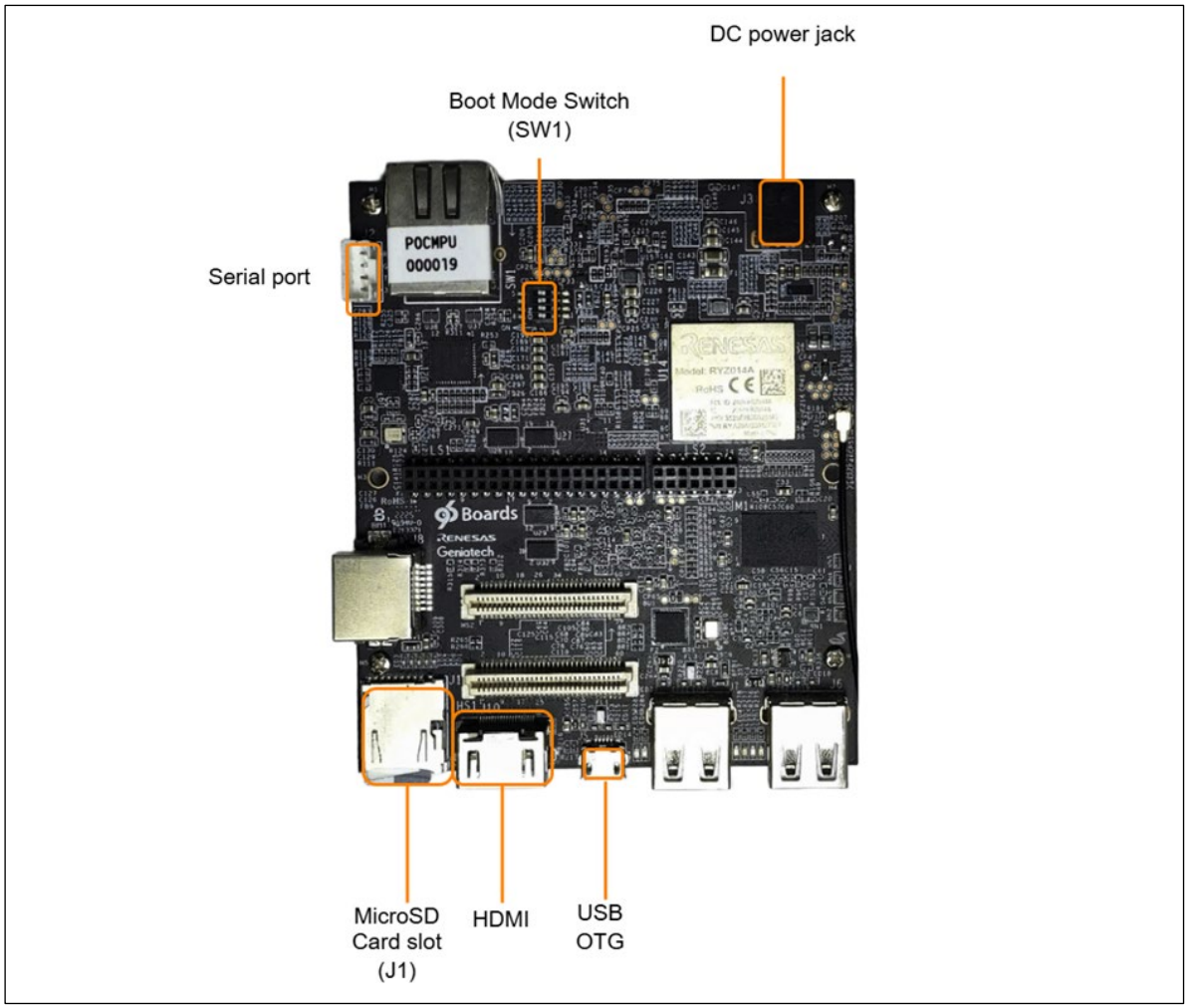


Figure 16. Essential minimum interfaces for RS-G2L100

1. **Prepare the microSD Card**

   Flash the provided image to the microSD card using a user-friendly tool such as Balena Etcher, as described in section SD/MMC Card Flashing for a simplified flashing experience.

2. **Insert the microSD Card**

   Insert the prepared SD card into J1. This slot is for rootfs/data only; it cannot be used to load BL2/FIP.

3. **Configure the Boot Mode**

   In this section, we focus on QSPI boot as the default boot method.

   - SW1 (on the carrier board): Set to QSPI boot.

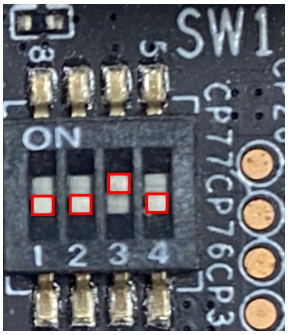


Figure 17. RS-G2L100 xSPI boot switch

4. **Connect Peripherals**
    - Attach an HDMI display
5. **Connect UART**
    - Use an FTDI RS232 UART cable to connect the board's UART console port to the host PC.
6. **Power on the Board**
    - Connect the Adapter power 12V 3A jack DC to the board
    - Wait for the UART serial device to appear on the host PC.
    - Open a terminal program on the host PC to monitor the boot log.

Once powered on, the boot log should appear on the UART console, and the Weston desktop will display on the HDMI screen.

2.4.4 Complete Hardware Setup

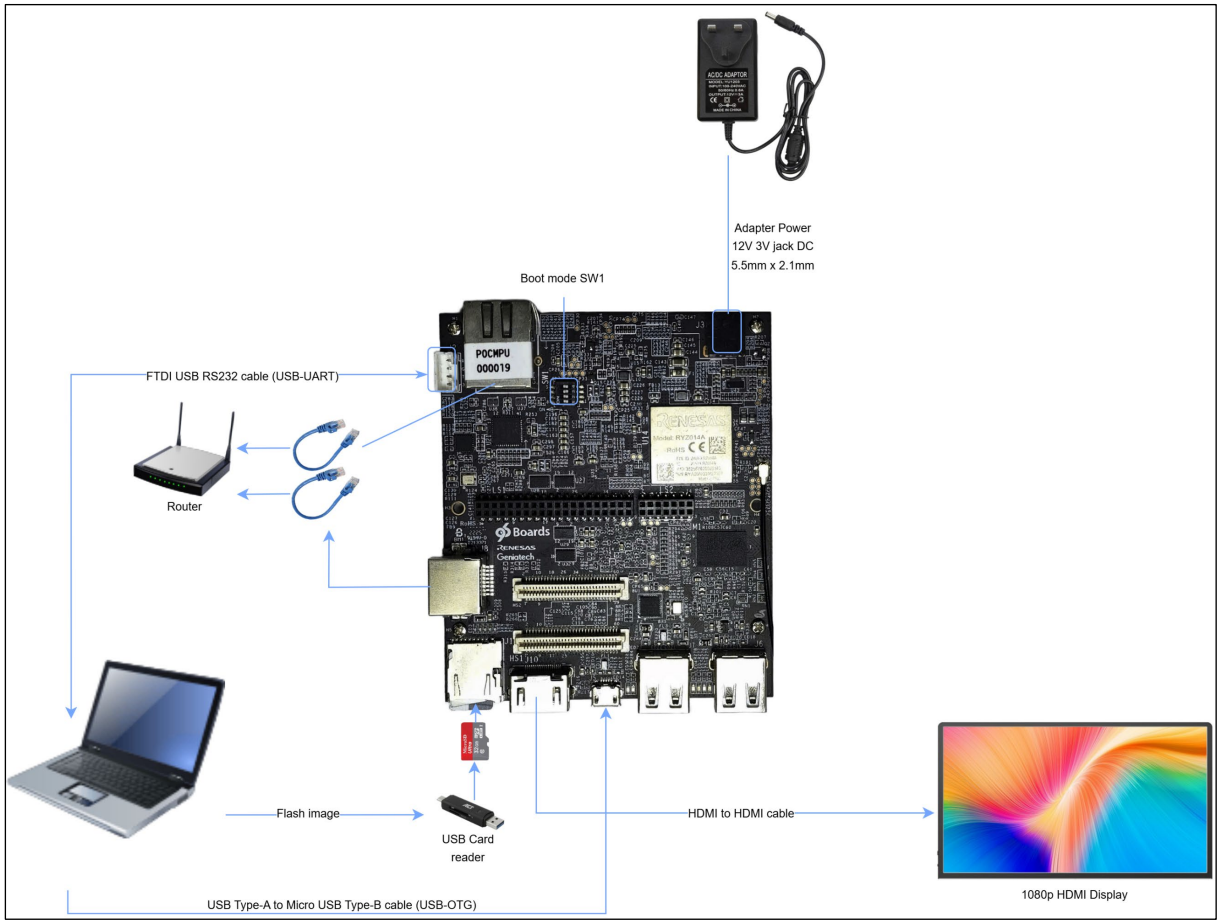


Figure 18. Complete setup for RS-G2L100

2.4.5 Booting

The boot process is straightforward:

1. Insert the prepared SD card into J1.
2. Connect a keyboard, mouse, HDMI display, and other devices. Then connect the DC power supply.
3. The boot log should appear on the UART console, and the Weston desktop should display on the HDMI screen.
4. Open any of the available applications to interact with the system.

The image provided is feature-complete, offering a desktop-grade experience.

If a different boot device is required, such as QSPI or eMMC, adjust the DIP switch settings as described in section **3.2.1** before powering on. The selected boot device determines where on-chip ROM code loads the Boot Loader stage 2 (BL2) and Firmware Image Package (FIP):

- QSPI boot loads BL2 and FIP from the onboard QSPI flash.
- eMMC boot is not supported in this release.

Notes: The default firmware shipped on the board may not recognize new images. If the board fails to boot, update the firmware using the Serial Download Mode (SCIF) procedure described in sections 3.1.2

2.4.5.1 Default Boot Behavior

When booting from QSPI, U-Boot can use either the SD card (J1) or eMMC (U2) as the root filesystem.

- The SD card on J1 is always available as mmc1.
- The onboard eMMC is always available as mmc0.
- Unless reconfigured, Linux will mount the rootfs from J1 (e.g., /dev/mmcblk1p1).

2.4.5.2 Using an eMMC Root Filesystem

The onboard eMMC may be used for:

- kernel/DTB only (root filesystem on SD card),
- root filesystem only (kernel/DTB on SD card),
- both kernel/DTB and root filesystem.

Steps:

1. Prepare the eMMC rootfs. Please refer to section **Prepare the eMMC root filesystem** for detailed information.
2. Reboot the board. During startup, output similar to the following will appear:

```
NOTICE:  BL2: v2.9(release):<release-tag>
NOTICE:  BL2: Booting BL31
NOTICE:  BL31: v2.9(release): <release-tag>

U-Boot 2021.10 <Date>

CPU:   Renesas Electronics CPU rev 1.0
Model: <board-name>
DRAM:  1.9 GiB
MMC:   sd@11c00000: 0, sd@11c10000: 1
Loading Environment from SPIFlash... SF: Detected mt25qu512a ...
*** Warning - bad CRC, using default environment

Hit any key to stop autoboot:  0
=>
```

   - When the message “Hit any key to stop autoboot” appears, press Enter (or any key) to interrupt the countdown.
   - The => prompt indicates that you are in the U-Boot console.
3. Choose one configuration and apply the corresponding commands:

   Detect current environment

```
=> printenv mmcdev mmc_args
```

   Device mapping
   - mmcdev
     - mmcdev=0 → eMMC

    - mmcdev=1 → SD card
- mmc_args
    - root=/dev/mmcblk0p2 → root filesystem from eMMC (MMC device 0, partition 2)
    - root=/dev/mmcblk1p2 → root filesystem from SD card (CN10) (MMC device 1, partition 2)

Platform default (typical): kernel/DTB load from SD; rootfs points to /dev/mmcblk1p2 = SD, partition 2.

```
=> printenv mmcdev mmc_args
mmcdev=1
mmc_args=setenv bootargs rw rootwait earlycon root=/dev/mmcblk1p2
```

Choose one of the following configurations

- Root filesystem on eMMC (kernel, DTB from SD card)

```
=> setenv mmcdev 1
=> setenv mmc_args 'setenv bootargs rw rootwait earlycon root=/dev/mmcblk0p2'
=> saveenv
=> boot
```

- Kernel/DTB on eMMC (root filesystem on SD card)

```
=> setenv mmcdev 0
=> setenv mmc_args 'setenv bootargs rw rootwait earlycon root=/dev/mmcblk1p2'
=> saveenv
=> boot
```

- Kernel/DTB and root filesystem on eMMC

```
=> setenv mmcdev 0
=> setenv mmc_args 'setenv bootargs rw rootwait earlycon root=/dev/mmcblk0p2'
=> saveenv
=> boot
```

2.4.6 Known Hardware and Functional Limitations

1. eMMC boot is not supported in this release.
2. Camera / CSI – DSI: RS-G2L100 does not support a standalone MIPI CSI-2 connection. The camera and CSI – DSI feature is not yet supported in this release.
3. Audio: Audio is non-functional on-board version 2 due to the absence of an audio codec in both the schematic and the hardware. Only HDMI audio is tested and verified.
4. Wi-Fi: The Wi-Fi module interfaces with the RZ/G2L processor via the QSPI1 interface. However, the DA16600 driver only supports SPI and SDIO communication for Wi-Fi. There is no implementation for QSPI, so the Wi-Fi feature is not supported in this release.
5. SIM – LTE: The SIM–LTE feature is unsupported because the RYZ014A LTE module is no longer in production and has reached EOL.
6. Bluetooth: The Bluetooth module DA16600 hardware design only includes a 2-pin UART layout, making it impossible to support Bluetooth functionality.

7. Ethernet: The KSZ9131 PHY address configuration pins (PHYAD1 and PHYAD0) are left floating on both PHY0 and PHY1. Pull-down resistors to GND are required on PHY0 (PHYAD1, PHYAD0), and pull-up resistors to VDD are required on PHY1 (PHYAD1, PHYAD0) to properly set the PHY addresses to 4 and 7, respectively. Without these resistor modifications, Ethernet functionality will be unreliable or non-functional.

2.4.7 Solutions and Workarounds

2.4.7.1 Ethernet Hardware Fix

The RS-G2L100 board requires manual resistor installation to configure the PHY addresses correctly so that PHY0 will have address 4 and PHY1 will have address 7, as required by the RS-G2L100 device tree and driver configuration.

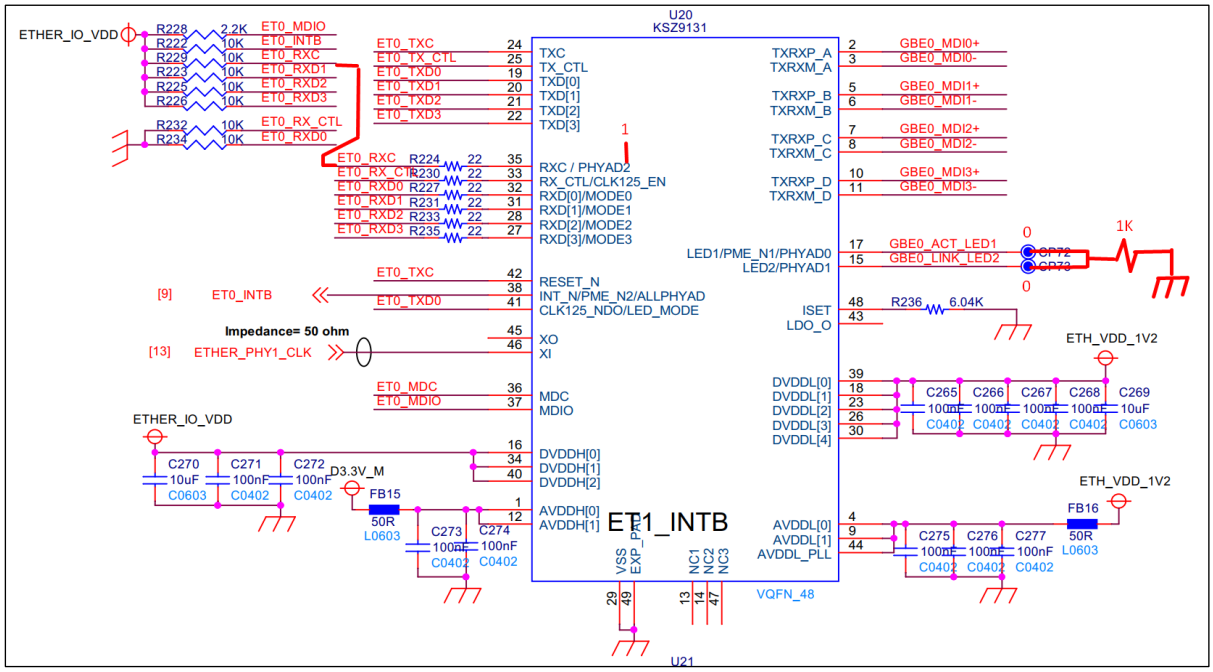


Figure 19. RS-G2L100 - Ethernet 0 Hardware Modifications

The schematic above shows the modification for Ethernet 0 by connecting GBE0_ACT_LED1 (Pin 17) and GBE0_LINK_LED2 (Pin 15) to GROUND through 1kΩ pull-down resistors to properly configure the PHY0 address to 4.

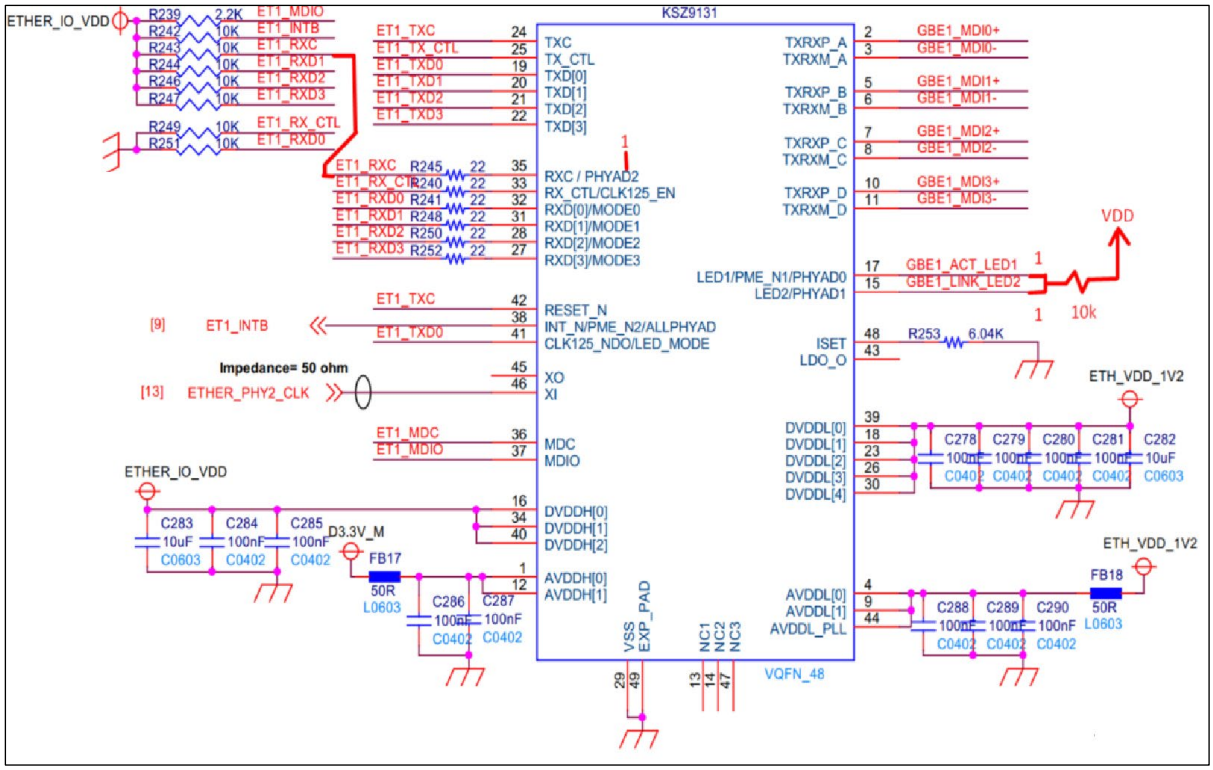


Figure 20. RS-G2L100 – Ethernet 1 Hardware Modifications

The schematic above shows the modification for Ethernet 1 by connecting GBE1_ACT_LED1 (Pin 17) and GBE1_LINK_LED2 (Pin 15) to VDD through 10kΩ pull-up resistors to properly configure the PHY1 address to 7.

Without these resistor modifications, Ethernet functionality will be unreliable or non-functional.

2.5 RZ/G2L-EVK and RZ/V2L-EVK

Both RZ/G2L-EVK and RZ/V2L-EVK platforms feature robust Linux images with desktop and display support. Their hardware setup and booting procedures are nearly identical, allowing a common approach for setup and evaluation.

Note:

- The release consists of images that have desktop and display support.
- At least one basic display, like a 1080p HDMI monitor, must be available for those images.

2.5.1 Overview of Connectors

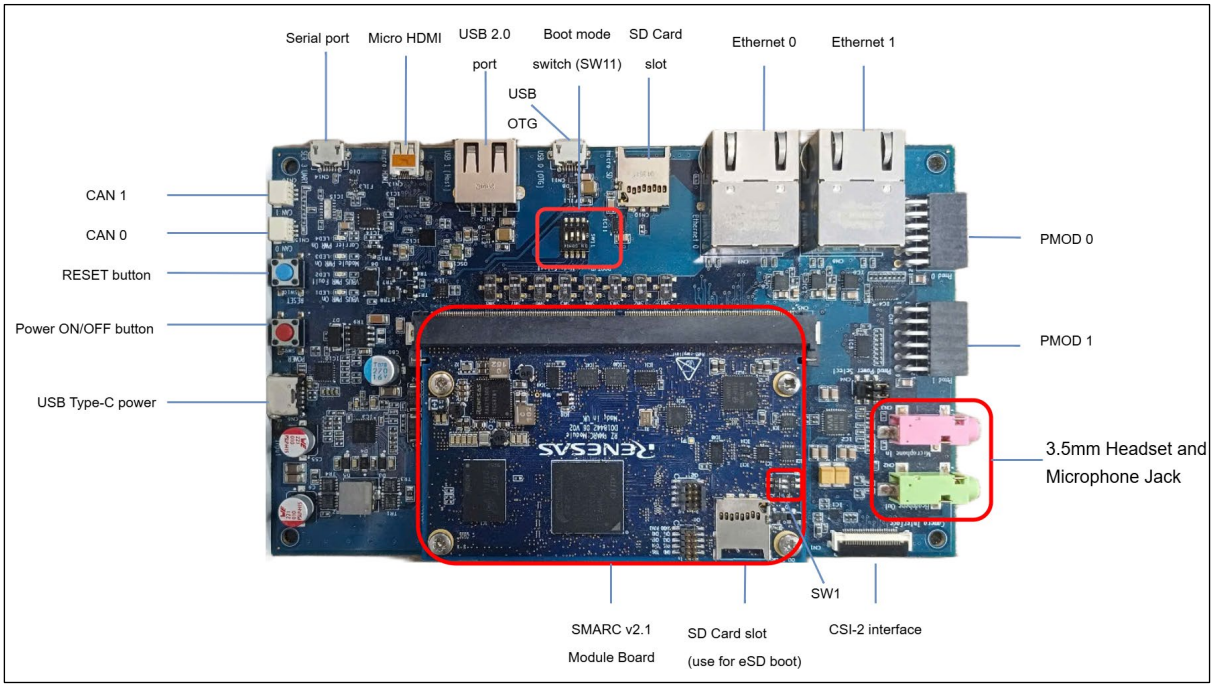


Figure 21. Overview of Connectors - RZ/G2L-EVK & RZ/V2L-EVK

2.5.2 Hardware Requirements

The hardware setup consists of the following:

Table 14. Accessory List for RZ/G2L-EVK and RZ/V2L-EVK

Item	Required/ Optional	Exact accessory	Interface / Connector Type	Notes
Board	Required	RZ/G2L-EVK or RZ/V2L-EVK	—	Main target board
Storage media	Required	microSD card, minimum 8GB	microSD	Used for Linux image
Debug cable	Required	USB Type-A to Micro USB Type-B cable	UART / debug serial (CN14)	—
Power supply	Required	USB-C 5V 3A+ power supply	USB-C power	Required for power source
Display cable	Required for graphical images	micro-HDMI to HDMI cable	HDMI	Required for HDMI display path

Display	Required for graphical images	1080p HDMI monitor	HDMI	Recommended for Weston / GUI images
CSI camera	Optional	OV5645 camera module	MIPI CSI	Required for camera usage
Ethernet cable	Optional / Recommended	Standard Ethernet cable	RJ45	Used for networking
USB keyboard	Optional	Standard USB keyboard	USB	Needed for local GUI use
USB mouse	Optional	Standard USB mouse	USB	Needed for local GUI use
Audio accessory	Optional	3.5mm headphone with microphone	Audio jack	Required for audio usage
Audio splitter	Optional	3.5mm Audio Stereo Y Splitter extension cable	Audio jack	Needed for separate mic/headphone use

2.5.3 Essential Hardware Setup and Booting

The figure below shows the basic essential hardware setup for RZ/G2L-EVK & RZ/V2L-EVK. Follow these steps to prepare and power on the board:

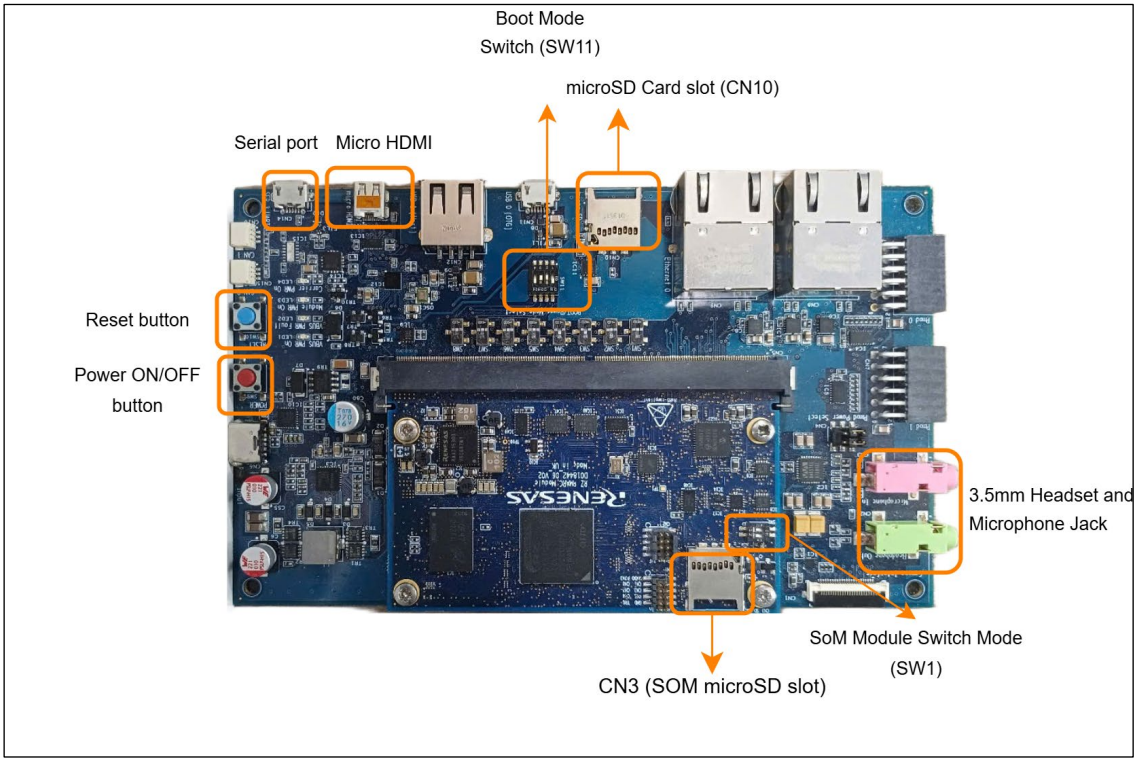


Figure 22. Essential minimum interfaces for RZ/G2L-EVK & RZ/V2L-EVK

Note:

- Boot source selection
  - In this release, all boot flows use QSPI or eMMC as selected by SW11.
- CN10 (Carrier microSD slot)
  - CN10 is not a boot device.
  - CN10 can be used only for rootfs or data storage after the system has booted.
  - No SW1 configuration is required for CN10.
- CN3 (SoM microSD slot)
  - BL2/FIP cannot be loaded from CN3
  - When the board boots from QSPI or eMMC, CN3 is accessible in U-Boot/Linux (commonly as mmc device 0).
  - CN3 may be used as a rootfs or additional storage device if enabled via SW1-2.

1. **Prepare the microSD card**

   Flash the provided image to the microSD card using a user-friendly tool such as Balena Etcher, as described in section SD/MMC Card Flashing for a simplified flashing experience.

2. **Insert the microSD card**

   Insert the prepared SD card into CN10. This slot is for rootfs/data only; it cannot be used to load BL2/FIP.

3. **Configure the boot mode**

   In this section, we focus on QSPI boot as the default boot method. The switch configuration below loads BL2 and FIP from QSPI flash, with the root filesystem provided from eMMC or the SD card in slot CN10/CN3.

   Default setup (QSPI boot):

   - SW11 (on the carrier board): Set to QSPI boot.

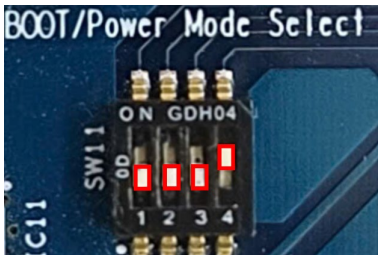


Figure 23. RZ/G2L-EVK & RZ/V2L-EVK QSPI boot mode

- SW1-2 (on the SoM): Controls access to storage devices on the SoM (Optional)
  - OFF: Enables access to the onboard eMMC
  - ON: Enables access to the microSD slot CN3

Table 15. RZ/G2L-EVK & RZ/V2L-EVK SW1 Switch Mode Position

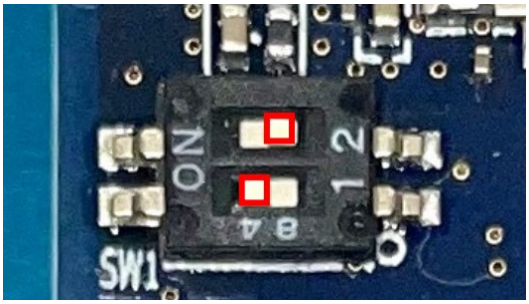


Figure 24. eMMC selection with normal operation

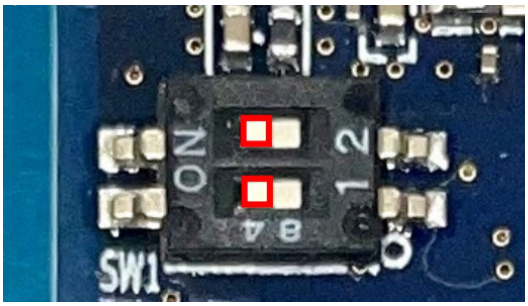


Figure 25. Enable the SD slot CN3 for normal operation

- Root filesystem: Use the SD card in slot CN10 (carrier board). CN10 remains accessible regardless of SW1-2.

Alternative setup (eMMC boot):

- SW11 (on the carrier board): Set to eMMC boot
- SW1-2 (on the SoM): Same as above for QSPI boot.

Refer to section **3.2.2** for the correct switch positions. In these modes, the board fetches BL2 and FIP from the chosen device.

4. **Connect peripherals**

   - Attach an HDMI display

5. **Connect UART**

   - Use a USB Type-A to Micro-USB Type-B cable to connect the board's UART console port to the host PC.

6. **Power on the board**

   - Connect the USB-C power supply (5V, 3A).
   - Press the red power button to turn on the board.
   - Wait for the UART serial device to appear on the host PC.
   - Open a terminal program on the detected serial port to monitor the boot log

Once powered on, the boot log should appear on the UART console, and the Weston desktop will display on the HDMI screen.

2.5.4 Complete Hardware Setup

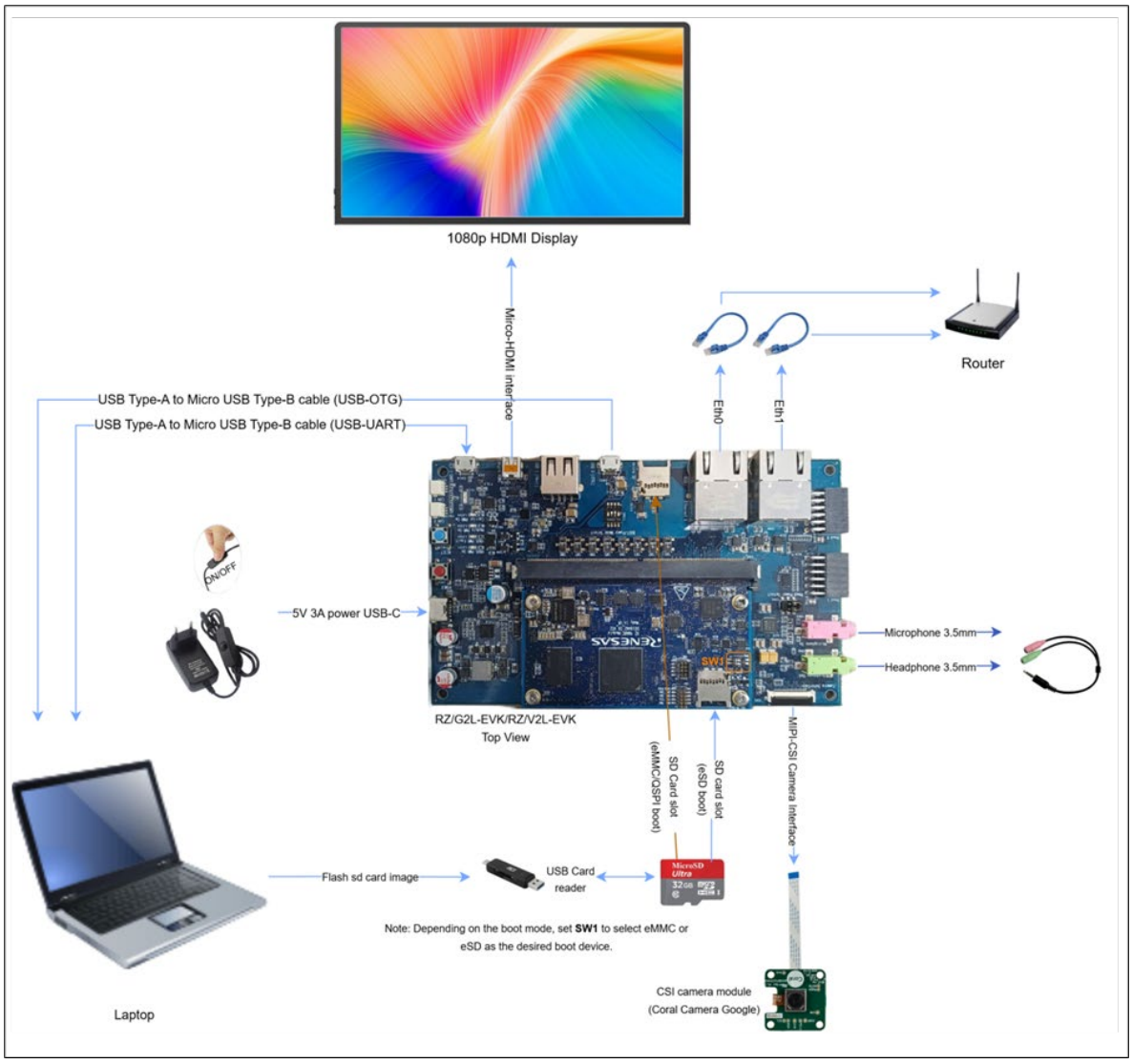


Figure 26. Complete setup for RZ/G2L-EVK & RZ/V2L-EVK

2.5.5 Booting

The boot process is straightforward:

1. Insert the prepared SD card into CN10.
2. Connect a keyboard, mouse, HDMI display, and other devices. Then connect the USB-C power supply and press the red power button to turn the board on.
3. The boot log should appear on the UART console, and the Weston desktop should display on the HDMI screen.
4. Open any of the available applications to interact with the system.

The image provided is feature-complete, offering a desktop-grade experience.

If a different boot device is required, such as QSPI, eMMC, or SD card, adjust the DIP switch settings as described in section **3.2.2** before powering on. The selected boot device determines where on-chip ROM code loads the Boot Loader stage 2 (BL2) and Firmware Image Package (FIP):

- QSPI boot loads BL2 and FIP from the onboard QSPI flash.

- eMMC boot loads BL2 and FIP from the onboard eMMC device.

Notes: The default firmware shipped on the board may not recognize new images. If the board fails to boot, update the firmware using the Serial Download Mode (SCIF) procedure described in section 3.1.3

2.5.5.1 Default Boot Behavior

When booting from QSPI or eMMC, the U-Boot environment defaults to using the SD card (CN10) as the root filesystem.

- The SD card on CN10 is always available as mmc1.
- The onboard eMMC is always available as mmc0.
- Unless reconfigured, Linux will mount the rootfs from CN10 (e.g., /dev/mmcblk1p1).

2.5.5.2 Using an eMMC Root Filesystem

The onboard eMMC may be used for:

- kernel/DTB only (root filesystem on SD/CN10),
- root filesystem only (kernel/DTB on SD),
- both kernel/DTB and root filesystem.

Steps:

1. Enable eMMC on the SoM: Set SW1-2 = OFF (see section 3.2.2 for SW1 description).
2. Prepare the eMMC rootfs. See section 3.3 for detailed information
3. Reboot the board. During startup, output similar to the following will appear:

```
NOTICE:  BL2: v2.9(release):<release-tag>
NOTICE:  BL2: Booting BL31
NOTICE:  BL31: v2.9(release): <release-tag>

U-Boot 2021.10 <Date>

CPU:   Renesas Electronics CPU rev 1.0
Model: <board-name>
DRAM:  1.9 GiB
MMC:   sd@11c00000: 0, sd@11c10000: 1
Loading Environment from SPIFlash... SF: Detected mt25qu512a ...
*** Warning - bad CRC, using default environment

Hit any key to stop autoboot:  0
=>
```

   - When the message “Hit any key to stop autoboot” appears, press Enter (or any key) to interrupt the countdown.
   - The => prompt indicates that we are in the U-Boot console.
4. Choose one configuration and apply the corresponding commands:

   Detect current environment

```
=> printenv mmcdev mmc_args
```

Device mapping

- mmcdev
  - mmcdev=0 → eMMC
  - mmcdev=1 → SD (CN10)
- mmc_args
  - root=/dev/mmcblk0p2 → root filesystem from eMMC (MMC device 0, partition 2)
  - root=/dev/mmcblk1p2 → root filesystem from SD card (CN10) (MMC device 1, partition 2)

Platform default (typical): kernel/DTB load from SD; rootfs points to /dev/mmcblk1p2 = SD, partition 2.

```
=> printenv mmcdev mmc_args
mmcdev=1
mmc_args=setenv bootargs rw rootwait earlycon root=/dev/mmcblk1p2
```

Choose one of the following configurations

- Root filesystem on eMMC (kernel, DTB from SD/CN10)

```
=> setenv mmcdev 1
=> setenv mmc_args 'setenv bootargs rw rootwait earlycon root=/dev/mmcblk0p2'
=> saveenv
=> boot
```

- Kernel/DTB on eMMC (root filesystem on SD/CN10)

```
=> setenv mmcdev 0
=> setenv mmc_args 'setenv bootargs rw rootwait earlycon root=/dev/mmcblk1p2'
=> saveenv
=> boot
```

- Kernel/DTB and root filesystem on eMMC

```
=> setenv mmcdev 0
=> setenv mmc_args 'setenv bootargs rw rootwait earlycon root=/dev/mmcblk0p2'
=> saveenv
=> boot
```

2.5.5.3 Using CN3 (eSD slot) as Root Filesystem

eSD boot from CN3 is supported in this release. The board can load the boot components from the SD card inserted in the CN3 slot, and CN3 can also be used as the root file system device in Linux.

Steps to use CN3 as the root filesystem:

1. Insert the SD card into the CN3 slot
2. Enable eSD on the SoM: Set SW1-2 = ON (see section 3.2.2 for SW1 description.
3. Reboot the board. During startup, output similar to the following will appear:

```
NOTICE:  BL2: v2.9(release):<release-tag>
NOTICE:  BL2: Booting BL31
NOTICE:  BL31: v2.9(release): <release-tag>

U-Boot 2021.10 <Date>

CPU:   Renesas Electronics CPU rev 1.0
Model: <board-name>
DRAM:  1.9 GiB
MMC:   sd@11c00000: 0, sd@11c10000: 1
Loading Environment from SPIFlash... SF: Detected mt25qu512a ...
*** Warning - bad CRC, using default environment

Hit any key to stop autoboot:  0
=>
```

- When the message “Hit any key to stop autoboot” appears, press Enter (or any key) to interrupt the countdown.
- The => prompt indicates that the system is now in the U-Boot console.

4. Update the U-Boot environment by changing the default device from 1 (CN10) to 0 (CN3):

```
=> setenv mmcdev 0
=> setenv mmc_args 'setenv bootargs rw rootwait earlycon root=/dev/mmcblk0p2'
=> saveenv
=> boot
```

2.5.6 Known Hardware and Functional Limitations

1. No onboard Wi-Fi/Bluetooth

   These EVKs do not include an on-chip or onboard Wi-Fi/Bluetooth module. Wireless connectivity must be provided through external modules (e.g., USB or SDIO-based Wi-Fi dongles).

2.6 RZ/V2H-EVK

This section describes the hardware-specific setup and booting process for the RZ/V2H-EVK

2.6.1 Overview of Connectors

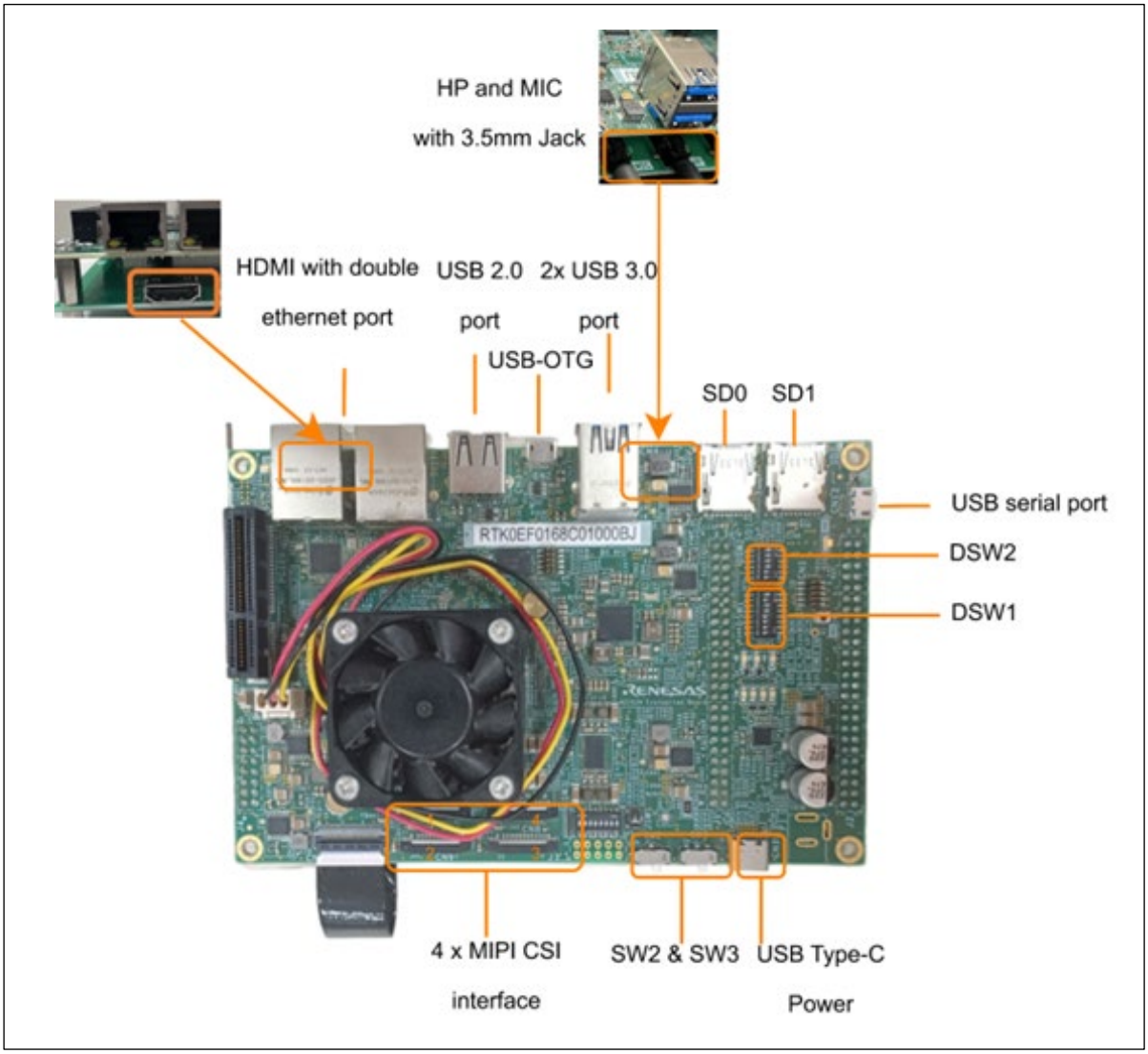


Figure 27. RZ/V2H-EVK Overview of Connectors

2.6.2 Hardware Requirements

The hardware setup consists of the following:

Table 16. Accessory List for RZ/V2H-EVK

Item	Required/ Optional	Exact accessory	Interface / Connector Type	Notes
Board	Required	RZ/V2H-EVK	—	Main target board
Storage media	Required	microSD card, minimum 8GB	microSD (default SD slot 0)	Used for Linux image

Debug cable	Required	USB Type-A to Micro USB Type-B cable	UART / debug serial (CN12)	—
Power supply	Required	Power supply that can provide up to 100W via USB-C PD	USB-C power	Required for power source
Display cable	Required for graphical images	HDMI to HDMI cable	HDMI	Required for HDMI display path
Display	Required for graphical images	1080p HDMI monitor	HDMI	Recommended for Weston / GUI images
CSI camera	Optional	OV5645 camera module	MIPI CSI	Required for camera usage
CSI converter	Optional	Camera conversion 22-pin to 24-pin FPC adapter	MIPI CSI	Required for camera connection
Ethernet cable	Optional / Recommended	Standard Ethernet cable	RJ45	Used for networking
USB keyboard	Optional	Standard USB keyboard	USB	Needed for local GUI use
USB mouse	Optional	Standard USB mouse	USB	Needed for local GUI use
Audio accessory	Optional	3.5mm headphone with microphone	Audio jack	Required for audio usage
Audio splitter	Optional	3.5mm Audio Stereo Y Splitter extension cable	Audio jack	Needed for separate mic/headphone use

2.6.3 Essential Hardware Setup and Booting

The figure below shows the basic essential hardware setup for RZ/V2H-EVK. Follow these steps to prepare and power on the board:

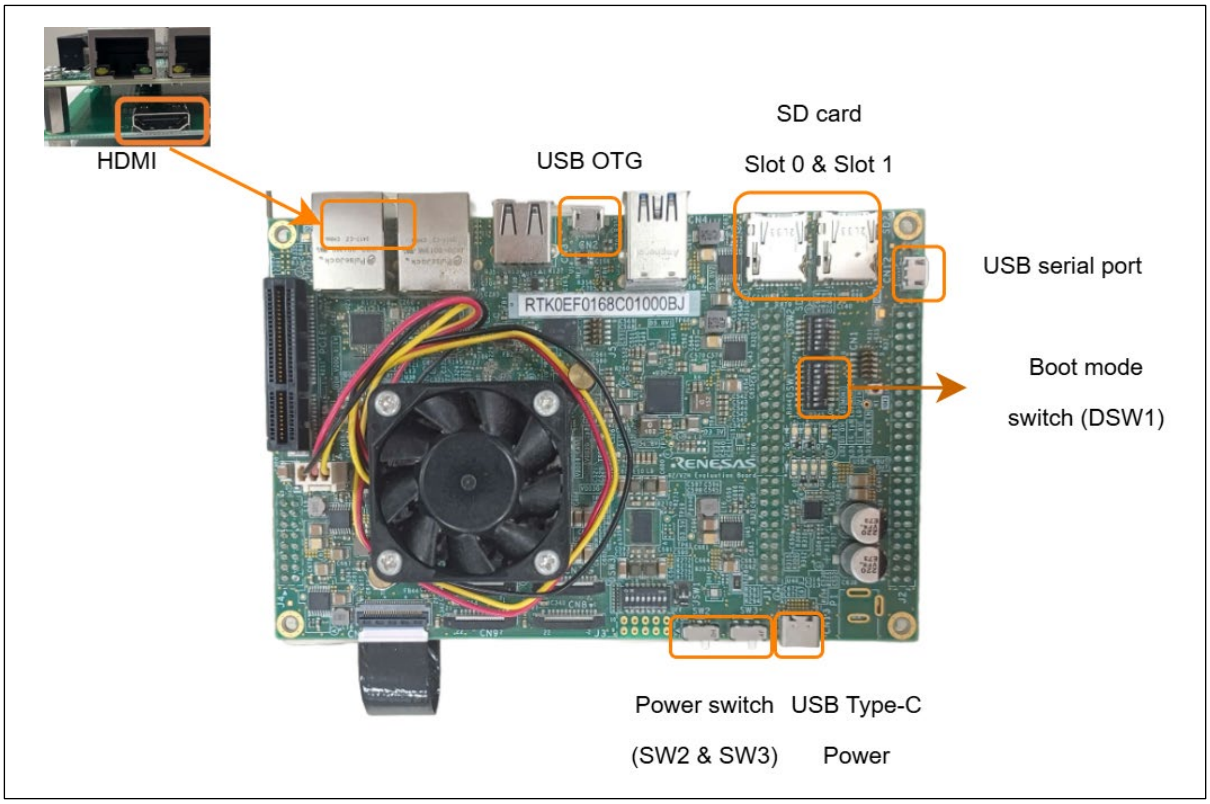


Figure 28. Essential minimum interfaces for RZ/V2H-EVK

1. **Prepare the microSD card**

   Flash the provided image to the microSD card using a user-friendly tool such as Balena Etcher, as described in section **2.1** for a simplified flashing experience.

   Insert the prepared microSD card into the SD card slot on the underside of the board.

2. **Configure the boot mode**

   For this release, configure DSW1 to boot from QSPI flash:

   - BL2 and FIP are loaded from QSPI.
   - Rootfs can be located on eMMC or SD card, as defined in the U-Boot environment.

   Set the DIP switches according to:

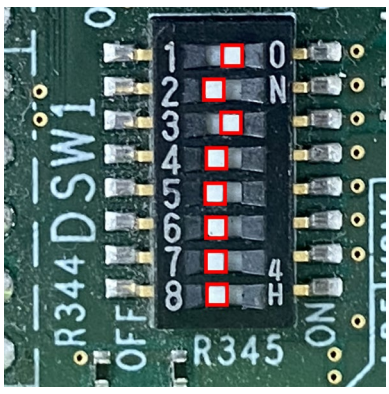


Figure 29. RZ/V2H-EVK xSPI boot mode

Note: Booting from eMMC and eSD is currently not supported in this release. These options may be enabled in future updates.

3. **Connect peripherals**
   - Attach the HDMI display, USB keyboard, and USB mouse.
4. **Connect UART**
   - Use a USB Type-A to micro-USB Type-B cable to connect the board's UART console port to the host PC.
5. **Power on the board**
   - Connect the USB-C power supply (100W).
   - Turn on SW3 first, then turn on SW2
   - Open a terminal program on the host PC to monitor the boot log.

Once powered on, the boot log should appear on the UART console, and the Weston desktop will display on the HDMI screen.

2.6.4 Complete Hardware Setup

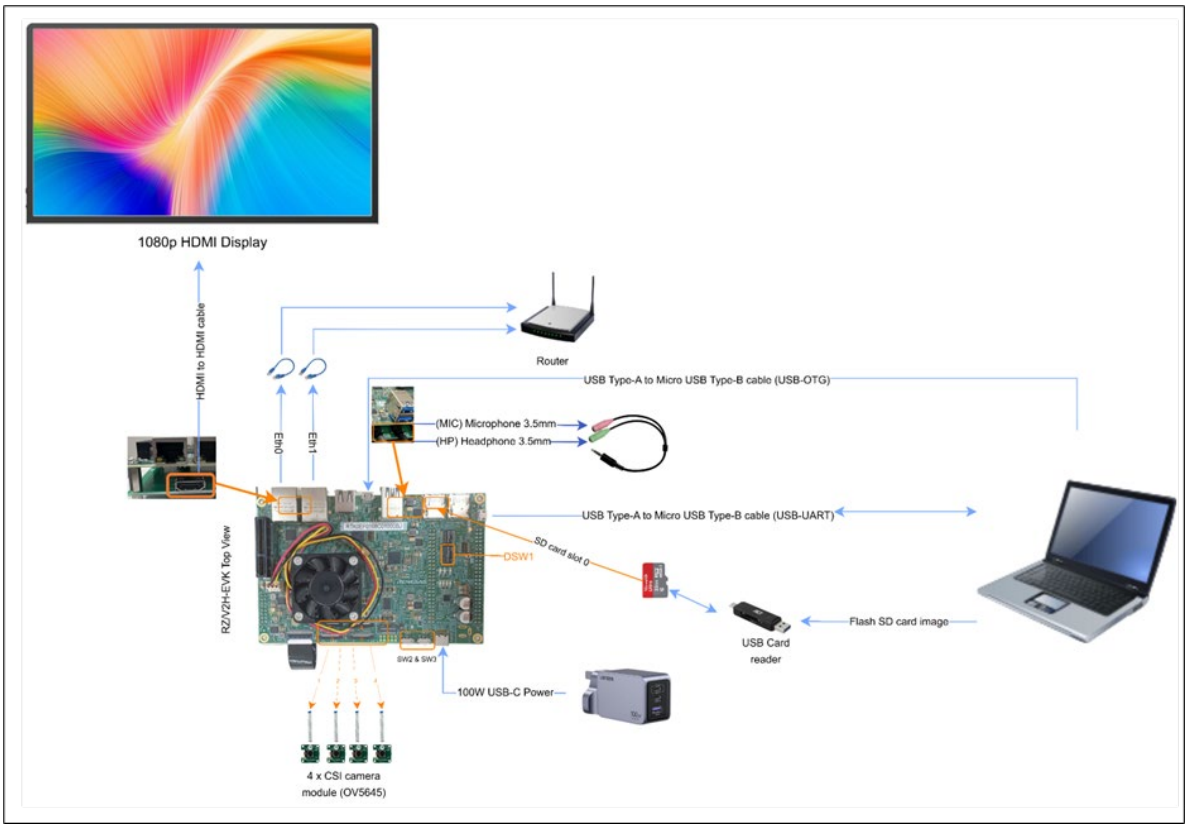


Figure 30. Complete setup for RZ/V2H-EVK board

Note: The RZ/V2H-EVK is available in two hardware versions, which differ in their storage options:

- **Version 1**: Equipped with two SD card slots and no onboard eMMC.
- **Version 2**: Equipped with one onboard eMMC (default boot device) and one SD card slot for alternate booting or data storage.

The complete setup shown above applies to Version 1. For Version 2, the setup procedure is the same as Version 1, with the only difference being the use of one eMMC and one SD card instead of two SD cards.

2.6.5 Booting

The boot process for RZ/V2H-EVK is very similar to RZ/G2L-EVK and RZ/V2L-EVK, as the boot mode must be selected using the DIP switches before power-on. Follow the steps below to boot the board:

1. Insert the SD card into the SD card slot (located on the underside of the carrier board; see the complete setup picture in the previous section).
2. Connect the keyboard, mouse, and HDMI display. Then connect the 100 W USB-C power supply and toggle the two power switches next to the USB-C port (SW2 and SW3)
3. The boot log will appear on the UART console, and the Weston desktop will display on the HDMI screen.
4. Launch any of the available applications to interact with the system.

The provided image is feature-complete and offers a desktop-grade user experience.

As with other RZ/G2L-EVK & RZ/V2L-EVK, the RZ/V2H-EVK requires selecting a boot device — such as xSPI, eMMC, or SD Card before powering on. Adjust the DIP switch settings as described in section **3.2.3** for detailed information.

Notes: The default firmware shipped on the board may not recognize new images. If the board fails to boot, update the firmware using the Serial Download Mode (SCIF) procedure described in section 3.1.4.

2.6.5.1 Power Switch Handling Precautions for RZ/V2H-EVK

Use caution when configuring the RZ/V2H-EVK power switches. Incorrect switch operation may damage the board or connected devices.

The RZ/V2H requires a defined power-on and power-off sequence. When using the RZ/V2H-EVK, follow these rules:

Power on

1. Confirm that power slide switches SW2 and SW3 are OFF.
2. Connect the USB Type-C cable to the Type-C connector (CN13). Do not connect the USB Type-C cable while SW2/SW3 are ON.
3. Turn SW3 ON
4. Turn SW2 ON

Power off

- Perform the software-controlled power-off sequence before switching off hardware power.

- After the software power-off sequence completes, turn SW2 OFF, then turn SW3 OFF, then disconnect the USB Type-C cable from CN13.
- Do not remove the USB Type-C cable while SW2/SW3 are ON, as this may cause device damage.

2.6.5.2 Default Boot Behavior

When booting from QSPI, the U-Boot environment defaults to using the SD card.

RZ/V2H-EVK hardware variants:

- Version 1 — SD-only (two SD slots, no eMMC): default boot device is SD0.
- Version 2 — SD + eMMC: default boot device is the SD card; it can be reconfigured to boot from eMMC if required.

2.6.5.3 Selecting the Root Filesystem Device

The RZ/V2H-EVK has two hardware variants, and the MMC device mapping differs depending on the variant. Select the root filesystem device according to the target board version.

Steps:

1. Prepare the rootfs:
   - For RZ/V2H-EVK Version 1, prepare an SD1 card by following the procedure in SD/MMC Card Flashing to flash the required image to another SD card.
   - For RZ/V2H-EVK Version 2, prepare a root filesystem on eMMC; refer to Prepare the eMMC root filesystem for detailed instructions.
2. Reboot the board. During startup, output similar to the following will appear:

```
NOTICE:  BL2: v2.9(release):<release-tag>
NOTICE:  BL2: Booting BL31
NOTICE:  BL31: v2.9(release): <release-tag>

U-Boot 2021.10 <Date>

CPU:   Renesas Electronics CPU rev 1.0
Model: <board-name>
DRAM:  1.9 GiB
MMC:   sd@11c00000: 0, sd@11c10000: 1
Loading Environment from SPIFlash... SF: Detected mt25qu512a ...
*** Warning - bad CRC, using default environment

Hit any key to stop autoboot:  0
=>
```

   - When the message “Hit any key to stop autoboot” appears, press Enter (or any key) to interrupt the countdown.
   - The => prompt indicates that we are in the U-Boot console.
3. Choose one configuration and apply the corresponding commands:

   Check the current environment

```
=> printenv mmcdev mmc_args
```

Table 17. RZ/V2H-EVK board variant mmcdev mapping

Board version	mmcdev=0	mmcdev=1	Default configuration
RZ/V2H-EVK version 1	SD0 card	SD1 card	SD0
RZ/V2H-EVK version 2	eMMC	SD1 card	SD1

Select the device used to load the kernel image and DTB by setting mmcdev.

```
=> setenv mmcdev <device_number>
```

For example, to load from mmc1, run the following command

```
=> setenv mmcdev 1
```

Use the following table to select the root filesystem device.

Table 18. RZ/V2H-EVK board variant mmc_args mapping

Board version	Root filesystem location	Settings
RZ/V2H-EVK version 1	SD0, partition 2	root=/dev/mmcblk0p2
RZ/V2H-EVK version 1	SD1, partition 2	root=/dev/mmcblk1p2
RZ/V2H-EVK version 2	eMMC, partition 2	root=/dev/mmcblk0p2
RZ/V2H-EVK version 1	SD1, partition 2	root=/dev/mmcblk1p2

Update mmc_args so that the root= value points to the required root filesystem device.

Generic command format:

```
=> setenv mmc_args 'setenv bootargs rw rootwait earlycon root=/dev/mmcblk<X>p2'
```

For example, on RZ/V2H-EVK Version 2, to use the root filesystem from eMMC, use:

```
=> setenv mmc_args 'setenv bootargs rw rootwait earlycon root=/dev/mmcblk0p2'
```

2.6.6 Known Hardware and Functional Limitations

1. No on-board Wi-Fi/Bluetooth:

   These EVKs do not include an on-chip or on-board Wi-Fi/Bluetooth module. Wireless connectivity must be provided through external modules (e.g., USB or SDIO-based Wi-Fi dongles).
2. eMMC boot is not supported in this release.

   The RZ/V2H board exists in two hardware versions, one of which does not include eMMC. As a result, eMMC flash support was not provided

2.7 RZ/V2H-RDK

This section describes the hardware-specific setup and booting process for the RZ/V2H-RDK

2.7.1 Overview of Connectors

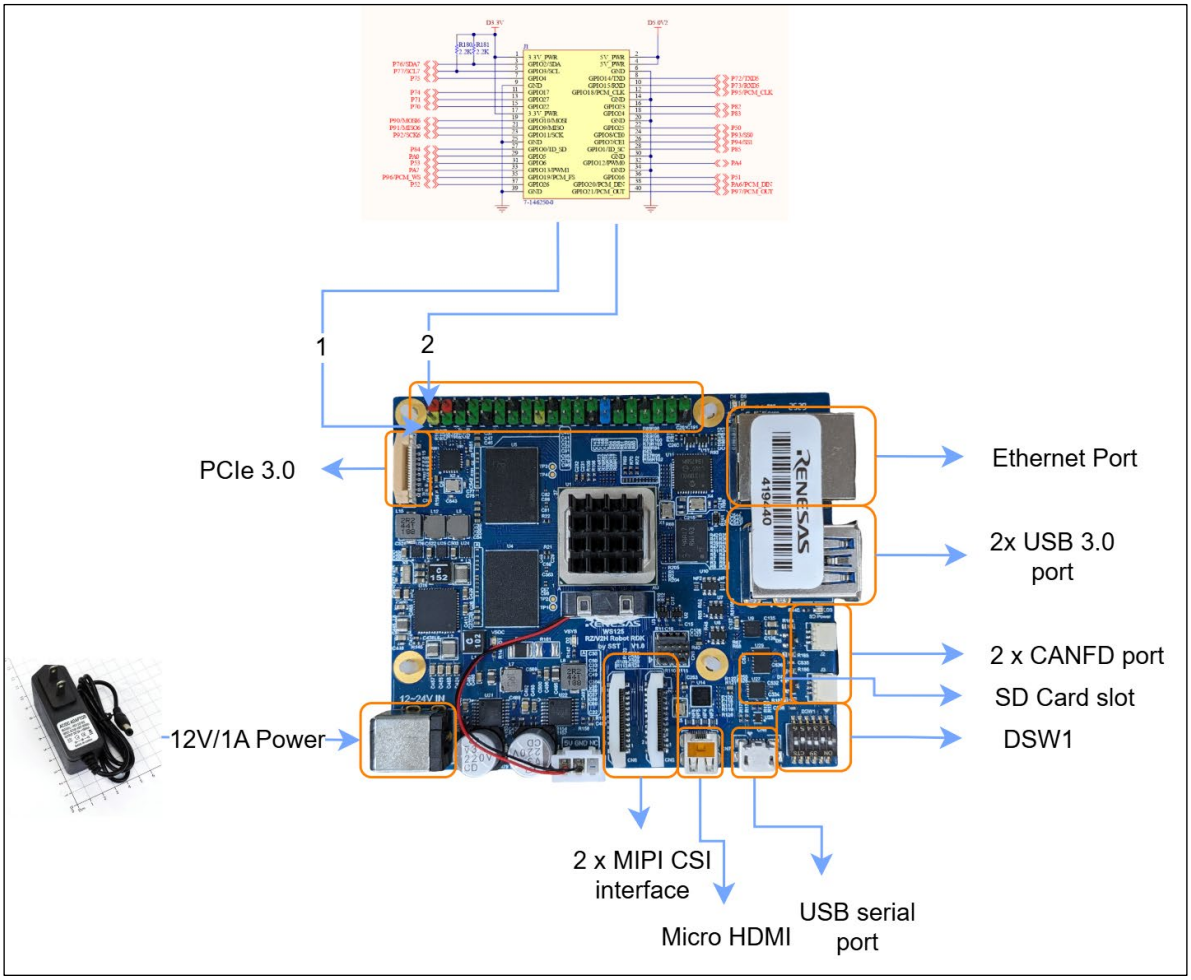


Figure 31. RZ/V2H-RDK - Overview of Connectors

2.7.2 Hardware Requirements

The hardware setup consists of the following:

Table 19. Accessory List for RZ/V2H-RDK

Item	Required/ Optional	Exact accessory	Interface / Connector Type	Notes
Board	Required	RZ/V2H-RDK	—	Main target board
Storage media	Required	microSD card, minimum 8GB	microSD	Used for Linux image

Debug cable	Required	USB Type-A to Micro USB Type-B cable	UART / debug serial (CN12)	—
Power supply	Required	Power supply, DC power supply: 12 V, ≥ 1 A (12 W or higher), regulated (AC input 100–240 V)	DC jack	Required for power source
Display cable	Required for graphical images	micro-HDMI to HDMI cable	HDMI	Required for HDMI display path
Display	Required for graphical images	1080p HDMI monitor	HDMI	Recommended for Weston / GUI images
CSI camera	Optional	OV5645 camera module	MIPI CSI	Required for camera usage
CSI converter	Optional	Camera conversion 22-pin to 24-pin FPC adapter	MIPI CSI	Required for camera connection
Ethernet cable	Optional / Recommended	Standard Ethernet cable	RJ45	Used for networking
USB keyboard	Optional	Standard USB keyboard	USB	Needed for local GUI use
USB mouse	Optional	Standard USB mouse	USB	Needed for local GUI use
Audio accessory	Optional	3.5mm headphone with microphone	Audio jack	Required for audio usage

2.7.3 Essential Hardware Setup and Booting

The figure below shows the basic essential hardware setup for RZ/V2H-RDK. Follow these steps to prepare and power on the board:

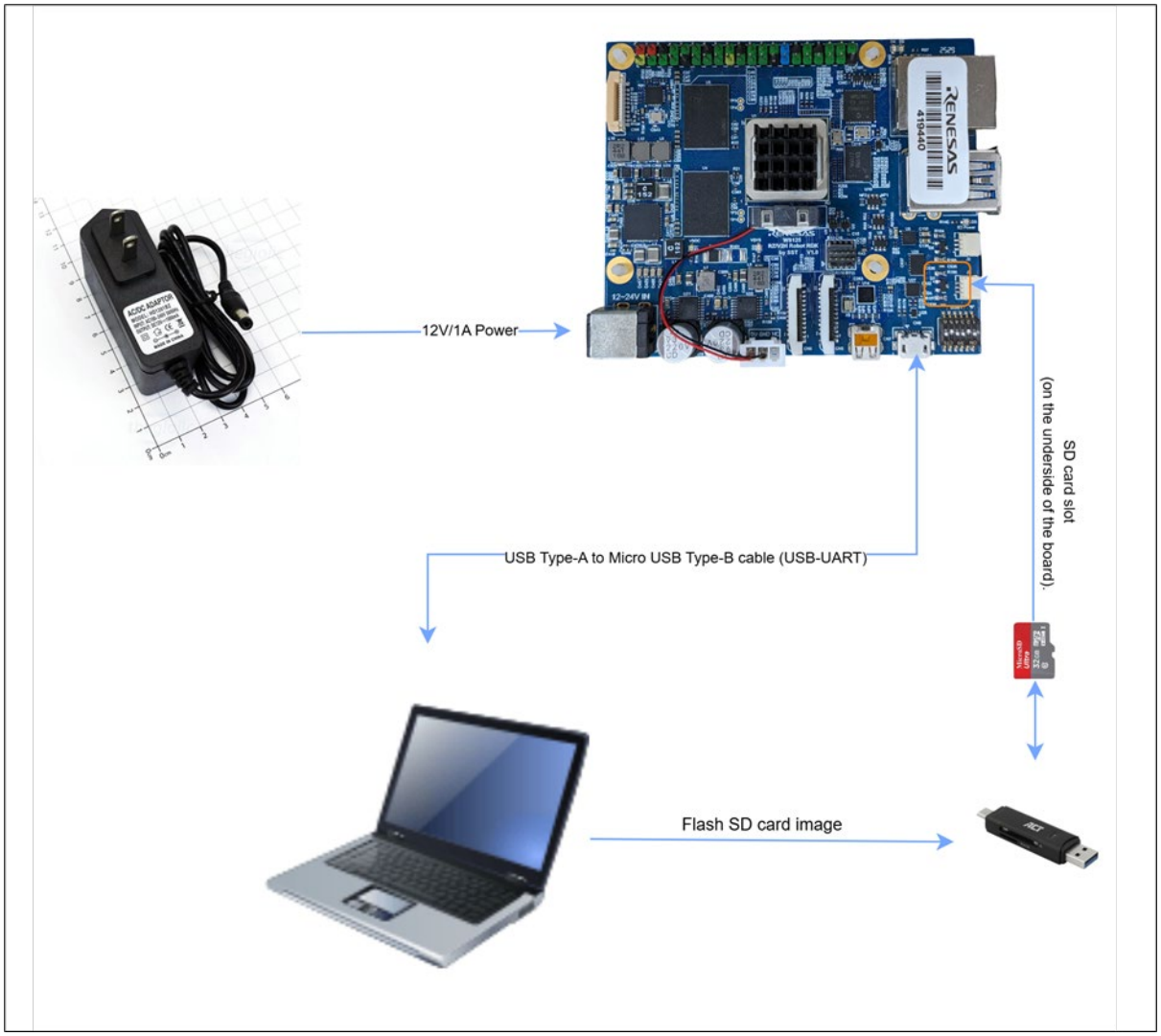


Figure 32. Essential minimum interfaces for RZ/V2H-RDK

1. **Prepare the microSD card**

   Flash the provided image to the microSD card using a user-friendly tool such as Balena Etcher, as described in Section SD/MMC Card Flashing for a simplified flashing experience.

   Insert the prepared microSD card into the SD card slot on the underside of the board.

2. **Configure the boot mode**

   For this release, configure DSW1 to boot from QSPI flash:

   - BL2 and FIP are loaded from QSPI.
   - Rootfs can be located on eMMC or SD card, as defined in the U-Boot environment.

   Set the DIP switches according to:

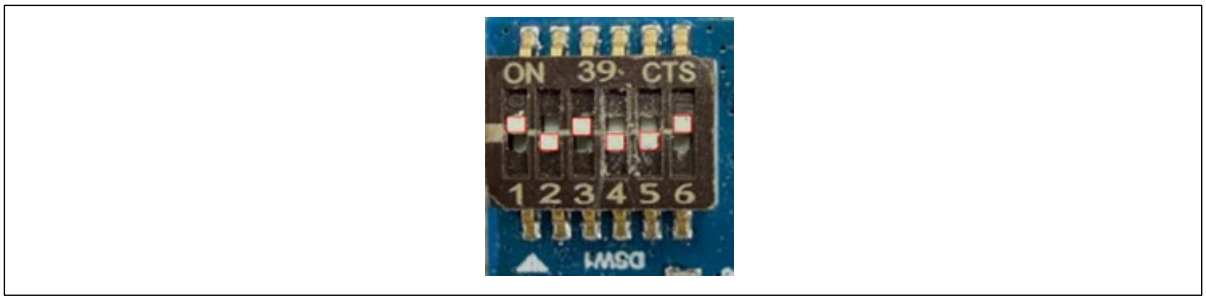


Figure 33. RZ/V2H-RDK xSPI boot switch

3. **Connect peripherals**
   - Attach the HDMI display, USB keyboard, and USB mouse.
4. **Connect UART**
   - Use a USB Type-A to micro-USB Type-B cable to connect the board's UART console port to the host PC.
5. **Power on the board**
   - Connect the 12V/1A power supply.
   - Open a terminal program on the host PC to monitor the boot log.

Once powered on, the boot log should appear on the UART console, and the Weston desktop will display on the HDMI screen.

2.7.4 Complete Hardware Setup

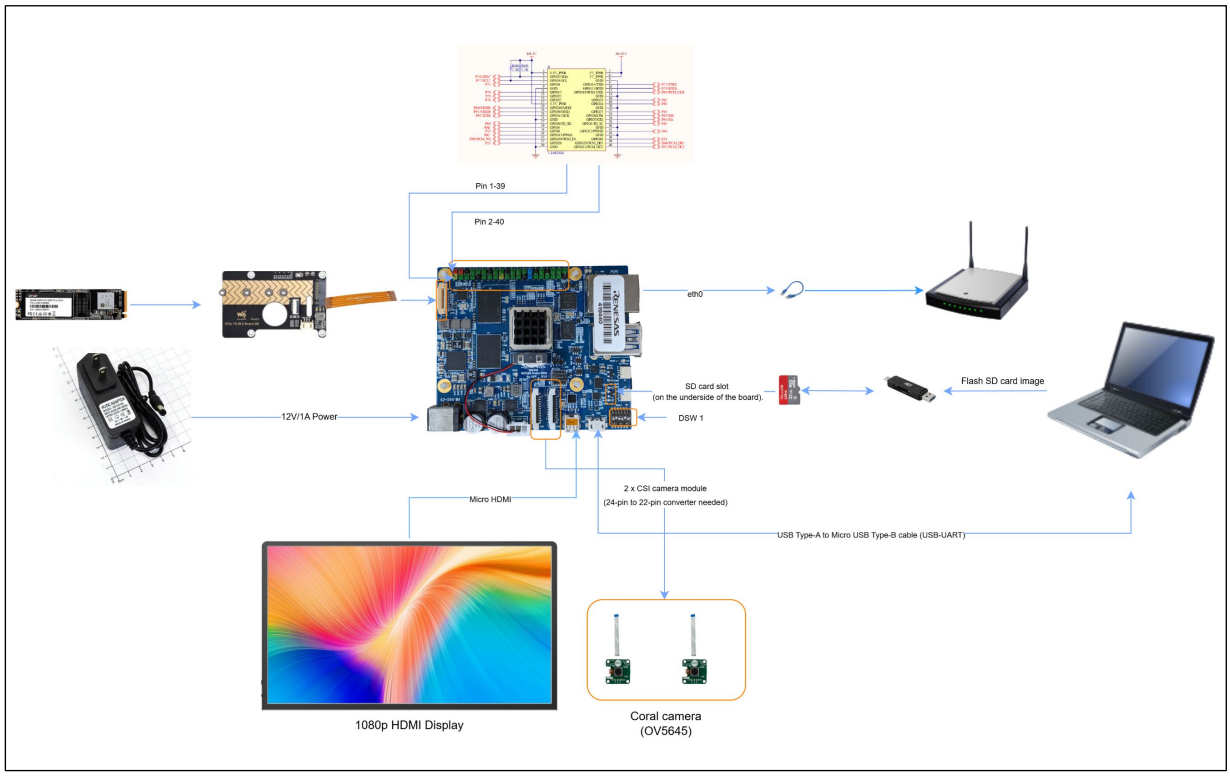


Figure 34. Complete setup for RZ/V2H-RDK board

2.7.5 Booting

The boot process for RZ/V2H-RDK is very similar to RZ/G2L-EVK, RZ/V2L-EVK, and RZ/V2H-EVK, as the boot mode must be selected using the DIP switches before power-on. Follow the steps below to boot the board:

1. Insert the SD card into the SD card slot.
2. Connect the keyboard, mouse, and HDMI display. Then connect the power supply.
3. The boot log will appear on the UART console, and the Weston desktop will display on the HDMI screen.
4. Launch any of the available applications to interact with the system.

The provided image is feature-complete and offers a desktop-grade user experience.

As with other RZ/G2L-EVK & RZ/V2L-EVK and RZ/V2H-EVK, the RZ/V2H-RDK requires selecting a boot device — such as xSPI, eMMC, or SD Card before powering on. Adjust the DIP switch settings as described in Section 3.2.4

Note: The default firmware shipped on the board may not recognize new images. If the board fails to boot, update the firmware using the Serial Download Mode (SCIF) procedure described in section 3.1.4

2.7.5.1 Default Boot Behavior

When booting from QSPI, the U-Boot environment defaults to using the SD card.

2.7.6 Known Hardware and Functional Limitations

1. No on-board Wi-Fi/Bluetooth:

   These EVKs do not include an on-chip or on-board Wi-Fi/Bluetooth module. Wireless connectivity must be provided through external modules (e.g., USB or SDIO-based Wi-Fi dongles).

2.8 IMDT V2H-SBC

This section describes the hardware-specific setup and booting process for the IMDT-V2H-SBC

2.8.1 Overview of Connectors

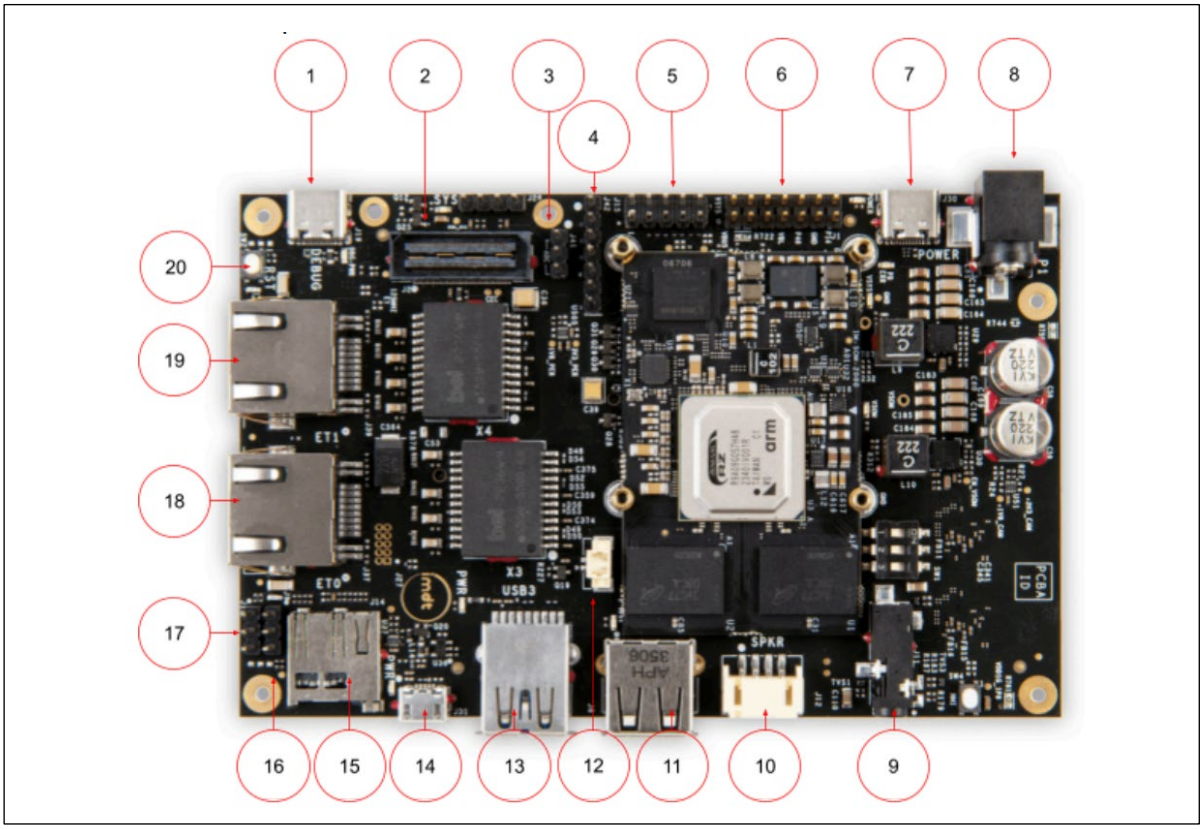


Figure 35. IMDT-V2H-SBC Top view

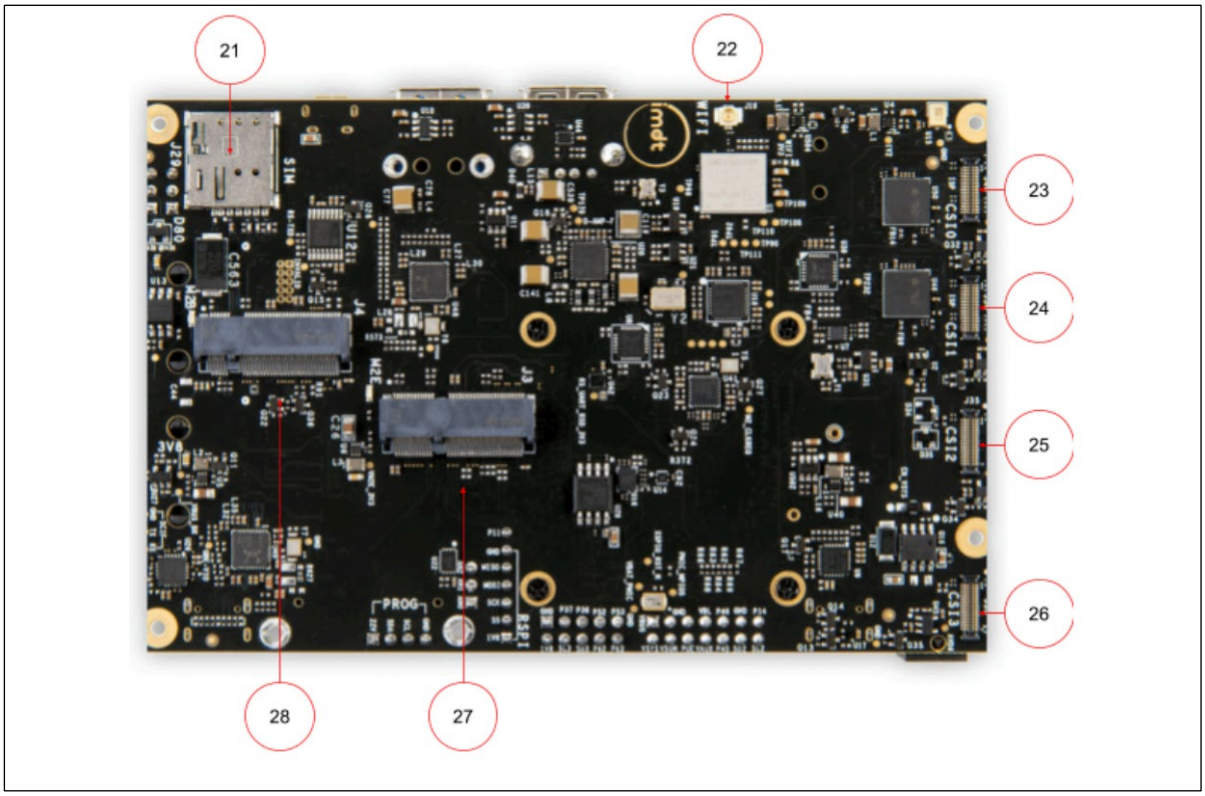


Figure 36. IMDT-V2H-SBC Bottom view

Table 20. Board Functionality Overview

Number	Functionality
1	DEBUG
2	Display
3	Analog Input
4	Auxiliary 1
5	Auxiliary 2
6	PSM (Power Supply Module)
7	Power In - USB-PD
8	Power In - DC Jack
9	3.5mm Audio Jack
10	Speaker Out
11	USB 2
12	Fan
13	USB 3
14	USB Micro-B
15	micro SD Card
16	RS-232
17	CAN
18	Ethernet 0
19	Ethernet 1
20	Reset Button
21	Nano SIM Card
22	WiFi Antenna
23	CSI 0 Connector
24	CSI 1 Connector
25	CSI 2 Connector
26	CSI 3 Connector
27	M.2 Key-E
28	M.2 Key-B

2.8.2 Hardware Requirements

The hardware setup consists of the following:

Table 21. Accessory List of IMDT-V2H-SBC

Item	Required/ Optional	Exact accessory	Interface / Connector Type	Notes
Board	Required	IMDT-V2H-SBC	—	Main target board
Storage media	Required	microSD card, minimum 8GB	microSD	Used for Linux image
Debug cable	Required	USB Type-A to Micro USB Type-C cable	UART / debug serial (J10)	—

Power supply	Required	Power supply DC9V/3A via USB-C PD (not included in the package) or DC Barrel jack with DC12V- 24V/2A minimum.	DC jack	Required for power source
Display cable	Required for graphical images	micro-HDMI to HDMI cable	HDMI	Required for HDMI display path
DSI to HDMI adapter	Required for graphical images	B-LCDAD-HDMI DSI to HDMI adapter	MIPI-DSI to HDMI	Converts the board's DSI display output to HDMI for an external monitor
Display	Required for graphical images	1080p HDMI monitor	HDMI	Recommended for Weston / GUI images
Ethernet cable	Optional / Recommended	Standard Ethernet cable	RJ45	Used for networking
USB keyboard	Optional	Standard USB keyboard	USB	Needed for local GUI use
USB mouse	Optional	Standard USB mouse	USB	Needed for local GUI use
Audio accessory	Optional	3.5mm headphone with microphone	Audio jack	Required for audio usage

2.8.3 Essential Hardware Setup and Booting

1. **Prepare the microSD card**

   Flash the provided image to the microSD card using a user-friendly tool such as Balena Etcher, as described in Section 2.1 for a simplified flashing experience.

   Insert the prepared microSD card into the SD card slot on the underside of the board.

2. **Configure the boot mode**

   Set the DIP switches according to:

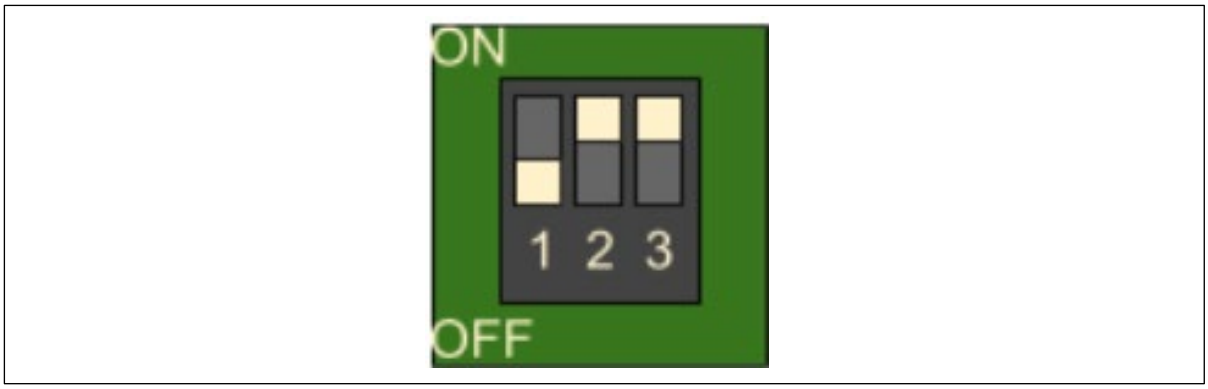


Figure 37. IMDT-V2H-SBC xSPI boot switch

- eMMC is not supported in this release.
- QSPI (xSPI) for the bootloader and an SD card for the kernel image.

3. **Connect peripherals**
   - Attach the DSI screen, USB keyboard, and USB mouse.
4. **Connect UART**
   - Use a USB Type-A to USB Type-C cable to connect the board's UART console port to the host PC.
   - Open a terminal program on the host PC to monitor the boot log.
5. **Power on the board**
   - The SBC can be powered in two ways:
     - DC Barrel Jack: 12–24V / 2A minimum
     - USB-C PD: 9V / 3A minimum

Once powered on, the boot log should appear on the UART console.

2.8.4 Complete Hardware Setup

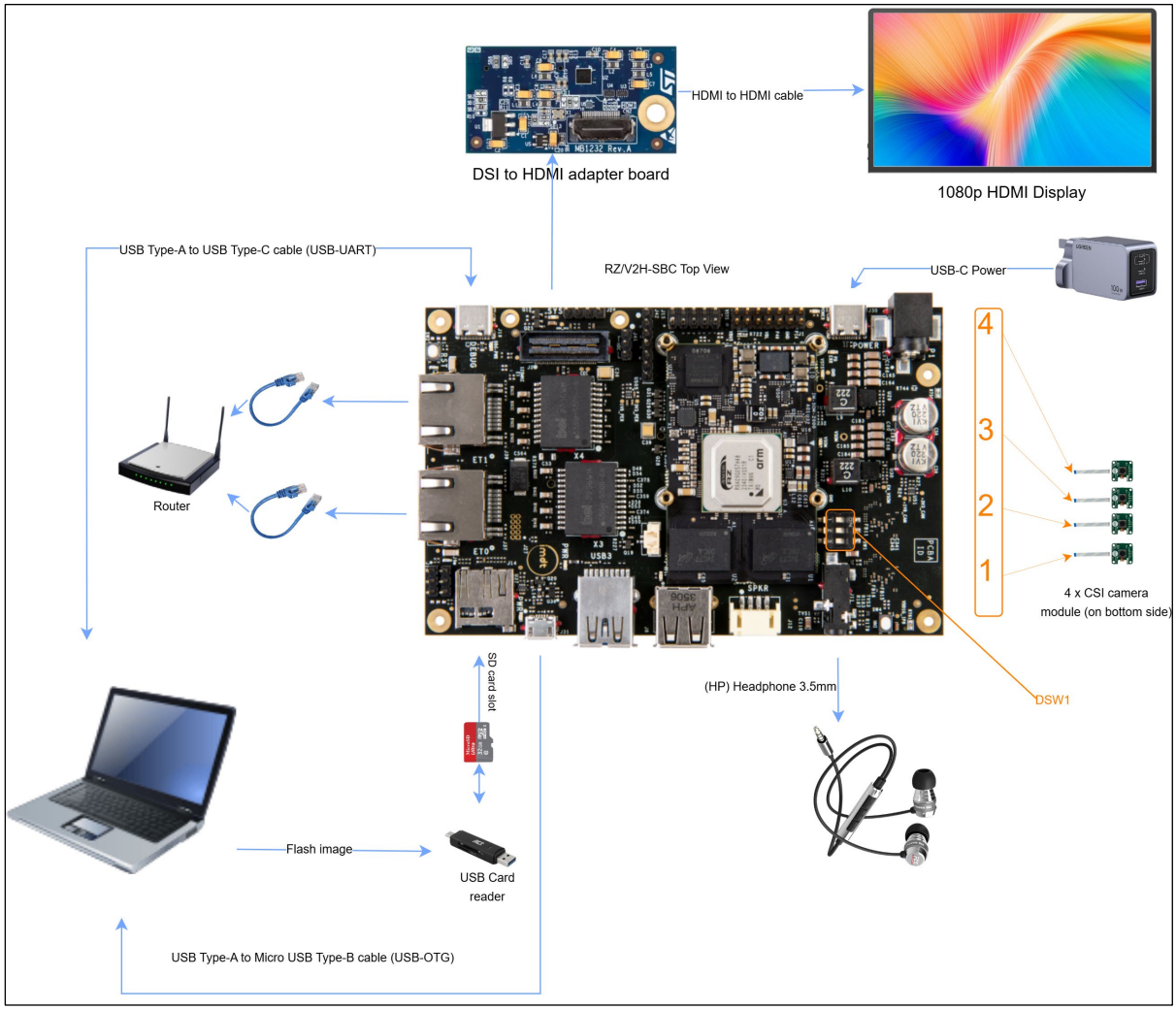


Figure 38. Complete setup for IMDT-V2H-SBC board

2.8.5 Booting

The boot process is straightforward, as the boot mode must be selected using the DIP switches before power-on. Follow the steps below to boot the board:

1. Insert the SD card into the SD card slot (located on the underside of the carrier board; see the complete setup picture in the previous section).
2. Connect the DSI screen, keyboard, and mouse. Then connect the USB-C power or DC barrel jack.
3. The boot log will appear on the UART console, and the Weston desktop will display on the HDMI screen.
4. Launch any of the available applications to interact with the system.

The provided image is feature-complete and offers a desktop-grade user experience.

Before powering the IMDT V2H SBC, set the DIP switches to select the desired boot device (for example, xSPI, eMMC). The board reads these switch positions at power-up to determine the boot source; changing switches while the board is powered may be ignored or cause undefined behavior. Refer to section **3.2.5** for the exact switch map.

Note: The default firmware shipped on the board may not recognize new images. If the board fails to boot, update the firmware using the Serial Download Mode (SCIF) procedure described in section 3.1.5

2.8.5.1 Default Boot Behavior

When booting from QSPI, U-Boot can use either the SD card or eMMC as the root filesystem.

- The onboard eMMC is always available as mmc0.
- The SD card is always available as mmc1.
- Unless reconfigured, Linux will mount the rootfs from the SD Card (e.g., /dev/mmcblk1p1).

2.8.5.2 Using an eMMC Root Filesystem

The onboard eMMC may be used for:

- kernel/DTB only (root filesystem on SD card),
- root filesystem only (kernel/DTB on SD card),
- both kernel/DTB and root filesystem.

Steps:

1. Prepare the eMMC rootfs. Refer to section **3.3** for detailed information
2. Reboot the board. During startup, output similar to the following will appear:

```
NOTICE:  BL2: v2.9(release):<release-tag>
NOTICE:  BL2: Booting BL31
NOTICE:  BL31: v2.9(release): <release-tag>

U-Boot 2021.10 <Date>

CPU:   Renesas Electronics CPU rev 1.0
Model: <board-name>
DRAM:  <ram-amount> GiB
MMC:   sd@11c00000: 0, sd@11c10000: 1
Loading Environment from SPIFlash... SF: Detected mt25qu512a ...

Hit any key to stop autoboot:  0
=>
```

   - When the message “Hit any key to stop autoboot” appears, press Enter (or any key) to interrupt the countdown.
   - The => prompt indicates that you are in the U-Boot console.
3. Choose one configuration and apply the corresponding commands:

   Detect current environment

```
=> printenv mmcdev mmc_args
```

   Device mapping
   - mmcdev
     - mmcdev=0 → eMMC
     - mmcdev=1 → SD card
   - mmc_args

- root=/dev/mmcblk0p2 → root filesystem from eMMC (MMC device 0, partition 2)
- root=/dev/mmcblk1p2 → root filesystem from SD card (MMC device 1, partition 2)

Platform default (typical): kernel/DTB load from SD; rootfs points to /dev/mmcblk1p2 = SD, partition 2.

```
=> printenv mmcdev mmc_args
mmcdev=1
mmc_args=setenv bootargs rw rootwait earlycon root=/dev/mmcblk1p2
```

Choose one of the following configurations

- Root filesystem on eMMC (kernel, DTB from SD card)

```
=> setenv mmcdev 1
=> setenv mmc_args 'setenv bootargs rw rootwait earlycon root=/dev/mmcblk0p2'
=> saveenv
=> boot
```

- Kernel/DTB on eMMC (root filesystem on SD card)

```
=> setenv mmcdev 0
=> setenv mmc_args 'setenv bootargs rw rootwait earlycon root=/dev/mmcblk1p2'
=> saveenv
=> boot
```

- Kernel/DTB and root filesystem on eMMC

```
=> setenv mmcdev 0
=> setenv mmc_args 'setenv bootargs rw rootwait earlycon root=/dev/mmcblk0p2'
=> saveenv
=> boot
```

2.8.6 Known Hardware and Functional Limitations

1. eMMC boot is not supported in this release.
2. IMDT-V2H-SBC UDP-based flashing: Ethernet operation in U-Boot may be unstable; if UDP-based flashing is required, Ethernet must be initialized in the Linux kernel first, and the system rebooted without disconnecting power; otherwise, OTG (default option) should be used for flashing.
3. Bluetooth: The on-board Bluetooth is non-operational. The Bluetooth module is not documented in the schematic; therefore, Bluetooth support is not currently developed. This feature may be implemented in a future release.
4. DSI and HDMI display functionality is currently unverified on the IMDT V2H-SBC platform and will be validated in a future release.

2.9 Boot Notice

On every reboot, U-Boot imports the fields listed in **Table 42. Auto-managed u-boot environment variables** and automatically resets the corresponding U-Boot environment variables to their default values. These variables are managed by the system and are reloaded at boot time.

If a custom environment is required, do not rely on one-off setenv/saveenv at the prompt—those changes will be overwritten on the next boot.

Running **env default -a** (reset environment to default) will clear all environment variables, including the board identification variables loaded at runtime. After running this command, the board must be rebooted for the board identification file to be loaded again and restore these auto-managed variables

3. Appendix

3.1 Factory Firmware Flashing Using Serial Downloader (SCIF) Mode

In most cases, the RZ boards are preloaded with the latest firmware. The preferred method of updating the firmware is through the SD card flashing method, as described in section **SD/MMC Card Flashing**.

However, there are cases where you might require the use of a serial downloader. This is more common in a factory environment where the boards are being programmed for the first time or in cases where the board is bricked.

This is considered hardware flashing because it requires the board to be put into the serial download mode (called SCIF mode) by altering the bootstrapping pins.

3.1.1 RZ/G2L-SBC

This section describes the firmware flashing process for the RZ/G2L-SBC board.

Note: The RZ/G2L-SBC does not have any interfaces on the main board to alter the boot mode. The bootstrapping pins are routed through the bottom connectors J12 & J13. Hence, the process requires the use of an adapter board, which is not included in the package.

3.1.1.1 Required Hardware

This flashing process requires the use of a boot mode change, which is achieved using an adapter board that is not included in the package.

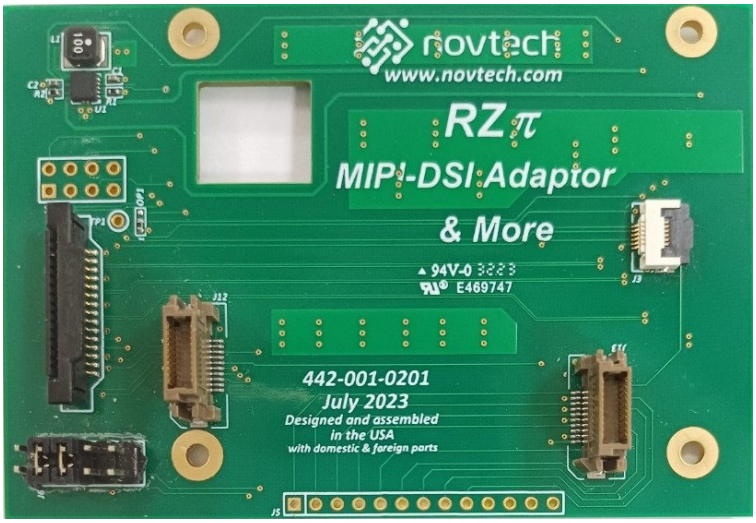


Figure 39. Adaptor board

After setting up the required hardware and configuring the boot mode through connectors J12 and J13 on the RZ/G2L-SBC, refer to section **2.2.2** for instructions on using the universal script to flash the firmware

When flashing completes, power off the board and restore the boot mode strapping on connectors J12 and J13 from SCIF Download Mode to the normal boot configuration for the RZ/G2L-SBC. Then power on the board again.

3.1.2 RS-G2L100

This section describes the firmware flashing process and boot mode configuration for each supported board. Firmware flashing is required to write bootloaders (BL2 and FIP) to the onboard flash memory. The process uses Renesas' Flash Writer tool and requires setting the board into SCIF Download Mode.

The RS-G2L100 features onboard DIP switches that allow direct selection of boot modes.

3.1.2.1 DIP Switch Settings

Use the DIP switch SW1 to configure the boot mode:

Table 22. SCIF Download Mode - RS-G2L100

Switch	SCIF Download Mode
SW1-1	OFF
SW1-2	ON
SW1-3	OFF
SW1-4	OFF

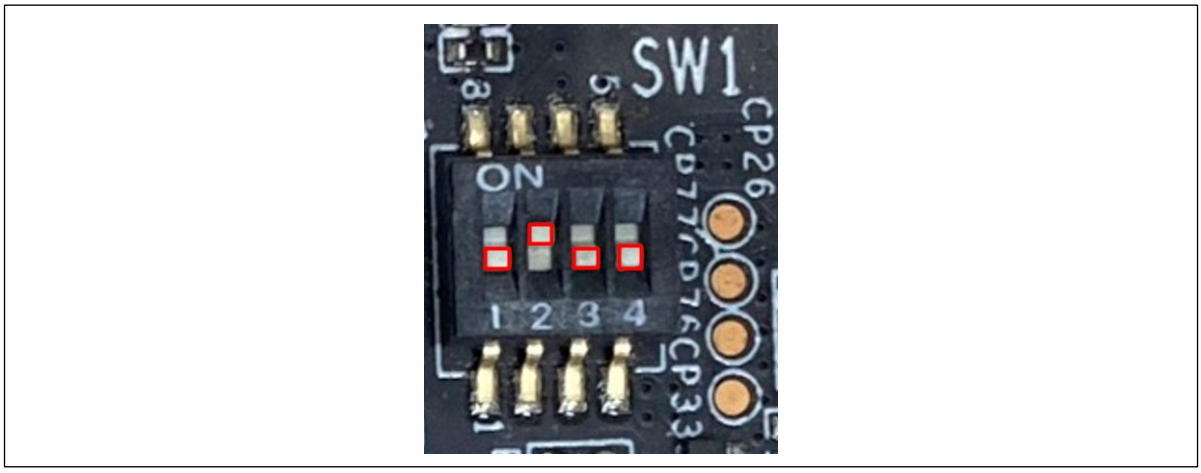


Figure 40. SW11 SCIF Download Mode - RS-G2L100

After setting up the required hardware and configuring the SCIF download mode for the RS-G2L100 or RS-G2L100, refer to section 2.2.2 for instructions on using the universal script to flash the firmware

When flashing completes, power off the board and restore the switch settings to the corresponding normal boot mode in Section 3.2.1 before powering on again

3.1.3 RZ/G2L-EVK and RZ/V2L-EVK

This section describes the firmware flashing process and boot mode configuration for each supported board. Firmware flashing is required to write bootloaders (BL2 and FIP) to the onboard flash memory. The process uses Renesas' Flash Writer tool and requires setting the board into SCIF Download Mode.

Unlike the RZ/G2L-SBC, which requires external hardware to configure boot mode, the RZ/G2L-EVK and RZ/V2L-EVK feature onboard DIP switches that allow direct selection of boot modes. This simplifies the flashing process and eliminates the need for manual signal strapping.

3.1.3.1 DIP Switch Settings

Use the DIP switch SW11 to configure the boot mode:

Table 23. SCIF Download Mode - RZ/G2L-EVK & RZ/V2L-EVK

Switch	SCIF Download Mode
SW11-1	OFF
SW11-2	ON
SW11-3	OFF
SW11-4	ON

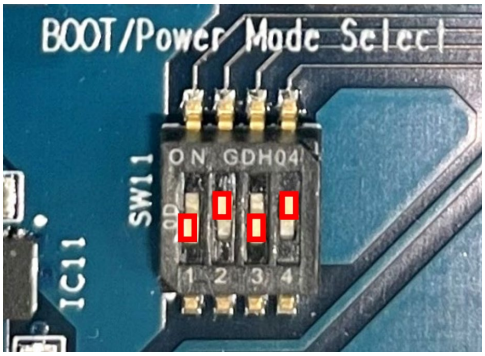


Figure 41. SW11 SCIF Download Mode - RZ/G2L-EVK & RZ/V2L-EVK

Use DIP switch 1 to select eMMC as the boot device:

Table 24. Select eMMC as the boot device

Switch	Select eMMC
SW1-1	ON
SW1-2	OFF

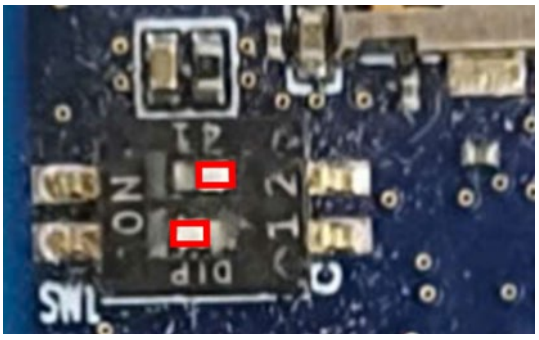


Figure 42. SW1 Settings for eMMC Boot - RZ/G2L-EVK & RZ/V2L-EVK

After setting up the required hardware and configuring the SCIF download mode for the RZ/G2L-EVK or RZ/V2L-EVK, refer to section 2.2.2 for instructions on using the universal script to flash the firmware

When flashing completes, power off the board and restore the switch settings to the corresponding normal boot mode in Section 3.2.2 before powering on again

3.1.4 RZ/V2H-EVK

Use the DIP switch DSW1 to configure the boot mode:

Table 25. SCIF Download Mode - RZ/V2H-EVK

Switch	Status	Function
DSW1-1	ON	Select the cold boot CPU - OFF: CM33

		- ON: CA55 (default)
DSW1-2	OFF	Input the CA55 frequency at the CA55 cold boot - [OFF: OFF]: 1.6GHz - [OFF: ON]: 1.7GHz (default) - [ON: OFF]: 1.1GHz - [ON: ON]: 1.5GHz
DSW1-3	ON	
DSW1-4	OFF	- [OFF: OFF]: xSPI - [OFF: ON]: SCIF - [ON: OFF]: SD (default) - [ON: ON]: eMMC
DSW1-5	ON	
DSW1-6	OFF	OFF: SSCG ON (default), ON: SSCG OFF
DSW1-7	OFF	OFF: Normal mode, ON: Debug mode
DSW1-8	OFF	Fixed to OFF

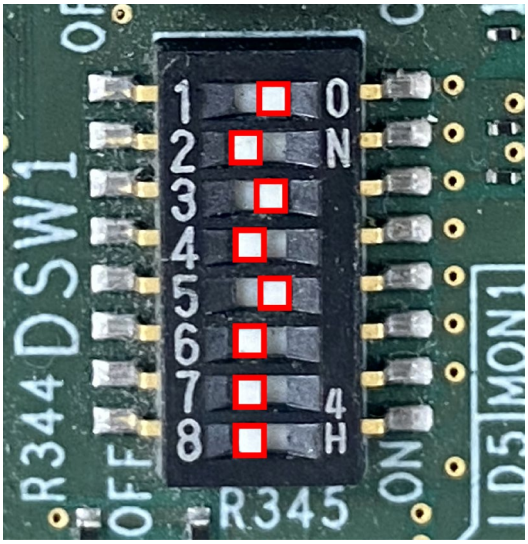


Figure 43. DSW1 - SCIF Boot Mode

To enable SCIF Download Mode, set DSW1-4 and DSW1-5 according to the SCIF configuration in the Table 25. SCIF Download Mode- RZ/V2H-EVK

Other switches (DSW1-1, DSW1-2, DSW1-3, DSW1-6, and DSW1-7) should remain in their default positions unless you need to change CPU selection, boot frequency, SSCG, or debug mode as described in the table.

After setting up the required hardware and configuring the SCIF download mode for the RZ/V2H-EVK and RZ/V2H-RDK, refer to section 2.2.2 for instructions on using the universal script to flash the firmware

When flashing completes, power off the board and restore the switch settings to the corresponding normal boot mode in Section 3.2.3 before powering on again

3.1.5 RZ/V2H-RDK

RZ/V2H-RDK also uses DIP switch DSW1 to configure the boot mode, but unlike RZ/V2H-EVK, it has fewer switches. To enable SCIF Download Mode, set DSW1-4 to OFF and DSW1-5 to ON as shown below. The other switches should remain in their default positions

Table 26. DSW1 Switch - RZ/V2H-RDK

Switch	Default settings	Function
DSW1-1	ON	Select the cold boot CPU - OFF: CM33 - ON: CA55 (default)
DSW1-2	OFF	Input the CA55 frequency at the CA55 cold boot - [OFF: OFF]: 1.6GHz - [OFF: ON]: 1.7GHz (default) - [ON: OFF]: 1.1GHz - [ON: ON]: 1.5GHz
DSW1-3	ON	
DSW1-4	OFF	- [OFF: OFF]: xSPI - [OFF: ON]: SCIF - [ON: OFF]: SD (default) - [ON: ON]: eMMC
DSW1-5	ON	
DSW1-6	OFF	Select JTAG debug mode - OFF: normal mode (default) - ON: JTAG

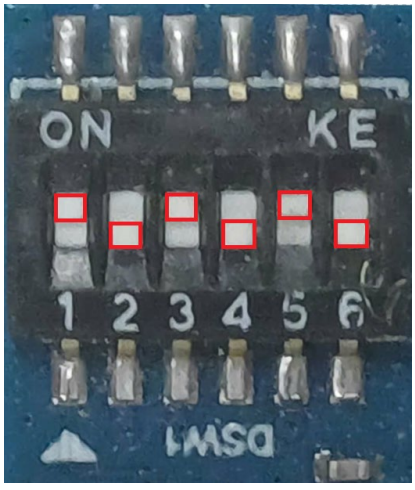


Figure 44. RZ/V2H-RDK - DSW1 SCIF Download Mode

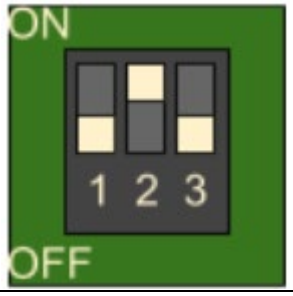
After configuring the DIP switches for SCIF Download Mode for RZ/V2H-RDK, refer to Section 2.2.2 for instructions on using the universal script to flash the firmware.

When flashing completes, power off the board and restore the switch settings to the corresponding normal boot mode in Section 3.2.4 before powering on again

3.1.6 IMDT V2H-SBC

Use the DIP switch DSW1 to configure the boot mode:

Table 27. SCIF Download Mode - IMDT-V2H-SBC

DSW1-Switch	Behavior
	In this configuration, the board is placed in SCIF Download mode. This mode is used for programming the bootloader into the xSPI Flash or the onboard eMMC over USB Serial.

To enable SCIF Download Mode, set DSW1-1, DSW1-2, and DSW1-3 according to the SCIF configuration in the Table 27. SCIF Download Mode- IMDT-V2H-SBC.

After setting up the required hardware and configuring the SCIF download mode for the IMDT V2H-SBC, refer to section 2.2.2 for instructions on using the universal script to flash the firmware.

3.2 Boot Mode Reference (Non-SCIF)

This section summarizes the switch/strap settings required for normal boot and boot-device selection across supported boards. Use these settings after completing factory flashing or when switching the boot device during bring-up.

3.2.1 RS-G2L100

The RS-G2L100 provides on-board DIP switches for boot mode selection.

The settings below tell the BootROM which device to read the initial firmware from, i.e., where to fetch BL2 (and Boot Parameter, if used) and subsequently the FIP.

Use these settings after factory flashing to boot from the programmed device.

Table 28. SW1 – Boot Device Selection (Normal Boot)

Boot device	SW1-1	SW1-2	SW1-3	SW1-4	Description
eMMC	OFF	ON	ON	OFF	Boot from on-board eMMC (BootROM loads BL2/BL2+BP from eMMC, then FIP).
QSPI	OFF	OFF	ON	OFF	Boot from QSPI NOR flash

Table 29. RS-G2L100 SW1 Switch Mode Position

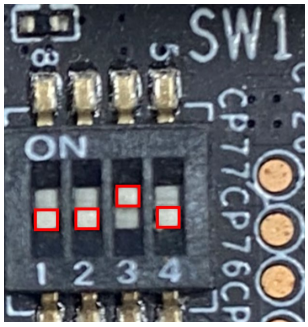


Figure 45. SW1 – QSPI Boot Mode

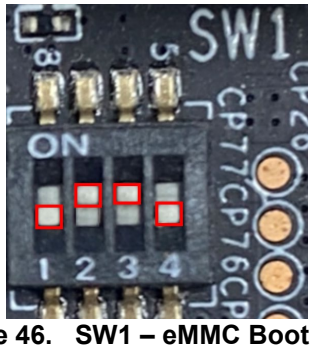


Figure 46. SW1 – eMMC Boot Mode

3.2.2 RZ/G2L-EVK & RZ/V2L-EVK

These EVKs provide on-board DIP switches for boot mode and boot device selection.

The settings below tell the BootROM which device to read the initial firmware from, i.e., where to fetch BL2 (and Boot Parameter, if used) and subsequently the FIP.

Use these settings after factory flashing to boot from the programmed device.

Table 30. SW11 – Boot Device Selection (Normal Boot)

Boot device	SW11-1	SW11-2	SW11-3	SW11-4	Description
eMMC	ON	OFF	OFF	ON	Boot from on-board eMMC (BootROM loads BL2/BL2+BP from eMMC, then FIP).
QSPI	OFF	OFF	OFF	ON	Boot from QSPI NOR flash
SD / eSD	ON	ON	OFF	ON	Boot from SD/eSD card (slot media)

Table 31. RZ/G2L-EVK & RZ/V2L-EVK SW11 Switch Mode Position



Figure 47. SW11 - eSD Boot Mode

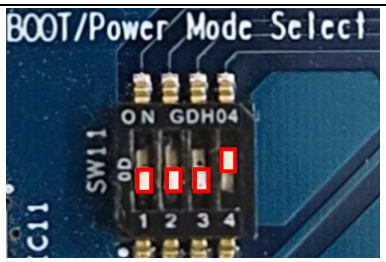


Figure 48. SW11 – QSPI Boot Mode

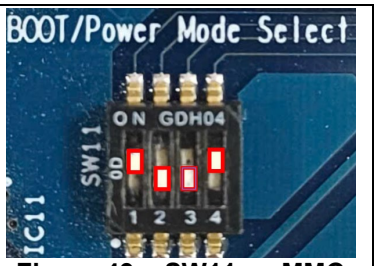


Figure 49. SW11 – eMMC Boot Mode

On the SOM module, SW1 selects eMMC or microSD boot mode. Please refer to the table below for the boot-mode options for each switch setting.

Table 32. SW1 – SOM module Switch mode

Boot device	ON	OFF
SW1-1	Normal Operation	JTAG debug mode
SW1-2	Select microSD slot on RTK9744L23C01000BE	OFF Select eMMC on RTK9744L23C01000BE

Table 33. RZ/G2L-EVK & RZ/V2L-EVK SW1 Switch Mode Position

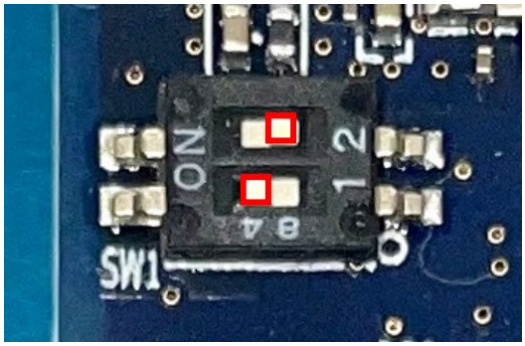


Figure 50. eMMC selection with normal operation

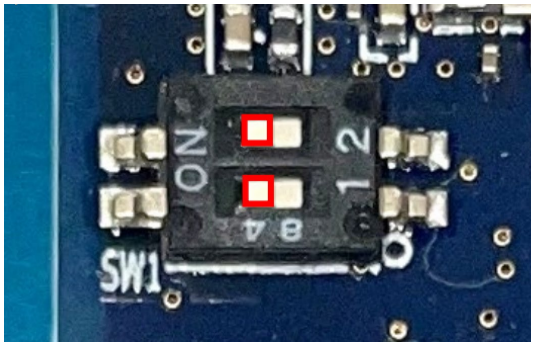


Figure 51. Enable the SD slot CN3 for normal operation

3.2.3 RZ/V2H-EVK

The RZ/V2H-EVK also provides on-board DIP switches for boot mode and boot device selection.

Use these settings after factory flashing to boot from the programmed device.

Table 34. DSW1 – Boot Device Selection (Normal Boot)

Boot device	DSW1-1	DSW1-2	DSW1-3	DSW1-4	DSW1-5	DSW1-6	DSW1-7	DSW1-8
eMMC	ON	OFF	ON	ON	ON	OFF	OFF	OFF
xSPI	ON	OFF	ON	OFF	OFF	OFF	OFF	OFF
SD / eSD	ON	OFF	ON	ON	OFF	OFF	OFF	OFF

Table 35. RZ/V2H-EVK DSW1 Switch Mode Position

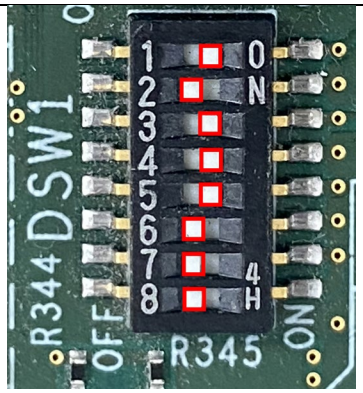


Figure 52. DSW1 - eMMC Boot Mode

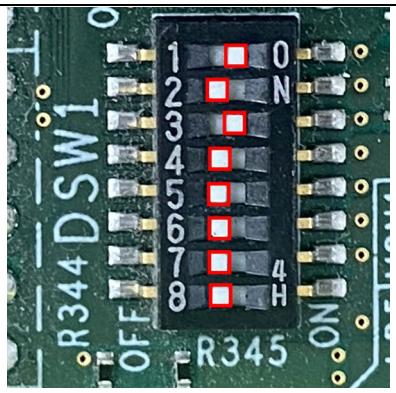


Figure 53. DSW1 – xSPI Boot Mode

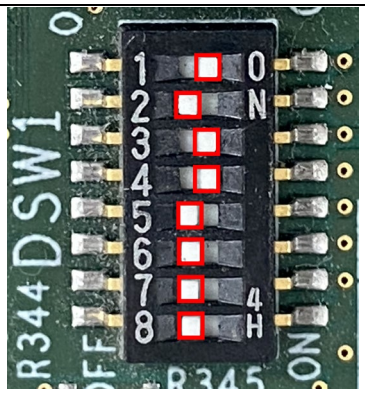


Figure 54. DSW1 – eSD Boot Mode

DSW2 controls output from the on-board clock generator (5P35023B) and one protected utility signal.

Table 36. DSW2 - Audio Clock / Utility DIP

Switch	Signal	OFF (default)	ON
1	Audio_CLKB_OE	Disables 5P35023B Audio_CLKB output	Enables Audio_CLKB output
2	Audio_CLKB	Audio_CLKB not supplied	Audio_CLKB is driven

3	Audio_CLKC_OE	Disables 5P35023B Audio_CLKC output	Enables Audio_CLKC output
4	Audio_CLKC	Audio_CLKC not supplied	Audio_CLKC is driven
5	NEN_VPROG	Must remain OFF	Prohibited — do not set ON
6	—	—	—

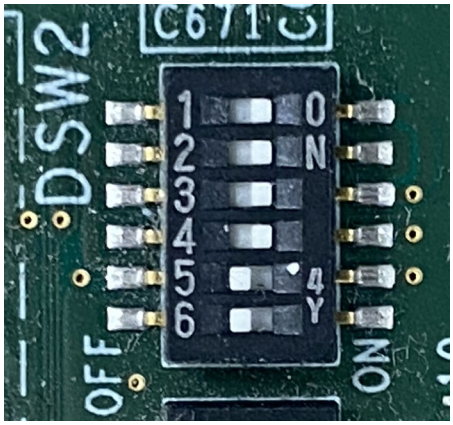


Figure 55. DSW2 - Audio Clock / Utility DIP

The table below lists the settings of the DIP switch (JSW1 on the RZ/V2H Secure Evaluation Board) and its functions.

Table 37. JSW1 Functions

Switch	Function
1-2	MIPI CSI-2 camera interface voltage: 1.8 V
2-3	MIPI CSI-2 camera interface voltage: 3.3 V (default)

Note: Set this switch according to the interface voltage of the camera module to be connected.

3.2.4 RZV2H-RDK

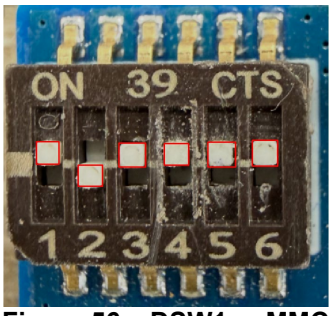
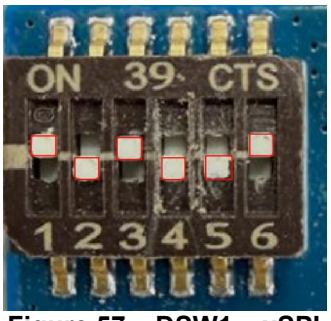
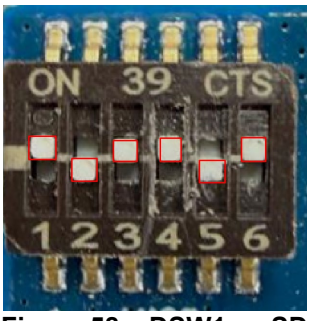
The RZ/V2H-RDK also provides on-board DIP switches for boot mode and boot device selection.

Use these settings after factory flashing to boot from the programmed device.

Table 38. DSW1 – Boot Device Selection (Normal Boot)

Boot device	DSW1-1	DSW1-2	DSW1-3	DSW1-4	DSW1-5	DSW1-6
eMMC	ON	OFF	ON	ON	ON	ON/OFF
xSPI	ON	OFF	ON	OFF	OFF	ON/OFF
SD / eSD	ON	OFF	ON	ON	OFF	ON/OFF

Table 39. RZ/V2H-RDK DSW1 Switch Mode Position

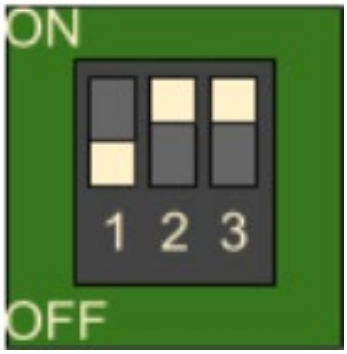
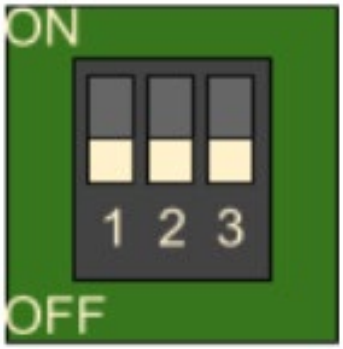
 **Figure 56. DSW1 - eMMC Boot Mode**	 **Figure 57. DSW1 – xSPI Boot Mode**	 **Figure 58. DSW1 – eSD Boot Mode**

3.2.5 IMDT V2H-SBC

The IMDT-V2H-SBC also provide on-board DIP switches for boot mode and boot device selection.

Use these settings after factory flashing to boot from the programmed device.

Table 40. DSW1 – Boot Device Selection (Normal Boot)

DSW1-Switch	Behavior
 **Figure 59. IMDT V2H-SBC xSPI Boot Mode Switch**	In this configuration, the bootROM attempts to load the bootloader from the xSPI Flash. If the Boot ROM fails to boot from the xSPI Flash, the board will enter SCIF Download mode. OFF, ON, ON
 **Figure 60. IMDT V2H-SBC xSPI Boot Mode Switch**	In this configuration, the bootROM attempts to load the bootloader from the eMMC. If the Boot ROM fails to boot from the eMMC Flash, the board will enter SCIF Download mode. OFF, OFF, OFF

3.3 Prepare the eMMC root filesystem

The onboard eMMC can be used as the main root filesystem, but it must first be initialized with a valid Linux rootfs. Since the eMMC is initially empty or unformatted, it cannot be used directly.

During preparation, the system must boot Linux using an SD card rootfs, while BL2/FIP are loaded from either QSPI or eMMC, depending on the boot mode. Once Linux is running, the onboard eMMC (e.g.,/dev/mmcblk0) becomes accessible and can be partitioned, formatted, and written with the rootfs.

Step 1. Prepare the rootfs archive

- Obtain the provided root filesystem archive (e.g., core-image-weston.tar.bz2). Obtain the provided root filesystem archive, which is included in the release images under:

```
rz-cmn-srp/target/images/rootfs
```

- Copy the archive to an SD card. This SD card will later be used as the source for writing the rootfs into the onboard eMMC.

Step2. Boot the board in QSPI/eMMC mode

- Set the DIP switch to select the boot source: QSPI or eMMC. Refer to section **3.2** for the correct switch positions for the target boards.
- Insert the prepared SD card (from Step 1) into the correct slot.

Table 41. SD card slot used for rootfs preparation

Board	SD card slot used for rootfs preparation	Notes
RZ/G2L-EVK	CN10 (Carrier microSD slot)	SW1-2 must be set to **ON (eMMC)**
RZ/V2L-EVK	CN10 (Carrier microSD slot)	SW1-2 must be set to **ON (eMMC)**
RZ/V2H-EVK	SoM microSD slot (on CPU board, underneath the module)	--
RS-G2L100	J1 (Carrier microSD slot)	-
IMDT V2H-SBC	J14 (Carrier microSD slot)	-

Note: Some early RZ/V2H-EVK revisions included two SD slots, and either slot could be used for rootfs preparation. The production version provides only the SoM slot underneath the CPU board. Always check the hardware manual for the board revision in use.

- Power on the board

Step 3. Identify the eMMC device

Determine which MMC index corresponds to the onboard eMMC. This can be checked in either U-Boot or Linux.

- In U-Boot: Run the following commands:
  1. Reboot or power on the board
  2. When the message “Hit any key to stop autoboot” appears, press a key to interrupt boot and enter the U-Boot console (=>).
  3. Run the following commands:

```
=> mmc rescan
=> mmc list
```

After rescanning, the list will show which device is the eMMC. The example below is for the RZ/G2L-EVK, where **mmc0** is identified as the eMMC device and **mmc1** as the SD card (CN10).

```
=> mmc list
sd@11c00000: 0
sd@11c10000: 1

=> mmc rescan
=> mmc list
sd@11c00000: 0 (eMMC)
sd@11c10000: 1
```

- Run *lsblk* to list all block devices. The numbering in Linux matches the U-Boot mapping: for example, if U-Boot shows mmc0 = eMMC, then in Linux the eMMC will appear as /dev/mmcblk0.

```
root@rz-cmn:~# lsblk
```

Example output:

```
mmcblk0      179:0    0 59.3G  0 disk
|-mmcblk0p1  179:1    0  100M  0 part
`-mmcblk0p2  179:2    0  2.5G  0 part
```

Step 4. Create the partition table

```
root@rz-cmn:~# fdisk /dev/mmcblk0
```

Inside fdisk, create the partition table as follows:

1. Press o → create a new DOS disklabel.
2. Press n, then p, then ENTER for defaults, then type +500M → creates Partition 1 (boot).
3. Press n, then p, then ENTER twice for defaults → creates Partition 2 (rootfs).
4. Press t, then select partition 1, then type b → set Partition 1 type to W95 FAT32.
5. Press w → write and exit.

Afterward, check the partition table with the command below:

```
root@rz-cmn:~# fdisk -l /dev/mmcblk0
```

Expected table:

```
root@rz-cmn:~# fdisk -l  /dev/mmcblk0
Disk /dev/mmcblk0: 59.28 GiB, 63652757504 bytes, 124321792 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: dos
Disk identifier: 0x076c4a2a

Device         Boot  Start     End Sectors  Size Id Type
/dev/mmcblk0p1 *        32  204831  204800  100M  c W95 FAT32 (LBA)
/dev/mmcblk0p2      204832 5376159 5171328  2.5G 83 Linux
```

Step 5. Format the partitions

Format Partition 1 as FAT32 (boot):

```
root@rz-cmn:~# mkfs.vfat -F 32 -n boot /dev/mmcblk0p1
```

Format Partition 2 as ext4 (rootfs)

```
root@rz-cmn:~# mkfs.ext4 -L rootfs /dev/mmcblk0p2
```

Step 6. Populate the rootfs and kernel Image

Once the partitions are formatted, the eMMC must be populated with the Linux root filesystem and the kernel image.

4. Mount the partitions

```
root@rz-cmn:~# mkdir -p /mnt/boot
root@rz-cmn:~# mkdir -p /mnt/rootfs

root@rz-cmn:~# mount /dev/mmcblk0p1 /mnt/boot
root@rz-cmn:~# mount /dev/mmcblk0p2 /mnt/rootfs
```

5. Extract the rootfs archive into the rootfs partition

```
root@rz-cmn:~# cd /mnt/rootfs
root@rz-cmn:~# tar xpf /home/root/core-image-weston.tar.bz2
root@rz-cmn:~# sync
```

6. Copy the kernel Image, DTBs and user environment (uEnv.txt) to the boot partition

```
root@rz-cmn:~# cp -rf /boot/* /mnt/boot/
```

7. Unmount the partitions

```
root@rz-cmn:~# umount /mnt/boot
root@rz-cmn:~# umount /mnt/rootfs
```

At this point, the eMMC contains:

- Partition 1 (boot): Kernel Image + DTBs + user environment (uEnv.txt)
- Partition 2 (rootfs): Full Linux root filesystem

The board can now boot using eMMC as the root filesystem.

3.4 Auto-managed U-boot Environment (imported each boot)

The following U-Boot variables are populated from the board identification file on every reboot. Therefore, any manual changes made with setenv, then saveenv, will be overwritten on the next boot.

Table 42. Auto-managed u-boot environment variables

Variable	Example
model_string	rzg2l-sbc
revision_major	1
revision_minor	0
mmcdev	0
mmcpart	1
image_addr	0x48080000
env_addr	0x48000000
dtb_addr	0x480C0000
dtbo_addr	0x48100000

3.5 How To Get the Console After Bootup

Once the RZ boards have booted, on the UART terminal, you will be able to log in using the default user ‘root’. There is no password. Leave the password field empty and just hit the return / enter key.

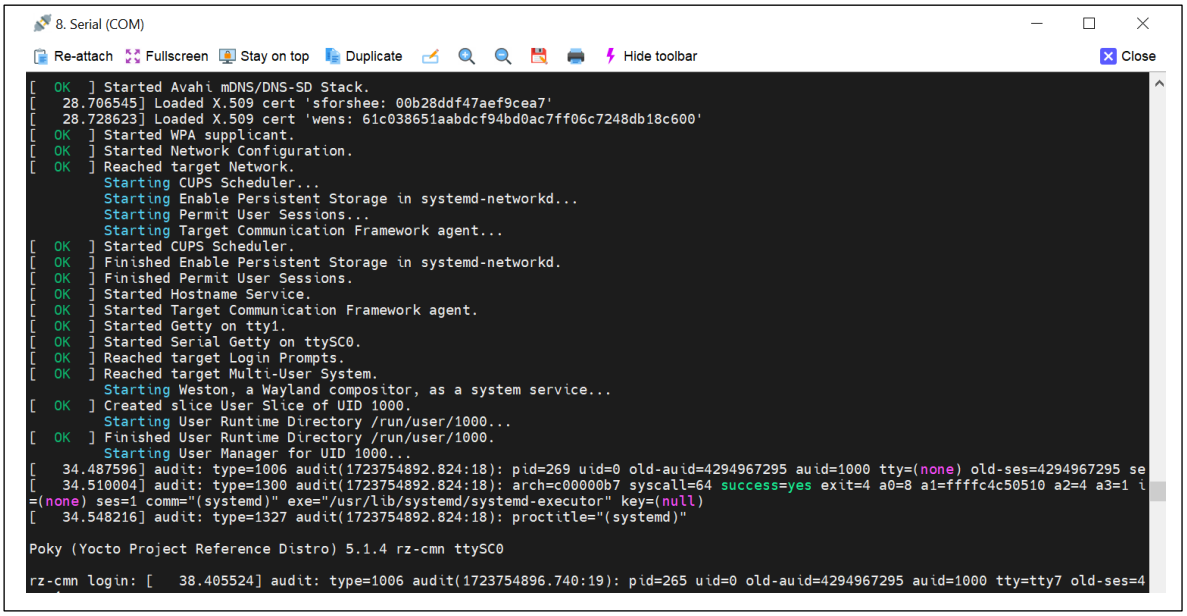


Figure 61. Root login of Linux console over UART 0.

Revision History

Rev.	Date	Description	
		Page	Summary
1.00	Mar.03.25	—	Initial release
1.10	Jun.04.25	—	Ubuntu release
2.00	Jul.02.25	—	Yocto Styhead release
3.00	Sep.25.25	—	Expanded release with support for multiple boards
3.10	Apr.17.26	—	Add partner boards to the current RZ common system
3.20	May.05.26	—	Add Real time kernel and eSD support
3.30	Sep.11.26	—	Add Multi-OS support

System Release Package, RZ Series – Quick Start Guide

Publication Date: Sep.11.26

Published by: Renesas Electronics Corporation

RZ Family/ RZ G/V Series

R11QS0072EU0330